

TRABAJO FIN DE GRADO INGENIERÍA Y SISTEMAS DE DATOS

TÍTULO: Sistema inteligente para la identificación de problemas de salud mental juvenil en redes sociales

AUTOR/A: Emma Nájera Ríos

TUTOR/A: María Luisa Martín Ruíz

DEPARTAMENTO: Ingeniería Telemática y Electrónica

VºBº TUTOR/A

Miembros del Tribunal Calificador:

PRESIDENTE/A: Daniel de la Prida

TUTOR/A: María Luisa Martín Ruíz

SECRETARIO/A: Mario Vega Barbas

Fecha de lectura: 29 de Septiembre de 2025

Calificación:

El Secretario/La Secretaria,

“Sufrimos más en la imaginación que en la realidad.”

- Séneca

AGRADECIMIENTOS

Muchas gracias a mi familia por haber incentivado siempre mi curiosidad de ingeniera, a Celia y a Clara por haber sido testigo de todas mis facetas y evolución personal, a Mati por haber sido mi primer profe de programación y seguir enseñándome tanto en otros aspectos importantes de la vida, a Carol por el apoyo y la compañía en momentos buenos y malos, Ester y Rai y el resto de haditas por hacer más divertida la uni, también a Fran y Ce por ser un pedacito de Berlín que volvió conmigo a Madrid y a Umbi por escucharme, aconsejarme y ser una enorme inspiración en mi vida sin importar la distancia.

Agradezco profundamente a mi tutora por su apoyo y disposición, así como por haberme ayudado a resolver todas mis dudas a lo largo del proceso.

RESUMEN

Este proyecto consiste en el desarrollo de una aplicación web capaz de analizar publicaciones de la red social Tumblr con el objetivo de detectar posibles signos de depresión en el texto, basándose en los criterios clínicos establecidos por el Manual Diagnóstico y Estadístico de los Trastornos Mentales (DSM-V). La solución combina un modelo de aprendizaje automático basado en arquitecturas transformer con una interfaz visual accesible que permite al usuario interactuar con los resultados del análisis.

El proyecto incluye la construcción de un dataset a partir de publicaciones extraídas de Reddit, ya que no existen datasets públicos etiquetados específicamente por síntomas de la depresión. Este etiquetado se ha llevado a cabo mediante un proceso semiautomático basado en la búsqueda de palabras clave por síntoma, completando cada uno de ellos con modelos de regresión logística independientes. El dataset resultante, se utiliza para entrenar un modelo multietiqueta, basado en DistilBERT, capaz de predecir la presencia de uno o varios síntomas en nuevos textos.

Basándose en este modelo, se ha desarrollado la aplicación de visualización de las predicciones. En esta, el usuario puede buscar publicaciones de Tumblr por etiqueta o cargar sus propios textos. El sistema analiza los textos, muestra un resumen estadístico de los síntomas detectados y permite acceder a cada publicación individual con una visualización que muestra la influencia de cada palabra en la predicción. Así, la aplicación sirve como herramienta para la identificación de patrones lingüísticos relacionados con la depresión.

En el desarrollo del proyecto se prioriza la interpretabilidad del modelo, tanto para facilitar la comprensión de los resultados como para alinearse con los principios éticos en el uso de modelos de inteligencia artificial aplicados a la salud mental.

Este proyecto no pretende sustituir un diagnóstico profesional, pero puede actuar como apoyo exploratorio en entornos de investigación psicológica. El proyecto demuestra la viabilidad de aplicar modelos multietiqueta de lenguaje natural al análisis de contenido digital emocionalmente significativo, respetando los principios de interpretabilidad y privacidad.

ABSTRACT

This project presents the development of a web application designed to analyze Tumblr publicaciones and detect potential signs of depression in the text, following the clinical criteria established by the Diagnostic and Statistical Manual of Mental Disorders (DSM-V). The solution combines a machine learning model based on transformer architectures with a user-friendly visual interface that allows the user to interactively explore the predictions.

The project includes the construction of a dataset from publicaciones extracted from Reddit, as there are no public datasets available with symptom-level labels for depression. The labelling process has been carried out semi-automatically, using keyword search by symptom, completing each of them with independent classical models (logistic regression with pre-trained embeddings). The resulting dataset is used to train a multi-label model, based on DistilBERT, capable of predicting the presence of one or several symptoms in previously unseen texts.

Next, an application was developed to visualize the predictions generated by the mode. Users can either retrieve Tumblr publicaciones by etiqueta or upload their own textual data. This input is then used to perform the text analysis, providing a statistical summary of the detected symptoms. Additionally, for each individual post, the application serves as a tool for the identification of linguistic patterns related to depression.

In the development of the project, the interpretability of the model is prioritised, both to facilitate the understanding of the results and to align with the ethical principles in the use of artificial intelligence models applied to mental health.

While it is not intended to replace professional diagnosis, it could serve as an exploratory aid in psychology research. This project demonstrates the feasibility of applying interpretable, multi-label NLP models to emotionally significant online content while respecting principles of transparency and privacy.

ÍNDICE

AGRADECIMIENTOS	v
RESUMEN	vii
ABSTRACT	ix
ÍNDICE DE TABLAS	xv
ÍNDICE DE FIGURAS	xxiii
1. Introducción	1
1.1. Motivación	1
1.2. Objetivos	1
1.3. Limitaciones	2
1.4. Estructura de la memoria	2
2. Marco tecnológico	3
2.1. La IA y el Aprendizaje automático	3
2.2. El Aprendizaje Supervisado como paradigma empleado en esta solución	5
2.2.1. Regresión Lineal	7
2.2.2. Regresión Logística	8
2.2.3. Máquinas de Soporte Vectorial	9
2.2.4. Clasificación multi-etiqueta	10
2.2.5. Procesamiento de Lenguaje Natural	12
2.3. Deep Learning	14
2.3.1. Nodo de una Red Neuronal	15
2.3.2. Principales modelos de aprendizaje profundo	16
2.3.3. Modelos Transformer	21
2.3.3.1. BERT y DistilBERT	25
2.3.4. Entrenamiento, Validación y Evaluación de Modelos de Deep Learning . .	28

2.3.4.1.	Entrenamiento	28
2.3.4.2.	Overfitting	31
2.3.4.3.	Validación	33
2.3.4.4.	Evaluación	34
2.3.5.	Explicabilidad	39
2.4.	Tecnologías web	42
2.4.1.	Arquitectura cliente-servidor	42
2.4.2.	Frontend	42
2.4.3.	Backend	42
2.4.4.	APIs RESTful	42
2.5.	Tumblr	43
2.5.1.	API de Tumblr	43
2.6.	Reddit	43
2.6.1.	API de Reddit	43
3.	Aspectos socio-sanitarios relevantes para el proyecto	45
3.1.	IA en Salud Mental	45
3.2.	Modelos Clásicos vs. Deep Learning	45
3.3.	Modelos basados en Transformer	45
3.4.	Uso del DSM-V para etiquetado clínico	46
4.	Descripción de la solución propuesta	47
4.1.	Modelo de predicción	47
4.1.1.	Definición del problema	47
4.2.	Construcción del conjunto de datos de Reddit y Tumblr	48
4.2.1.	Conjuntos de datos utilizados	50
4.2.1.1.	Pasos previos al entrenamiento	54
4.2.1.2.	Arquitectura del modelo	55
4.2.1.3.	Entrenamiento	57
4.3.	Desarrollo de la aplicación web	61

4.3.1. Arquitectura del diseño	61
4.3.2. Backend	61
4.3.2.1. Arquitectura modular	62
4.3.2.2. Flujo funcional	62
4.3.3. Frontend	64
4.3.3.1. Arquitectura modular	64
4.3.3.2. Flujo funcional	65
5. Resultados	67
5.1. Evaluación del modelo	67
5.1.1. Evaluación de predicción de Síntomas con dataset de Reddit	67
5.1.2. Evaluación con publicaciones recogidas de Tumblr	69
5.1.3. Evaluación de la predicción de la etiqueta de depresión	72
5.2. Evaluación de la aplicación web	75
5.2.1. Prueba práctica del sistema	77
6. Impacto del proyecto	81
7. Presupuesto	83
7.1. Resumen de Gastos Totales	84
8. Conclusiones y líneas futuras	85
8.1. Conclusiones	85
8.2. Líneas futuras	86
BIBLIOGRAFÍA	89

ÍNDICE DE TABLAS

2.1. Tokenización y asignación de IDs según el vocabulario de BERT (modelo Transformer). Los IDs corresponden a un índice fijo dentro del vocabulario con el que se preentrenó el modelo. En caso de que una palabra no exista en dicho vocabulario, esta se descompone en subpalabras conocidas o, si no es posible, se reemplaza por el token especial [UNK] (unknown) durante la tokenización.	14
2.2. Resumen comparativo de las principales arquitecturas de aprendizaje profundo y sus aplicaciones más comunes.	20
2.3. Comparación entre BERT-base y DistilBERT en términos de arquitectura, entrenamiento y eficiencia. DistilBERT logra una reducción significativa en tamaño y tiempo de inferencia, manteniendo un rendimiento cercano al de BERT.	27
4.1. Comparación de métricas de clasificación entre regresión logística y SVM para el primer síntoma. Los valores de precisión, recall y F1-score por clase (0 = sin síntoma, 1 = con síntoma), así como las métricas globales de exactitud y promedios, muestran diferencias mínimas (menores al 1 %) entre ambos modelos.	49
4.2. Matriz de confusión del modelo de regresión logística. Cada celda indica el número de publicaciones clasificadas en cada categoría. La diagonal (383 y 336) corresponde a aciertos: textos sin el síntoma identificados correctamente como “0” y textos con el síntoma como “1”. Los valores fuera de la diagonal (17 y 9) son errores: textos mal clasificados. Por lo tanto, el modelo logró clasificar correctamente la gran mayoría de los casos.	49
4.3. Matriz de confusión del modelo SVM. Al igual que en la regresión logística, la diagonal (385 y 339) indica aciertos y los valores fuera de ella (15 y 6) representan errores. El SVM cometió ligeramente menos errores que la regresión logística, aunque con un coste de entrenamiento mayor.	50
4.4. Métricas para entrenamiento y validación por época con dropout de 0.3 (se destacan los mejores valores para cada métrica)	59
4.5. Métricas para entrenamiento y validación por época con weight decay de 0.01	59
4.6. Resultados de entrenamiento y validación por época con un lote de tamaño 32 y una tasa de aprendizaje de de 3e-5.	59
5.1. Resultados de entrenamiento y validación para la época 4	67
5.2. Estructura del archivo CSV generado con las publicaciones y sus predicciones por síntoma.	78
7.1. Coste de Recursos de Software	83
7.2. Coste de Recursos de Hardware	83
7.3. Coste de Recursos Humanos	84
7.4. Resumen de Gastos Totales	84

ÍNDICE DE FIGURAS

2.1. Diagrama de los principales tipos de Aprendizaje Automático, indicando el tipo de datos utilizados, el enfoque general y el conocimiento que se obtiene del modelo. [9]	3
2.2. Ejemplo de un modelo de clasificación SVM que ha aprendido la superficie de decisión óptima para separar las clases verde y roja. [9]	4
2.3. Ejemplo de reducción de la dimensionalidad mediante PCA. Proyecta los datos sin etiquetas en un nuevo espacio de componentes principales, revelando patrones y agrupaciones ocultas que no eran evidentes en las variables originales. [11]	4
2.4. Ejemplo de aprendizaje semi-supervisado mediante propagación de etiquetas. A partir de unos pocos nodos etiquetados y la estructura del grafo, el modelo infiere las etiquetas de los nodos no anotados. [13]	5
2.5. Clasificación vs. regresión. En clasificación, la línea discontinua representa una frontera que separa las dos clases; en regresión, modela la relación lineal entre las dos variables. [15]	5
2.6. La línea azul representa el modelo de regresión, los puntos rojos son los datos y la distancia entre ambos es el error de predicción. [18]	7
2.7. La curva en forma de S modela la probabilidad de que la variable dependiente y tome el valor 1 en función de la variable independiente x , asegurando que las predicciones se mantengan en el rango $[0,1]$. [20]	8
2.8. Representación gráfica de un clasificador SVM. El hiperplano óptimo maximiza el margen entre las clases, definido por los vectores de soporte (puntos más cercanos al límite de decisión). [22]	9
2.9. Diferencia entre clasificación multiclase y multietiqueta. En la multiclase cada muestra pertenece a una única categoría, mientras que en la multietiqueta puede estar asociada simultáneamente a varias categorías. [24]	10
2.10. Ejemplos de un problema de clasificación multietiqueta: cada instancia puede asociarse con uno o varios elementos del conjunto de etiquetas L .	10
2.11. Representación del enfoque BR: cada clasificador aprende de forma independiente si una etiqueta (sol, viento, nubes, lluvia o nieve) está presente en cada ejemplo.	11
2.12. Esquema del método CC: los clasificadores se enlazan, de modo que la predicción de cada uno alimenta a los siguientes, incorporando dependencias entre etiquetas	11
2.13. Conversión de conjuntos de etiquetas a categorías únicas mediante el enfoque LP: cada combinación de condiciones climáticas se asigna a una categoría única.	12
2.14. Estructura de una red neuronal profunda con dos capas ocultas [30]	14
2.15. Esquema de una neurona artificial: los valores de entrada $(x_0, x_1, \dots, x_n)$ se combinan con sus respectivos pesos (w) y un sesgo (b) . Luego, una función de suma calcula el valor agregado, que pasa por una función de activación para generar la salida (y) . [33]	16

2.16. Arquitectura de un MLP: la información fluye desde la capa de entrada, pasando por una o varias capas ocultas, hasta la capa de salida, donde se generan las predicciones. [34]	16
2.17. Esquema de una CNN: las capas de convolución y pooling extraen características de la imagen, que posteriormente son procesadas por capas totalmente conectadas para realizar la clasificación final. [35]	17
2.18. Comparación entre una red neuronal feedforward, donde la información fluye en una sola dirección, y una RNN, que incorpora conexiones hacia atrás para modelar dependencias temporales en secuencias. [36]	18
2.19. Comparación de arquitecturas de redes neuronales recurrentes: RNN simple con retroalimentación básica; LSTM con compuertas de entrada, olvido y salida para manejar dependencias a largo plazo; y GRU, una variante más ligera que combina compuertas de actualización y reinicio. [37]	18
2.20. Arquitectura de un AE. El <i>encoder</i> transforma los datos de entrada en una representación comprimida o espacio latente (<i>latent features</i>), mientras que el <i>decoder</i> utiliza dicha representación para reconstruir la salida, buscando minimizar la diferencia entre entrada y salida. Este proceso permite el aprendizaje no supervisado de representaciones compactas. [38]	19
2.21. Estructura de un VAE. El <i>encoder</i> transforma la entrada en distribuciones latentes caracterizadas por una media (μ) y una varianza (σ). A partir de estas distribuciones se muestrean variables latentes (z), que posteriormente son utilizadas por el <i>decoder</i> para reconstruir la entrada original. Este enfoque permite un aprendizaje probabilístico de representaciones latentes continuas. [39]	19
2.22. Esquema de una GAN. El generador crea datos sintéticos a partir de ruido aleatorio, mientras que el discriminador evalúa si los datos corresponden a muestras reales o falsas. Ambos modelos se entrenan de forma competitiva hasta que el generador produce datos cada vez más realistas. [40]	20
2.23. Arquitectura general del modelo Transformer con estructura encoder-decoder. A la izquierda se representa el encoder , formado por N bloques que incluyen atención multi-cabeza y redes feed-forward, cada uno con normalización y conexiones residuales. A la derecha, el decoder incluye atención enmascarada, atención cruzada con las salidas del encoder y componentes similares. Las codificaciones posicionales se suman a los embeddings de entrada para preservar el orden secuencial. [43]	21
2.24. Arquitectura básica de un Transformer: la secuencia de entrada se transforma mediante embeddings y codificación posicional, pasa por capas de self-attention en el encoder y se entrega al decoder, donde la atención combinada (self y encoder-decoder attention) genera la secuencia de salida. [45]	22
2.25. Esquema del mecanismo de autoatención en transformadores: los vectores de consulta (Q) y clave (K) se multiplican para generar un mapa de atención, donde cada celda representa la relación entre un token y los demás en la secuencia. [42]	23

- 2.26. Estructura apilada de capas de **encoder** en el modelo Transformer. Cada capa contiene una subcapa de self-attention seguida de una red feed-forward, ambas con normalización y conexiones residuales. La entrada incluye embeddings sumados a codificaciones posicionales. El apilamiento de bloques permite construir representaciones jerárquicas cada vez más abstractas de la secuencia de entrada. [46] . . . 23
- 2.27. Estructura interna de un bloque de **decoder** en la arquitectura Transformer. Cada bloque incluye: una capa de **self-attention enmascarada**, una capa de **atención encoder-decoder** (cross-attention), una red feed-forward, conexiones residuales y normalización de capa. Esta estructura permite generar secuencias autoregresivas basadas tanto en la salida previa del decoder como en las representaciones del encoder. [46] 24
- 2.28. Comparación entre atención unidireccional y bidireccional. Mientras que los modelos tradicionales de lenguaje utilizan atención unidireccional (izquierda), BERT se basa en atención bidireccional (derecha), permitiendo que cada token incorpore información tanto del pasado como del futuro en la secuencia. Esto mejora considerablemente la calidad de las representaciones contextuales. [48] 25
- 2.29. Tareas de preentrenamiento de BERT: **Masked Language Modeling (izquierda)** consiste en predecir palabras ocultas a partir del contexto circundante. **Next Sentence Prediction (derecha)** busca determinar si una oración sigue lógicamente a otra, ayudando a capturar relaciones discursivas. Ambas tareas permiten a BERT adquirir un entendimiento profundo del lenguaje antes del ajuste fino. [49] 26
- 2.30. Esquema general del proceso de preentrenamiento y fine-tuning en BERT. Durante el preentrenamiento, el modelo aprende con tareas generales (MLM y NSP) sobre datos no etiquetados. Posteriormente, se adapta a tareas específicas (como clasificación, NER o QA) mediante fine-tuning sobre datos anotados. [50] 26
- 2.31. Proceso de distilación en DistilBERT. El modelo estudiante (derecha) aprende a imitar el comportamiento del modelo maestro BERT (izquierda), reduciendo el número de capas y eliminando la tarea NSP. Esto resulta en un modelo más ligero y eficiente, manteniendo un rendimiento competitivo. [52] 27
- 2.32. Efecto de diferentes tasas de aprendizaje en el descenso de gradiente: una tasa demasiado baja requiere muchas actualizaciones para converger, la tasa óptima alcanza el mínimo eficientemente, y una tasa demasiado alta provoca oscilaciones o divergencia del entrenamiento. [54] 28
- 2.33. Representación del proceso de entrenamiento en redes neuronales: los datos de entrenamiento se dividen en lotes, cada lote corresponde a una iteración, y un ciclo completo sobre todos los lotes constituye una época. [55] 28
- 2.34. Trayectorias de optimización hacia el mínimo en el espacio de parámetros: Batch Gradient Descent (azul) sigue un camino estable y directo, Mini-batch Gradient Descent (verde) combina estabilidad con algo de variabilidad, y Stochastic Gradient Descent (morado) muestra un recorrido más errático debido a su alta varianza en cada actualización. [57] 30

2.35. Ilustración de tres escenarios de ajuste de un modelo: sobreajuste (overfitting), donde el modelo se adapta excesivamente al ruido de los datos; ajuste óptimo, que capta correctamente la tendencia subyacente; y subajuste (underfitting), donde el modelo es demasiado simple para representar la relación entre variables. [58] . . .	31
2.36. Comparación de dos estrategias de validación de modelos: en Hold-out se divide el conjunto de datos en entrenamiento y validación una sola vez; en K-fold Cross Validation los datos se particionan en K subconjuntos, rotando el subconjunto de validación en cada iteración. [60]	33
2.37. Esquema de validación cruzada anidada (Nested Cross-Validation): el conjunto total de datos se divide en subconjuntos (folds). En cada división, un fold actúa como validación mientras los demás se usan para entrenar, repitiendo el proceso en cada iteración para ajustar parámetros. Finalmente, el rendimiento del modelo se evalúa con un conjunto de prueba independiente. [62]	33
2.38. Comparación de estrategias de validación temporal: en Sliding Window el tamaño de la ventana de entrenamiento se mantiene constante desplazándose en el tiempo, mientras que en Expanding Window la ventana de entrenamiento crece incorporando nuevos datos históricos en cada iteración, antes de realizar la predicción. [64]	34
2.39. Matriz de confusión para la evaluación de modelos de clasificación binaria. Se muestran las categorías de verdaderos positivos (TP), falsos positivos (FP), falsos negativos (FN) y verdaderos negativos (TN), junto con las fórmulas de métricas clave: precisión, exhaustividad (recall) y exactitud (accuracy). [67]	37
2.40. Curva ROC (Receiver Operating Characteristic) para evaluar el rendimiento de un clasificador. El eje X representa la tasa de falsos positivos (False Positive Rate) y el eje Y la tasa de verdaderos positivos (True Positive Rate). La línea azul muestra el clasificador ideal, la línea gris indica el comportamiento de un clasificador aleatorio, y la curva roja corresponde al clasificador práctico evaluado. [68]	38
2.41. Diagrama de fiabilidad utilizado para evaluar la calibración de un modelo de clasificación. La línea discontinua representa la calibración perfecta, mientras que la línea negra muestra la calibración real del modelo. Las áreas sombreadas indican zonas de subconfianza (underconfidence) y sobreconfianza (overconfidence) según la discrepancia entre la probabilidad predicha y la precisión observada. [69] . . .	38
2.42. Ilustración conceptual del método LIME. El modelo global, complejo y no lineal, se aproxima localmente mediante un modelo lineal interpretable en torno a una instancia concreta. De esta manera, se obtiene una explicación sencilla y comprensible de cómo el modelo toma decisiones para un caso específico. [71]	39
2.43. Gráfico de resumen SHAP que muestra la contribución de cada variable al resultado del modelo. El eje horizontal representa el impacto de cada característica sobre la predicción, mientras que el color indica el valor de la variable (azul = bajo, rojo = alto). Las variables más influyentes aparecen en la parte superior, permitiendo identificar de manera visual qué factores condicionan más las decisiones del modelo. [74]	40

2.44. Visualización de mapas de atención generados por un modelo basado en transformers: (a) imagen original, (b) mapa de atención que resalta las regiones de la imagen más relevantes para la predicción, y (c) mapa de explicación visual que muestra explícitamente la zona de interés identificada por el modelo. [75]	40
2.45. Ejemplo de interpretabilidad local mediante el método Integrated Gradients, implementado en Captum. Se representan las frases con etiquetas reales y predichas junto con la puntuación de atribución de cada palabra. Los términos que contribuyen positivamente a la predicción se resaltan en verde, mientras que los que influyen negativamente aparecen en rojo, lo que permite comprender qué fragmentos del texto guían la decisión del modelo. [76]	41
2.46. Ejemplo de búsqueda por etiqueta en Tumblr	43
2.47. Ejemplo de publicación en Reddit dentro del subreddit <code>r/geography</code> , mostrando una discusión sobre ciudades europeas.	44
4.1. Esquema general de la solución propuesta. Se muestran las fuentes de datos empleadas para el entrenamiento, validación y evaluación del modelo de clasificación, así como su integración en la aplicación web.	47
4.2. Flujo de recolección y etiquetado de publicaciones en Reddit. Se seleccionan publicaciones con síntomas en <code>r/depression</code> y textos sin síntomas en otros subreddits para entrenar modelos de regresión logística específicos por síntoma.	48
4.3. Ilustración del proceso de generación de subconjuntos balanceados para el síntoma 4.	49
4.4. Integración del proceso de recolección y modelado. Cada modelo de regresión logística entrenado por síntoma asigna etiquetas al conjunto total de publicaciones, produciendo un dataset final con anotaciones para todos los síntomas.	50
4.5. Visualización de las estadísticas del conjunto <code>train.csv</code> , que contiene las publicaciones recopiladas de Reddit. Se muestran algunas columnas: el identificador (<code>#</code>), <code>texto</code> , que incluye el contenido de la publicación; <code>sintoma_1</code> , correspondiente al síntoma "estado de ánimo deprimido"; y <code>probabilidad_1</code> , que indica la probabilidad asociada a la categoría 1 para este síntoma.	51
4.6. Histograma de las variables <code>probabilidad_1</code> a <code>probabilidad_10</code> . Cada gráfico incluye una línea roja punteada que indica el umbral de decisión utilizado para clasificar una observación como positiva.	52
4.7. Visualización de las estadísticas del conjunto <code>predicciones_completo.csv</code> , que contiene las publicaciones recopiladas de Tumblr. Se muestran las columnas <code>texto</code> (contenido de la publicación), <code>sintoma_1</code> (estado de ánimo deprimido) y <code>sintoma_2</code> (disminución del interés).	53
4.8. Visualización de las estadísticas del conjunto de datos <i>Depression: Reddit Dataset (Cleaned)</i> , almacenado en <code>depression_dataset_reddit_cleaned.csv</code> . Se observan las columnas <code>clean_text</code> y <code>is_depression</code>	54

4.9. Arquitectura general de la aplicación web. La pantalla de búsqueda, el dashboard y la visualización de las publicaciones se muestran en el frontend. El backend se encarga del procesamiento de los datos introducidos por el usuario, de la adquisición de información mediante la API de Tumblr y de la clasificación de los resultados utilizando el modelo distilBERT.	61
4.10. Arquitectura modular del backend con sus componentes y dependencias.	62
4.11. Flujo funcional para la extracción y clasificación de publicaciones en <code>/api/fetch-publicaciones</code>	63
4.12. Flujo funcional para la generación de mapas de saliencia en <code>/api/get-importance</code>	64
4.13. Arquitectura modular del frontend y sus dependencias.	65
4.14. Diagrama de flujo funcional del frontend.	66
5.1. F1 Micro, F1 Macro y Hamming Loss en entrenamiento y validación para cada época.	67
5.2. Distribución de Precision, Recall y F1-score en los distintos síntomas.	69
5.3. Ejemplos comparativos de publicaciones sobre salud mental en Reddit y Tumblr. En Reddit, el texto se presenta de forma estructurada y explícita, con una narrativa clara y una solicitud directa de ayuda. En contraste, la publicación de Tumblr emplea un estilo más fragmentado y emocional, utilizando eufemismos ("suic*de") y emoticones para expresar malestar, lo que refleja el uso frecuente de jerga juvenil en esta plataforma.	70
5.4. Distribución de etiquetas para un síntoma específico en los conjuntos de Reddit y Tumblr. Se observa un desequilibrio más marcado en el conjunto de Tumblr, con una menor proporción de ejemplos positivos. Esta diferencia en la distribución puede afectar la capacidad del modelo para generalizar correctamente, especialmente al enfrentarse a clases subrepresentadas durante la inferencia.	71
5.5. Curvas ROC por clase para el conjunto de publicaciones de Tumblr.	72
5.6. Curva ROC con la aproximación binaria basada en ≥ 5 síntomas.	73
5.7. Curva ROC del método de suma ponderada de probabilidades ($AUC = 0.94$).	74
5.8. Curva ROC del método de escalado y truncamiento de probabilidades ($AUC = 0.90$).	74
5.9. Interfaz del sistema antes de una búsqueda con la etiqueta <code>#mentally ill</code>	77
5.10. Barra de progreso durante la recolección de publicaciones.	77
5.11. Barra de progreso durante la predicción de síntomas.	77
5.12. Resultados mostrados en consola.	78
5.13. Panel de visualización con resultados y gráficos de análisis.	78
5.14. Distribución del nivel de depresión detectado.	79

5.15. Frecuencia de aparición de cada síntoma.	79
5.16. Correlación entre síntomas detectados.	79
5.17. Publicación con predicción positiva para el síntoma "dificultad para concentrarse".	79
5.18. Publicación con predicción positiva para los síntomas "estado de ánimo deprimido", "anhedonia", erróneamente, "fatiga".	79
5.19. Activación de múltiples síntomas a partir de la presencia de la palabra "depresión".	80

LISTA DE ACRÓNIMOS

AE	Autoencoder
API	Application Programming Interface(s)
AP	Average Precision
AUC	Area Under the Curve
BART	Bidirectional and Auto-Regressive Transformers
BERT	Bidirectional Encoder Representations from Transformers
BiLSTM	Bidirectional Long Short-Term Memory
BR	Binary Relevance
CC	Classifier Chains
CLS	Classification token
CNN	Convolutional Neural Network (Red Neuronal Convolutacional)
DBSCAN	Density-Based Spatial Clustering of Applications with Noise
DistilBERT	Versión ligera de BERT mediante Knowledge Distillation
DL	Deep Learning (Aprendizaje Profundo)
DSM-5	Diagnostic and Statistical Manual of Mental Disorders, Fifth Edition
EMR	Exact Match Ratio
FCN	Feed-Forward Network
FP	False Positive (Falso Positivo)
FN	False Negative (Falso Negativo)
F1	F1-score, métrica combinando precisión y recall
GAN	Generative Adversarial Network
GDPR	General Data Protection Regulation
GELU	Gaussian Error Linear Unit
GPT	Generative Pre-trained Transformer
GRU	Gated Recurrent Unit
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
IA	Inteligencia Artificial
JSON	JavaScript Object Notation
KL Divergence	Kullback-Leibler Divergence
L1	Norma L1 (Least Absolute Deviations)
L2	Norma L2 (Least Squares)
Label Ranking Loss (LRL)	Porcentaje de pares de etiquetas mal ordenadas según probabilidad
LGBM	Light Gradient Boosting Machine
LIME	Local Interpretable Model-agnostic Explanations
LR	Learning Rate (tasa de aprendizaje)
LSTM	Long Short-Term Memory
MAE	Mean Absolute Error (Error Absoluto Medio)
MAPE	Mean Absolute Percentage Error (Error Porcentual Absoluto Medio)
MedAE	Median Absolute Error (Error Absoluto Mediano)
ML	Machine Learning (Aprendizaje Automático)
MLM	Masked Language Modeling
MLP	Multilayer Perceptron (Perceptrón Multicapa)
NAG	Nesterov Accelerated Gradient
NER	Named Entity Recognition (Reconocimiento de Entidades Nombradas)
NSP	Next Sentence Prediction
NLP	Natural Language Processing (Procesamiento de Lenguaje Natural)

OAuth	Open Authorization
ODS	Objetivos de Desarrollo Sostenible
PCA	Principal Component Analysis (Análisis de Componentes Principales)
PR	Precision–Recall
QA	Question Answering (Respuesta a Preguntas)
Q, K, V	Query, Key, Value — vectores fundamentales en el mecanismo de atención
RBF	Radial Basis Function (Función de Base Radial)
RFDTBR	Random Forests based on Transformations
RFPCT	Random Forest Predictive Clustering Trees
ReLU	Rectified Linear Unit
REST	Representational State Transfer
RMSE	Root Mean Squared Error (Raíz del Error Cuadrático Medio)
ROC	Receiver Operating Characteristic
ROC AUC	Receiver Operating Characteristic – Area Under Curve
RNN	Recurrent Neural Network (Red Neuronal Recurrente)
SVM	Support Vector Machine (Máquina de Vectores de Soporte)
SHAP	SHapley Additive exPlanations
SGD	Stochastic Gradient Descent (Descenso de Gradiente Estocástico)
TF-IDF	Term Frequency–Inverse Document Frequency
TP	True Positive (Verdadero Positivo)
TN	True Negative (Verdadero Negativo)
URL	Uniform Resource Locator
VAE	Variational Autoencoder
T5	Text-to-Text Transfer Transformer

1. Introducción

1.1. Motivación

Gracias a los dispositivos móviles, contamos con una amplia accesibilidad a internet, que puede conllevar un uso indebido. Este resulta especialmente preocupante entre los jóvenes, ya que son más vulnerables a hacer un uso excesivo, repercutiendo de manera negativa en su salud mental y física. Varios estudios [1] demuestran que los adolescentes que abusan de las redes sociales muestran niveles de ansiedad más elevados, en contraste con aquellos que las emplean de manera responsable. Asimismo, se vincula con alteraciones en los hábitos de sueño, lo que repercute negativamente en el desarrollo cognitivo, rendimiento académico y bienestar social, mental y físico [2].

Algunas redes sociales han evolucionado más allá de ser meros espacios de interacción, convirtiéndose en plataformas de expresión personal, brindando la posibilidad de crear una identidad digital. Así, los adolescentes se sienten cómodos para expresar sus sentimientos y problemas como forma de desahogo y búsqueda de apoyo [3].

Tumblr es una plataforma de microblogging que permite a sus usuarios publicar textos, imágenes, vídeos, enlaces, citas y audio a manera de tumblelog [4]. Esta red social destaca como un espacio donde los usuarios (sobre todo adolescentes) pueden compartir sus experiencias anónimamente, formando comunidades de apoyo, sobre todo en torno a la salud mental. Sin embargo, también pueden reforzar comportamientos dañinos, como la autolesión o los trastornos alimenticios, al generar un efecto de cámara de eco que normaliza e incluso glorifica estas problemáticas.

Estas llamadas “cámaras de eco” son comunidades online, cuyos miembros retroalimentan su negatividad y refuerzan pensamientos tóxicos, ya que están expuestos de manera continua a publicaciones de ese tipo. Del apoyo y la reducción del estigma surge la estetización e idealización de las enfermedades mentales, que pasan a verse como algo deseable, misterioso y especial, lo cual hace que los adolescentes adopten una visión pesimista y tiendan al autodiagnóstico. El 30 % de la Generación Z se ha autodiagnosticado algún trastorno mental, sobre todo la ansiedad (48 %) y depresión (37 %) [5].

Para analizar este fenómeno, el proyecto propone una aplicación, que, mediante algoritmos de aprendizaje automático, identifica patrones lingüísticos, relacionados con la salud mental de los usuarios.

1.2. Objetivos

Como se ha mencionado anteriormente, el proyecto consiste en el desarrollo de una aplicación basada en aprendizaje automático para analizar publicaciones en Tumblr relacionadas con la salud mental. Su objetivo es identificar patrones de discurso que reflejen síntomas asociados a la depresión, asignando a cada publicación un índice que indique la presencia de indicios de dicho trastorno.

Esta iniciativa responde a la necesidad de contar con sistemas capaces de procesar grandes volúmenes de datos en redes sociales. Para ello, se entrenará un modelo de Deep Learning que clasificará el contenido según su posible relación con el trastorno.

Además, esta herramienta podría aplicarse en estudios de investigación y, potencialmente, en

plataformas digitales para mejorar la moderación de contenido y desarrollar estrategias de intervención temprana. Al aprovechar los testimonios de los usuarios en estas plataformas, se facilita el análisis sin depender exclusivamente de encuestas, que suelen ser más costosas y menos accesibles.

1.3. Limitaciones

Las siguientes limitaciones definen el diseño del proyecto:

- El sistema deberá procesar datos de publicaciones en Tumblr por lotes, respetando las limitaciones de la API en cuanto a volumen de solicitudes y acceso a información.
- El modelo de aprendizaje automático deberá ser capaz de identificar patrones relacionados con la salud mental y posibles riesgos. Para ello, el algoritmo de clasificación deberá alcanzar un nivel de precisión suficiente para garantizar resultados confiables.
- El modelo debe ser lo suficientemente simple y estar optimizado para garantizar que su entrenamiento es posible con los recursos disponibles.
- El sistema deberá cumplir con los principios de privacidad y ética en el manejo de datos, asegurando el anonimato de los usuarios, dada la sensibilidad de la información sobre la salud mental (deberá alinearse con el GDPR).
- El sistema debe ser intuitivo y accesible para investigadores y usuarios sin experiencia en ingeniería.
- Por último, el sistema debe ser interpretable y explicable, de modo que los resultados sean comprensibles y transparentes, y ofrezcan el contexto necesario para la investigación.

1.4. Estructura de la memoria

Esta memoria se organiza en seis capítulos, comenzando por la presente introducción. Los capítulos dos y tres están dedicados a contextualizar el proyecto: el capítulo dos, expone el marco teórico, incluyendo los conceptos relevantes para su desarrollo, mientras que el capítulo tres resume el estado actual de la investigación del campo.

El capítulo cuatro describe la aplicación web desarrollada, abordando en primer lugar el proceso de construcción del modelo y, en segundo lugar, la arquitectura de la aplicación.

Finalmente, el capítulo cinco presenta los resultados obtenidos, y el capítulo seis expone las conclusiones del trabajo.

2. Marco tecnológico

2.1. La IA y el Aprendizaje automático

La inteligencia artificial (IA) es un campo de la informática que se ocupa del diseño y desarrollo de sistemas capaces de realizar tareas que normalmente requieren inteligencia humana, como el aprendizaje, el razonamiento, la percepción y la toma de decisiones [6]. Su objetivo, es crear máquinas que puedan imitar o complementar la inteligencia humana, permitiendo automatizar y mejorando la eficiencia de procesos complejos.

El aprendizaje automático, una rama de la inteligencia artificial, dota a los sistemas de la capacidad de aprender a partir de datos y experiencia, sin necesidad de ser programados explícitamente para cada tarea [7]. Para ello, los modelos extraen relaciones estadísticas, patrones, dependencias y estructuras ocultas que caracterizan los datos.

Las técnicas de aprendizaje automático se pueden estructurar en distintos campos:

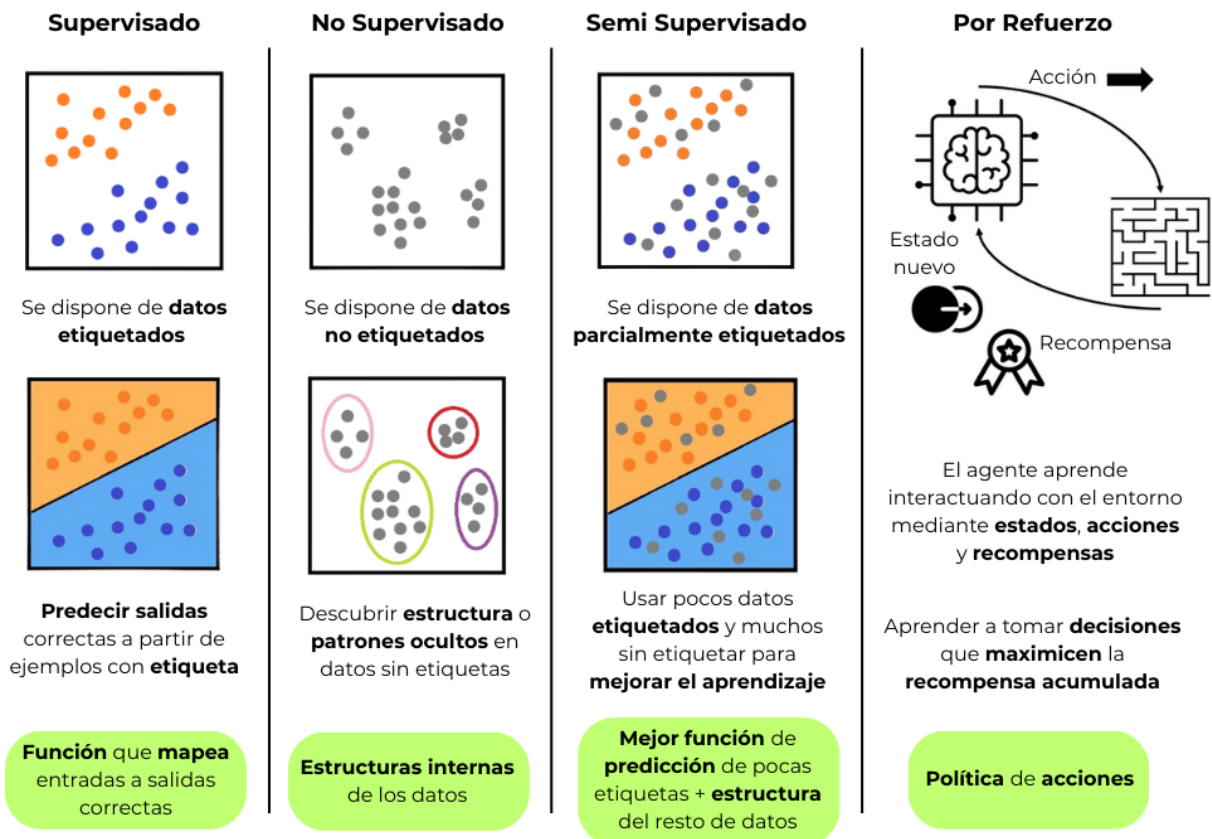


Figura 2.1: Diagrama de los principales tipos de Aprendizaje Automático, indicando el tipo de datos utilizados, el enfoque general y el conocimiento que se obtiene del modelo.

1) Aprendizaje supervisado. Cuando este proceso de aprendizaje (el entrenamiento) se realiza a partir de datos etiquetados, se busca modelar cómo influyen dichas características en la salida esperada, ajustando una función.

$$\hat{y} = f(\mathbf{x}; \theta) \quad (2.1)$$

Donde $\mathbf{x}$ representa las características de entrada, θ los parámetros del modelo aprendidos a partir de los datos, y $\hat{y}$ la predicción realizada [8]. Ejemplos clásicos incluyen la regresión lineal, la regresión logística, las máquinas de soporte vectorial (SVM), y las redes neuronales artificiales. Estas técnicas se aplican en tareas como clasificación de textos, predicción de series temporales, y diagnóstico médico asistido.

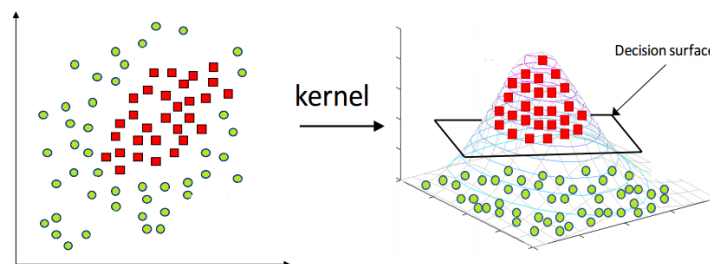


Figura 2.2: Ejemplo de un modelo de clasificación SVM que ha aprendido la superficie de decisión óptima para separar las clases verde y roja. [9]

2) Aprendizaje no supervisado. En este paradigma, el entrenamiento se realiza con datos sin etiquetar, con el objetivo de descubrir estructuras subyacentes o representaciones útiles en el conjunto de datos. En lugar de predecir una salida conocida, el modelo identifica patrones, agrupamientos o proyecciones en espacios de menor dimensión. Entre los métodos más representativos se encuentran el análisis de componentes principales (PCA), los algoritmos de *clustering* como *k-means* o *DBSCAN*, y las técnicas de reducción de dimensionalidad no lineales. Sus aplicaciones abarcan la segmentación de clientes, la detección de anomalías y la compresión de datos [10].

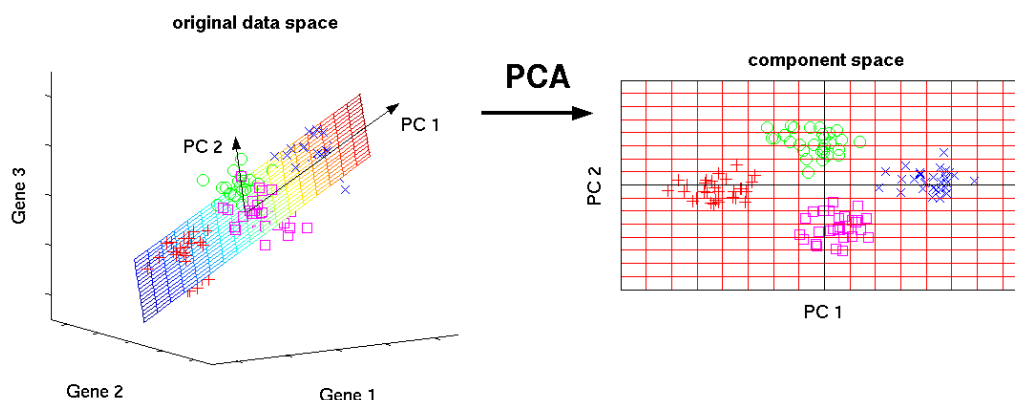


Figura 2.3: Ejemplo de reducción de la dimensionalidad mediante PCA. Proyecta los datos sin etiquetas en un nuevo espacio de componentes principales, revelando patrones y agrupaciones ocultas que no eran evidentes en las variables originales. [11]

3) Aprendizaje semi-supervisado. Este enfoque combina datos etiquetados con un gran volumen de datos no etiquetados, aprovechando la abundancia de estos últimos para mejorar el rendimiento de los modelos. La idea central es que, incluso con un conjunto reducido de ejemplos con anotaciones, es posible extraer información útil de la distribución global de los datos. Métodos como el autoentrenamiento, el uso de grafos para propagación de etiquetas y técnicas modernas

basadas en redes neuronales profundas han demostrado eficacia en contextos donde el etiquetado es costoso o difícil de obtener, como en la biomedicina o el análisis de lenguaje natural [12].

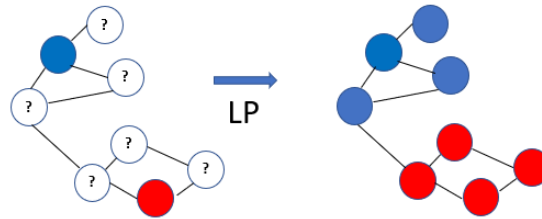


Figura 2.4: Ejemplo de aprendizaje semi-supervisado mediante propagación de etiquetas. A partir de unos pocos nodos etiquetados y la estructura del grafo, el modelo infiere las etiquetas de los nodos no anotados. [13]

4) Aprendizaje por refuerzo. A diferencia de los paradigmas anteriores, el aprendizaje por refuerzo se fundamenta en la interacción de un agente con un entorno. El agente toma decisiones (acciones) y recibe retroalimentación en forma de recompensas o penalizaciones, lo que le permite aprender una política de comportamiento óptima. Matemáticamente, este proceso suele modelarse mediante marcos como los Procesos de Decisión de Markov (MDP). Entre las técnicas más relevantes se encuentran los algoritmos *Q-Learning*, *SARSA* y los métodos basados en gradiente de política. Sus aplicaciones incluyen el control de robots, la optimización en redes de comunicaciones y los sistemas de recomendación adaptativos [14].

2.2. El Aprendizaje Supervisado como paradigma empleado en esta solución

El proyecto desarrollado se basa en una herramienta orientada a la identificación de síntomas a partir de texto. Para ello, se requiere un modelo de aprendizaje automático capaz de **mapear** las entradas textuales hacia las categorías de síntomas correspondientes.

Este problema se enmarca dentro del paradigma de **aprendizaje supervisado**, que, como se explicó anteriormente, permite **aprender relaciones explícitas** entre las características de **entrada** y las **salidas** esperadas, generando así la predicciones sobre datos no observados previamente.

Asimismo, los modelos supervisados pueden clasificarse en función del tipo de **variable objetivo**: si la tarea consiste en predecir una **etiqueta categórica**, se trata de un problema de clasificación; mientras que, si el objetivo es predecir un **valor numérico continuo**, se considera un problema de regresión.

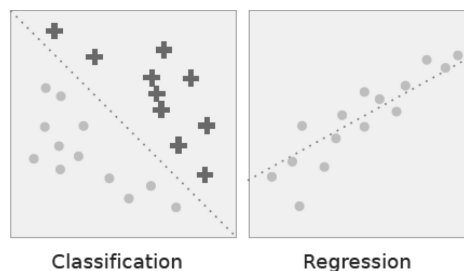


Figura 2.5: Clasificación vs. regresión. En clasificación, la línea discontinua representa una frontera que separa las dos clases; en regresión, modela la relación lineal entre las dos variables. [15]

En el caso de los modelos de **clasificación**, su finalidad es asignar una o varias **etiquetas discretas** a cada instancia de entrada. En nuestro contexto, estos modelos se emplean para identificar y categorizar textos en función de la presencia de ciertos síntomas. Entre las técnicas más representativas destacan la Regresión Logística y las Máquinas de Vectores de Soporte.

Por otro lado, los modelos de **regresión** están orientados a la predicción de **valores continuos**. Aplicado a este caso, podrían emplearse, por ejemplo, para estimar un porcentaje de probabilidad asociado a la presencia de depresión. Dentro de este enfoque destacan métodos como la Regresión Lineal y las redes neuronales con salidas continuas.

Para la **construcción** de un modelo supervisado, se realizan las siguientes fases [16]:

1. **Adquisición y comprensión de los datos:** El primer paso consiste en obtener los datos que servirán como base para el aprendizaje. Estos pueden proceder de múltiples fuentes: bases de datos estructuradas, documentos de texto, imágenes, sensores o, como en este caso, redes sociales. Una vez recopilados, se realiza un análisis exploratorio que permita comprender sus características principales, su distribución, la presencia de valores faltantes y posibles sesgos.
2. **Preprocesamiento y limpieza:** Los datos en bruto rara vez están listos para ser utilizados directamente en un modelo. Por ello, se aplican técnicas de limpieza que incluyen la eliminación de duplicados, el tratamiento de valores ausentes, la normalización de variables numéricas y la codificación de variables categóricas.
3. **División de los datos:** Para evaluar de forma objetiva la capacidad predictiva del modelo, los datos se dividen habitualmente en tres subconjuntos: entrenamiento, validación y prueba.
 - El **conjunto de entrenamiento** se emplea para ajustar los parámetros del modelo.
 - El **conjunto de validación** permite afinar hiperparámetros y prevenir el sobreajuste.
 - Finalmente, el **conjunto de prueba** ofrece una estimación imparcial del rendimiento final sobre datos no vistos.
4. **Selección y extracción de características:** Las características son las variables que el modelo utiliza para aprender patrones. En muchos casos, es necesario aplicar técnicas de selección de características para reducir la dimensionalidad y mejorar la generalización. Métodos como el *Principal Component Analysis* (PCA) o técnicas basadas en importancia de variables ayudan a identificar las representaciones más informativas.
5. **Elección y entrenamiento del modelo:** Una vez preparados los datos, se selecciona el algoritmo de aprendizaje más adecuado en función de la tarea (clasificación, regresión, *clustering*, etc.). El entrenamiento consiste en ajustar los parámetros internos del modelo mediante iteraciones sucesivas, de manera que se minimice una función de pérdida que mide la discrepancia entre las predicciones y los valores reales.
6. **Evaluación y validación:** El rendimiento del modelo debe evaluarse mediante métricas adecuadas al tipo de problema. En clasificación, se utilizan métricas como la precisión, el *recall*, la medida F1 o el área bajo la curva ROC (AUC). En regresión, son comunes el error cuadrático medio (MSE) o el coeficiente de determinación (R^2). La validación cruzada es una técnica ampliamente utilizada para obtener estimaciones más robustas del rendimiento.
7. **Ajuste de hiperparámetros:** Los modelos de aprendizaje automático suelen depender de hiperparámetros que no se aprenden automáticamente, como la profundidad de un árbol de

decisión o la tasa de aprendizaje de una red neuronal. Su optimización se realiza mediante métodos como la búsqueda en cuadrícula (*grid search*), búsqueda aleatoria (*random search*) o algoritmos más avanzados como *Bayesian optimization*.

8. **Despliegue y monitorización:** Una vez entrenado y evaluado, el modelo puede integrarse en aplicaciones reales para realizar predicciones sobre nuevos datos.

2.2.1. Regresión Lineal

La **regresión lineal** es un método estadístico y de aprendizaje supervisado utilizado para modelar la relación entre una variable dependiente y y una o más variables independientes x . Su objetivo principal es aproximar esta relación mediante una función lineal, generalmente expresada como:

$$\hat{y} = \mathbf{w}^\top \mathbf{x} + b \quad (2.2)$$

donde $\mathbf{w}$ es el vector de coeficientes que determina la pendiente de la relación y b es el término independiente.

En el caso de una única variable independiente se denomina regresión lineal simple, mientras que con múltiples variables se habla de regresión lineal múltiple. El modelo se entrena minimizando una función de pérdida, comúnmente el error cuadrático medio (MSE), lo que permite estimar los parámetros $\mathbf{w}$ y b [17].

La regresión lineal es simple, interpretable y computacionalmente eficiente. No obstante, supone que la relación entre las variables es lineal y que los errores son independientes, normalmente distribuidos y con varianza constante, lo que limita su aplicabilidad en contextos con relaciones no lineales o con multicolinealidad en los datos.

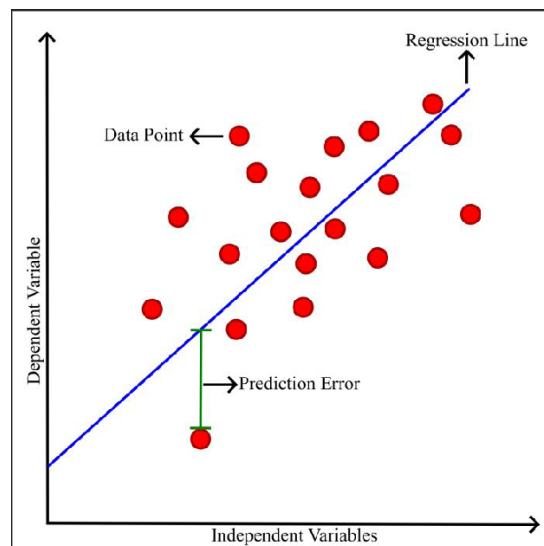


Figura 2.6: La línea azul representa el modelo de regresión, los puntos rojos son los datos y la distancia entre ambos es el error de predicción. [18]

2.2.2. Regresión Logística

La **regresión logística** es una técnica de aprendizaje supervisado utilizada para tareas de clasificación, donde la variable objetivo y es discreta, típicamente **binaria** ($y \in \{0, 1\}$). A diferencia de la regresión lineal, que predice valores continuos, la regresión logística modela la probabilidad de que un evento ocurra en función de las características de entrada $\mathbf{x}$.

Matemáticamente, la regresión logística se define como:

$$P(y = 1 | \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b), \quad (2.3)$$

donde $\mathbf{w}$ es el vector de pesos, b es el sesgo y σ es la función sigmoide:

$$\sigma(z) = \frac{1}{1 + e^{-z}}. \quad (2.4)$$

La **función sigmoide** transforma la combinación lineal de las características en un valor comprendido entre 0 y 1, que se interpreta como la **probabilidad de pertenencia** a la clase positiva. La predicción final se obtiene mediante un umbral, usualmente 0,5:

$$\hat{y} = \begin{cases} 1, & \text{si } P(y = 1 | \mathbf{x}) \geq 0,5, \\ 0, & \text{si } P(y = 1 | \mathbf{x}) < 0,5. \end{cases} \quad (2.5)$$

El entrenamiento del modelo consiste en ajustar los parámetros $\mathbf{w}$ y b para maximizar la probabilidad de las etiquetas observadas en el conjunto de datos. Esto se realiza típicamente mediante la maximización de la función de verosimilitud o la **minimización de la pérdida logarítmica** (log-loss):

$$\mathcal{L}(\mathbf{w}, b) = -\frac{1}{N} \sum_{i=1}^N [y_i \log \hat{y}_i + (1 - y_i) \log (1 - \hat{y}_i)]. \quad (2.6)$$

La regresión logística es sencilla de usar y permite entender fácilmente cómo cada característica de entrada influye en la probabilidad de que ocurra un determinado evento. Funciona especialmente bien en problemas donde las clases pueden separarse aproximadamente con una frontera lineal. Sin embargo, su desempeño puede verse limitado cuando las relaciones entre las variables son muy complejas o no lineales, en cuyo caso técnicas más avanzadas, como redes neuronales o máquinas de soporte vectorial, podrían ofrecer mejores resultados [19].

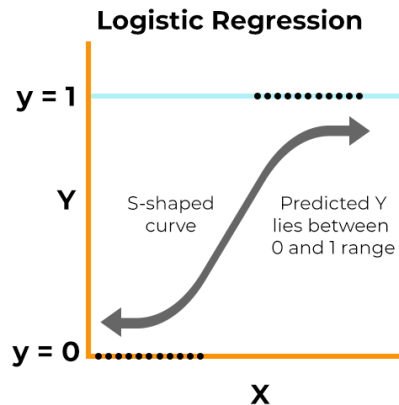


Figura 2.7: La curva en forma de S modela la probabilidad de que la variable dependiente y tome el valor 1 en función de la variable independiente x , asegurando que las predicciones se mantengan en el rango $[0,1]$. [20]

2.2.3. Máquinas de Soporte Vectorial

Las **máquinas de soporte vectorial** (SVM) son un método de aprendizaje supervisado empleado principalmente para tareas de **clasificación**, aunque también existen variantes para problemas de regresión. El objetivo de una SVM es encontrar un **hiperplano óptimo** que separe las clases de datos de manera que la margen entre ellas sea máxima, es decir, que la distancia entre el hiperplano y los puntos de entrenamiento más cercanos (denominados vectores de soporte) sea lo mayor posible.

Para un conjunto de datos linealmente separable, el hiperplano se define como:

$$\mathbf{w}^\top \mathbf{x} + b = 0 \quad (2.7)$$

donde $\mathbf{w}$ es el vector normal al hiperplano y b es el sesgo. La predicción de clase para un nuevo punto $\mathbf{x}$ se realiza mediante:

$$\hat{y} = \text{sign}(\mathbf{w}^\top \mathbf{x} + b) \quad (2.8)$$

El criterio de optimización de la SVM consiste en maximizar el margen $\frac{2}{\|\mathbf{w}\|}$ bajo la restricción de que todos los puntos de entrenamiento sean correctamente clasificados:

$$y_i (\mathbf{w}^\top \mathbf{x}_i + b) \geq 1, \quad i = 1, \dots, N \quad (2.9)$$

En situaciones donde los datos no son linealmente separables, se introduce un parámetro de penalización C que permite errores de clasificación, o bien se aplica un *kernel* que transforma los datos a un espacio de mayor dimensión donde sí resulten separables.

Las SVM presentan varias ventajas: son eficaces en espacios de alta dimensión, resultan robustas frente al sobreajuste y ofrecen flexibilidad mediante el uso de kernels, lo que permite modelar relaciones no lineales. Sin embargo, también presentan algunas limitaciones. Su rendimiento puede ser sensible a la elección de los hiperparámetros C y γ (en el caso de *kernels RBF*) y, además, no escalan bien a conjuntos de datos extremadamente grandes [21].

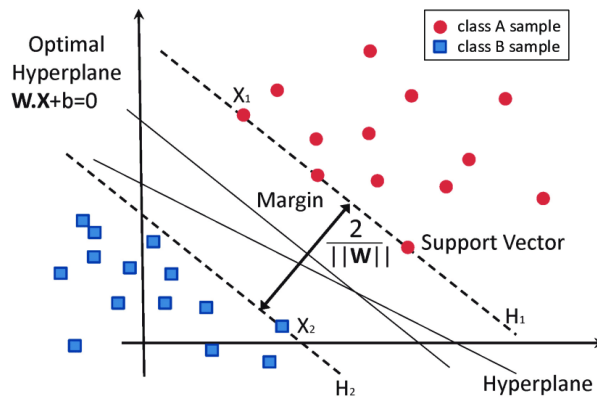


Figura 2.8: Representación gráfica de un clasificador SVM. El hiperplano óptimo maximiza el margen entre las clases, definido por los vectores de soporte (puntos más cercanos al límite de decisión). [22]

2.2.4. Clasificación multi-etiqueta

Cuando la etiqueta no es binaria, se habla de clasificación multiclase. Por otro lado, si una misma entrada puede estar asociada a varias etiquetas de manera simultánea, se trata de **clasificación multi-etiqueta** [23]. Este enfoque es adecuado para problemas en los que las **categorías no son mutuamente excluyentes**.

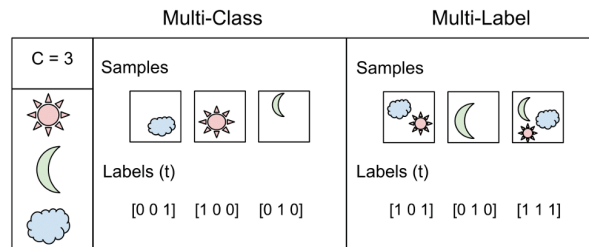


Figura 2.9: Diferencia entre clasificación multiclase y multietiqueta. En la multiclase cada muestra pertenece a una única categoría, mientras que en la multietiqueta puede estar asociada simultáneamente a varias categorías. [24]

Existen técnicas, con las cuales se permite considerar las correlaciones entre etiquetas, en vez de tratarlas de manera independiente.

$$\hat{\mathbf{y}} = f(\mathbf{x}) \in \{0, 1\}^K$$

donde K es el número de etiquetas posibles y $\hat{\mathbf{y}}$ es un vector binario que indica la presencia o ausencia de cada etiqueta.

Etiquetas: $L = \{ \text{☀️} \text{☁️} \text{☔️} \text{☁️} \text{☔️} \}$

Nº Ejemplo	Conjunto de etiquetas
1	☀️ ☁️ ☔️
2	☁️ ☔️ ☔️
3	☁️ ☔️ ☔️
4	☀️

Figura 2.10: Ejemplos de un problema de clasificación multietiqueta: cada instancia puede asociarse con uno o varios elementos del conjunto de etiquetas L .

Entre las técnicas más comunes para clasificación multi-etiqueta se encuentran:

- **Binary Relevance:** descompone un problema multi-etiqueta en **múltiples problemas de clasificación binaria independientes**, entrenando un clasificador por etiqueta. Aunque destaca por su simplicidad y escalabilidad, ignora por completo las posibles dependencias

entre etiquetas, lo que puede limitar su rendimiento en dominios donde estas correlaciones son relevantes.

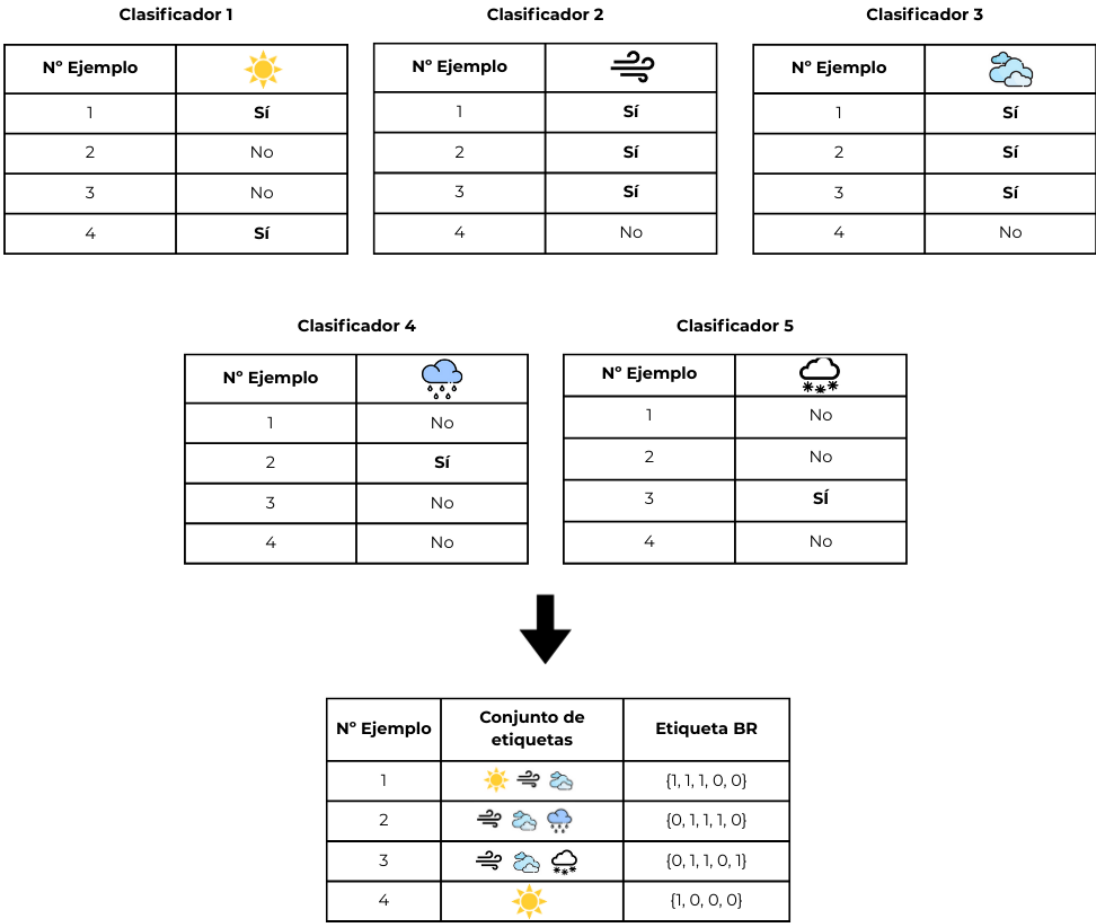


Figura 2.11: Representación del enfoque BR: cada clasificador aprende de forma independiente si una etiqueta (sol, viento, nubes, lluvia o nieve) está presente en cada ejemplo.

- Classifier Chains:** extienden BR modelando explícitamente las **dependencias entre etiquetas**. Cada clasificador en la cadena predice una etiqueta utilizando no solo las características originales, sino también las predicciones anteriores como entradas adicionales. Esta secuenciación permite capturar correlaciones entre etiquetas, aunque introduce sensibilidad al orden de la cadena.

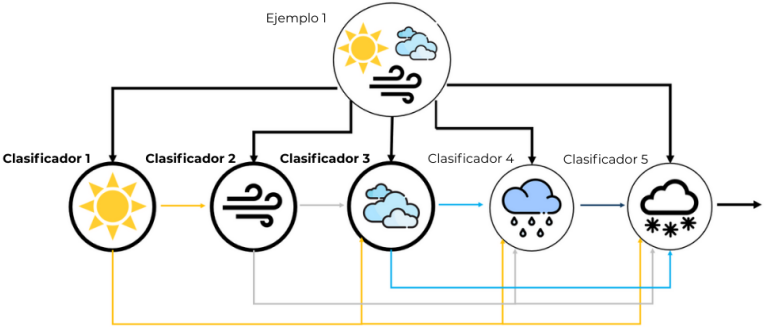


Figura 2.12: Esquema del método CC: los clasificadores se enlazan, de modo que la predicción de cada uno alimenta a los siguientes, incorporando dependencias entre etiquetas

- **Label Powerset:** convierte el problema multi-etiqueta en uno **multi-clase**, tratando **cada combinación única de etiquetas** como una clase independiente. Este enfoque permite capturar relaciones complejas entre etiquetas, pero es limitado en cuanto a escalabilidad y capacidad de generalización ya que el número de combinaciones posibles crece exponencialmente.

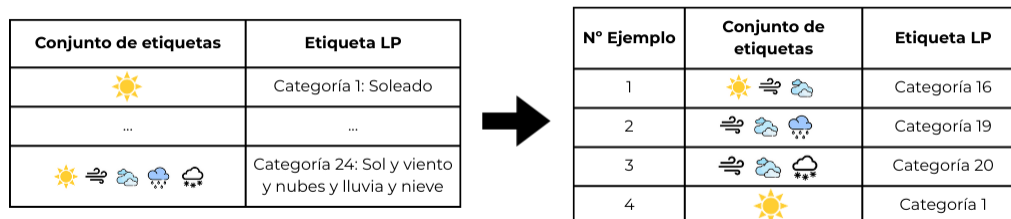


Figura 2.13: Conversión de conjuntos de etiquetas a categorías únicas mediante el enfoque LP: cada combinación de condiciones climáticas se asigna a una categoría única.

- **Ensamblados:** Los métodos de **ensamblado** adaptados a multi-etiqueta, como *Random Forest Predictive Clustering Trees* (RFPCT) y *Random Forests* basados en transformaciones (RFDTBR), **combinan múltiples clasificadores base** para mejorar la robustez y generalización. Estos enfoques han demostrado ser altamente efectivos en evaluaciones empíricas recientes, logrando un buen equilibrio entre precisión y eficiencia computacional.
- **Transformers:** Los enfoques basados en Deep Learning, en particular los Transformers (p. ej., BERT), han revolucionado la clasificación multi-etiqueta, especialmente en tareas de procesamiento de texto. Gracias a su **capacidad para modelar dependencias contextuales** y relaciones semánticas entre etiquetas, superan a los métodos tradicionales en precisión, especialmente en contextos con datos no estructurados y gran dimensionalidad.

En consecuencia, en este proyecto se ha optado por emplear un enfoque basado en *Transformers*, dada su idoneidad para tareas con datos textuales.

2.2.5. Procesamiento de Lenguaje Natural

Dado que los datos del modelo desarrollado en este proyecto corresponden a texto, resulta necesario un procesamiento específico que permita convertir el lenguaje natural en una representación numérica apta para su utilización en modelos de aprendizaje automático. Este conjunto de técnicas y metodologías se enmarca en el campo del **Procesamiento de Lenguaje Natural** (NLP).

El NLP es una subdisciplina de la inteligencia artificial que estudia la interacción entre los sistemas computacionales y el lenguaje humano. Su objetivo principal es permitir que las máquinas comprendan, analicen, interpreten y generen texto o habla en lenguaje natural de forma automatizada [25].

Entre las tareas más comunes del NLP se incluyen:

- **Clasificación de texto:** asignación de etiquetas o categorías a documentos.
- **Extracción de información:** identificación de entidades, relaciones o conceptos clave en un texto.

- **Resumen automático:** generación de resúmenes breves a partir de documentos extensos.
- **Traducción automática:** conversión de texto entre distintos idiomas.
- **Análisis de sentimientos:** determinación de la polaridad (positiva, negativa o neutral) de una opinión expresada en un texto.

Dentro del NLP se realizan los siguientes pasos específicos para la preparación de los datos [26]:

1. **Preparación de datos.** En esta fase inicial se lleva a cabo una limpieza preliminar que incluye la eliminación de duplicados, la corrección de errores tipográficos y la unificación de formatos. El objetivo es garantizar la calidad y coherencia de los datos antes de proceder a su análisis.
2. **Preprocesamiento del texto.** El texto en bruto se somete a operaciones de normalización que permiten reducir su complejidad y estandarizar la información. Entre las técnicas más habituales se encuentran la *tokenización* (segmentación en palabras o subpalabras), la conversión a minúsculas, la eliminación de signos de puntuación y *stopwords*, así como la lematización o *stemming*, que reducen las palabras a su forma base.
3. **Representación vectorial.** Para que el texto sea interpretable por un modelo de aprendizaje automático, es necesario transformarlo en una representación numérica. Entre los métodos clásicos destacan el modelo de *Bag of Words* y el esquema *TF-IDF*, que pondera los términos en función de su frecuencia relativa. En enfoques más avanzados, se utilizan representaciones densas conocidas como *embeddings* (*Word2Vec*, *GloVe*, *FastText*) o *embeddings* contextuales derivados de arquitecturas profundas como BERT, capaces de capturar relaciones semánticas y dependencias contextuales.
4. **Extracción y construcción de características.** En esta etapa se generan variables adicionales relevantes para la tarea específica, como medidas de longitud del texto, patrones sintácticos, presencia de entidades nombradas o secuencias de *n*-gramas. Estas características enriquecen la representación vectorial y favorecen la capacidad predictiva del modelo.

En el caso de emplear un modelo Transformer, el procesamiento del texto se realiza de la siguiente manera:

1. **Tokenización:** separar el texto en unidades (tokens).
2. **Conversión a IDs:** asignar a cada token un número según un vocabulario predefinido, el cual corresponde al conjunto de subpalabras y tokens especiales que el modelo aprendió durante su preentrenamiento.
3. **Embeddings:** transformar los IDs en vectores numéricos de dimensión fija.
4. **Modelo:** pasar los vectores por un modelo para obtener una representación contextualizada.
5. **Capa de salida:** aplicar una función de activación para obtener predicciones.

Token	ID en el vocabulario
[CLS]	101
I	146
feel	1247
so	2061
tired	12090
[SEP]	102

Tabla 2.1: Tokenización y asignación de IDs según el vocabulario de BERT (modelo Transformer). Los IDs corresponden a un índice fijo dentro del vocabulario con el que se preentrenó el modelo. En caso de que una palabra no exista en dicho vocabulario, esta se descompone en subpalabras conocidas o, si no es posible, se reemplaza por el token especial [UNK] (unknown) durante la tokenización.

2.3. Deep Learning

Los modelos Transformer, previamente mencionados en el contexto de clasificación multi-etiqueta y NLP, son un ejemplo de arquitectura avanzada de **Aprendizaje Profundo** o Deep Learning. El aprendizaje profundo es un subcampo del ML cuyas técnicas emplean **redes neuronales artificiales** para dotar a los modelos de una mayor capacidad de representación, permitiéndoles detectar relaciones complejas entre datos.

Para ello, cada **capa de la red neuronal** transforma su entrada en una representación progresivamente más abstracta, extrayendo características de mayor nivel en cada etapa. De este modo, el modelo automatiza la selección y diseño de características, a diferencia del aprendizaje automático clásico, donde la extracción de características es manual y requiere conocimientos específicos del dominio [27].

Sin embargo, debido a esa capacidad para modelar relaciones complejas, el DL requiere grandes volúmenes de datos para mantener su capacidad de generalización y evitar el sobreajuste [28].

Las redes neuronales se estructuran en **capas**, cada una compuesta por múltiples **nodos**. Su organización habitual es la siguiente [29]:

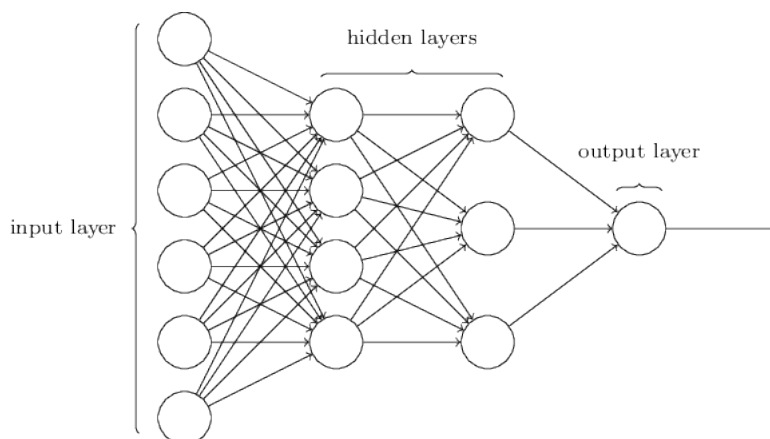


Figura 2.14: Estructura de una red neuronal profunda con dos capas ocultas [30]

- **Capa de entrada:** Recibe directamente los datos a procesar. El número de nodos corresponde al número de características del vector de entrada.

- **Capas ocultas:** Encargadas de construir representaciones intermedias mediante combinaciones no lineales de las entradas.
- **Capa de salida:** Produce el resultado final del modelo, que puede ser una predicción de clase, una regresión, o cualquier otra salida según la tarea.

2.3.1. Nodo de una Red Neuronal

Los **nodos**, también denominado neuronas artificiales, constituye la unidad fundamental de una red neuronal. Su función principal es procesar información recibida de capas anteriores y generar una salida que puede ser utilizada por capas subsecuentes o por la capa de salida del modelo [31].

Cada nodo está compuesto por los siguientes elementos [32]:

- **Entradas** ($x_1, x_2, \dots, x_n$): Representan los valores que el nodo recibe. En la capa de entrada corresponden directamente a las características del conjunto de datos, mientras que en las capas ocultas se trata de las salidas de las neuronas de la capa anterior.
- **Pesos** ($w_1, w_2, \dots, w_n$): Cada entrada está asociada a un peso que determina su relevancia relativa en el cálculo de la salida del nodo. Durante el entrenamiento de la red, estos pesos se ajustan mediante algoritmos de optimización para minimizar el error del modelo.
- **Sesgo** (b): Valor adicional que permite desplazar la función de activación, proporcionando mayor flexibilidad al nodo y posibilitando su activación incluso cuando las entradas sean nulas.
- **Función de activación** (f): Aplicada sobre la combinación lineal de entradas y pesos más el sesgo, esta función introduce no linealidad en el modelo, lo que habilita a la red para aprender relaciones complejas entre los datos. Entre las funciones más comunes se encuentran ReLU, Sigmoid y Tanh.

Formalmente, la operación de un nodo puede expresarse como:

$$z = \sum_{i=1}^n x_i w_i + b$$

$$y = f(z)$$

donde z representa la suma ponderada de las entradas más el sesgo, y y es la salida del nodo después de aplicar la función de activación.

Así, las entradas recibidas son ponderadas, ajustando la respuesta mediante un sesgo y generando una salida transformada mediante una función de activación.

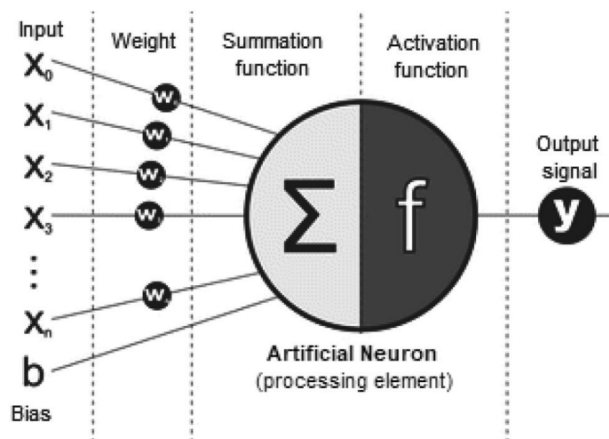


Figura 2.15: Esquema de una neurona artificial: los valores de entrada ($x_0, x_1, \dots, x_n$) se combinan con sus respectivos pesos (w) y un sesgo (b). Luego, una función de suma calcula el valor agregado, que pasa por una función de activación para generar la salida (y). [33]

2.3.2. Principales modelos de aprendizaje profundo

El aprendizaje profundo incluye diversas arquitecturas de redes neuronales, cada una optimizada para distintos tipos de datos y tareas. A continuación se describen los modelos más representativos [8]:

- **Perceptrón Multicapa (MLP):** El MLP es la arquitectura más básica de red neuronal profunda, formada por capas de **nodos totalmente conectados**. Cada nodo combina linealmente las entradas mediante pesos, añade un sesgo y aplica una función de activación no lineal, como ReLU o Sigmoid, para introducir capacidad de modelado de relaciones complejas. Los MLP son adecuados para tareas de clasificación y regresión en datos tabulares o estructurados, aunque presentan limitaciones al procesar datos con estructura espacial o secuencial.

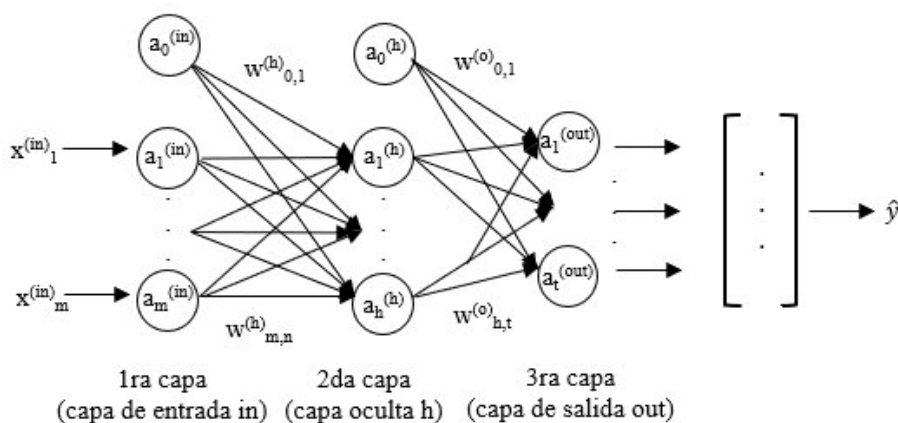


Figura 2.16: Arquitectura de un MLP: la información fluye desde la capa de entrada, pasando por una o varias capas ocultas, hasta la capa de salida, donde se generan las predicciones. [34]

- **Redes Neuronales Convolucionales (CNN):** Las CNN son un tipo de arquitectura de aprendizaje profundo especialmente diseñada para procesar **datos con estructura espacial**, como imágenes o secuencias bidimensionales. A diferencia de los MLP, que conectan

cada neurona de una capa con todas las neuronas de la siguiente, las CNN emplean convoluciones locales, lo que significa que cada neurona se conecta solo a una pequeña región de la capa anterior. Esto permite capturar patrones espaciales y características locales, como bordes, texturas o formas en las imágenes.

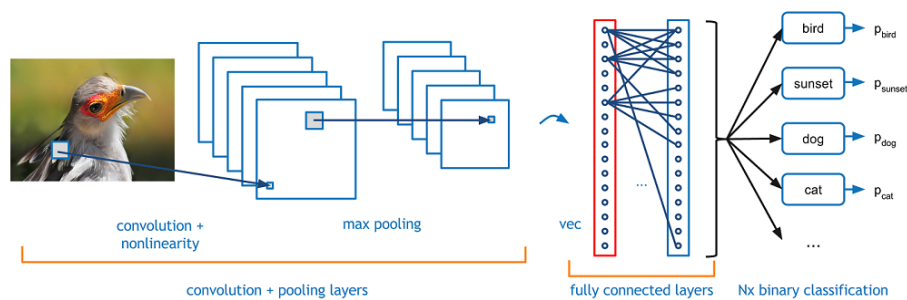


Figura 2.17: Esquema de una CNN: las capas de convolución y pooling extraen características de la imagen, que posteriormente son procesadas por capas totalmente conectadas para realizar la clasificación final. [35]

Una CNN típica está compuesta por varios tipos de capas:

- **Capas convolucionales:** aplican filtros o *kernels* que recorren la entrada para detectar características locales. Cada filtro produce un **mapa de activación**, que resalta la presencia de ciertos patrones en la región de entrada.
- **Capas de pooling:** reducen la dimensión espacial de los mapas de activación mediante operaciones como *max pooling* o *average pooling*, lo que disminuye la complejidad computacional y ayuda a la invariancia a traslaciones pequeñas.
- **Capas totalmente conectadas:** al final de la red, los mapas de activación se aplanan y se conectan a una capa densa que realiza la clasificación o regresión final.

Además, las CNN utilizan técnicas como *normalización*, *dropout* y funciones de activación no lineales para mejorar la generalización y reducir el sobreajuste. Gracias a estas características, las CNN han demostrado ser altamente efectivas en tareas de reconocimiento de imágenes, segmentación, detección de objetos y, en menor medida, en procesamiento de lenguaje natural.

- **Redes recurrentes (RNN) y variantes:** Las RNN son un tipo de arquitectura de aprendizaje profundo diseñada para procesar **secuencias de datos** donde el **orden y la dependencia temporal** son relevantes, como series temporales o texto. A diferencia de los MLP y CNN, las RNN cuentan con **conexiones recurrentes** que permiten que la salida de una capa en un instante temporal se retroalimente como entrada en el siguiente. Esto dota a la red de una **memoria** interna que captura información de pasos anteriores, lo que le permite modelar dependencias a lo largo de la secuencia. Sin embargo, las RNN estándar presentan el problema del desvanecimiento del gradiente, lo que dificulta aprender relaciones a largo plazo.

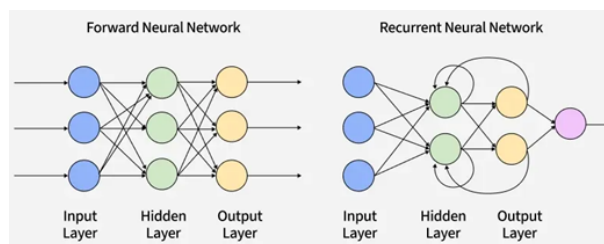


Figura 2.18: Comparación entre una red neuronal feedforward, donde la información fluye en una sola dirección, y una RNN, que incorpora conexiones hacia atrás para modelar dependencias temporales en secuencias. [36]

Para superar esta limitación se desarrollaron variantes como **Long Short-Term Memory (LSTM)** y **Gated Recurrent Units (GRU)**:

- **LSTM**: es una arquitectura de RNN que incorpora celdas de memoria y tres puertas (entrada, olvido y salida) para controlar el flujo de información. Esto permite que la red recuerde o olvide información relevante a lo largo de secuencias largas, mitigando el problema del desvanecimiento del gradiente y mejorando la captura de dependencias a largo plazo.
- **GRU**: es una versión simplificada de LSTM que combina algunas de las puertas en dos: actualización y reinicio. Aunque tiene una estructura más simple y menos parámetros que LSTM, las GRU son capaces de capturar dependencias a largo plazo de manera eficiente y suelen entrenarse más rápido, manteniendo un rendimiento comparable en muchas tareas de secuencias.

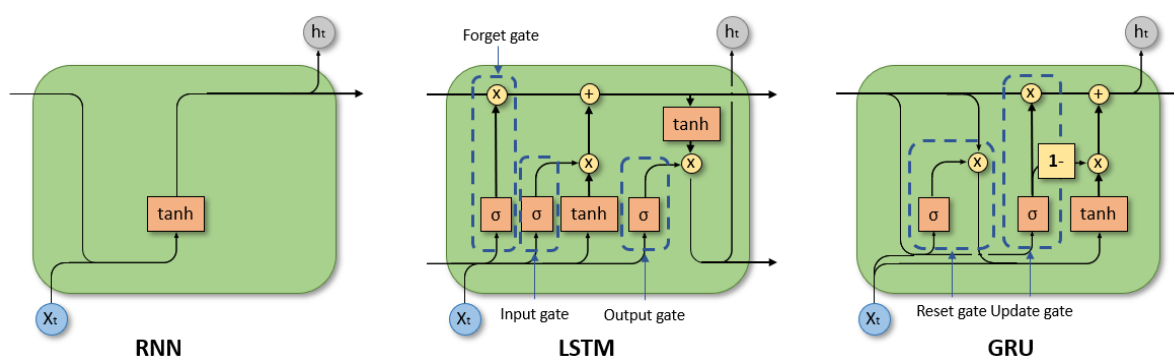


Figura 2.19: Comparación de arquitecturas de redes neuronales recurrentes: RNN simple con retroalimentación básica; LSTM con compuertas de entrada, olvido y salida para manejar dependencias a largo plazo; y GRU, una variante más ligera que combina compuertas de actualización y reinicio. [37]

Estas arquitecturas se utilizan ampliamente en procesamiento de lenguaje natural, traducción automática, generación de texto, reconocimiento de voz, series temporales financieras, entre otros dominios donde la información secuencial es fundamental.

■ Redes generativas y no supervisadas:

Estas arquitecturas pertenecen al aprendizaje profundo y se caracterizan por aprender patrones subyacentes en los datos sin necesidad de etiquetas explícitas, lo que las hace especialmente útiles para tareas de descubrimiento de características, reducción de dimensionalidad y generación de nuevos datos. Entre las más relevantes se encuentran los *Autoencoders* (AE), los *Variational Autoencoders* (VAE) y los *Generative Adversarial Networks* (GAN).

- **Autoencoders (AE):** Los AE son redes neuronales diseñadas para aprender una representación compacta de los datos de entrada. Están compuestos por dos partes: un *codificador* que transforma la entrada en un espacio latente de menor dimensión, y un *decodificador* que reconstruye la entrada original a partir de esa representación comprimida. Esta estructura permite que los AE se utilicen para reducción de dimensionalidad, eliminación de ruido (*denoising*), detección de anomalías y, en algunos casos, generación de datos sintéticos al manipular el espacio latente.

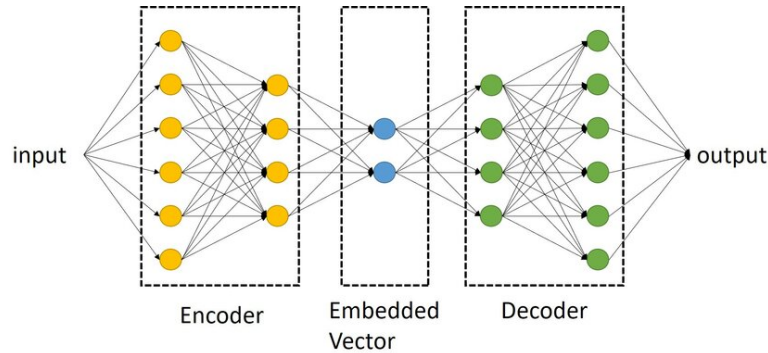


Figura 2.20: Arquitectura de un AE. El *encoder* transforma los datos de entrada en una representación comprimida o espacio latente (*latent features*), mientras que el *decoder* utiliza dicha representación para reconstruir la salida, buscando minimizar la diferencia entre entrada y salida. Este proceso permite el aprendizaje no supervisado de representaciones compactas. [38]

- **Variational Autoencoders (VAE):** Los VAE extienden el concepto de AE introduciendo un enfoque probabilístico. En lugar de codificar la entrada como un único punto en el espacio latente, los VAE modelan una distribución (generalmente gaussiana) para cada dimensión latente. Durante el entrenamiento, se aprende a muestrear del espacio latente de manera que las muestras reconstruidas sean coherentes con la distribución de los datos originales. Esto permite generar nuevas instancias de datos realistas y controlar propiedades específicas mediante la manipulación del espacio latente, siendo muy útiles en tareas de generación de imágenes, audio o secuencias de texto.

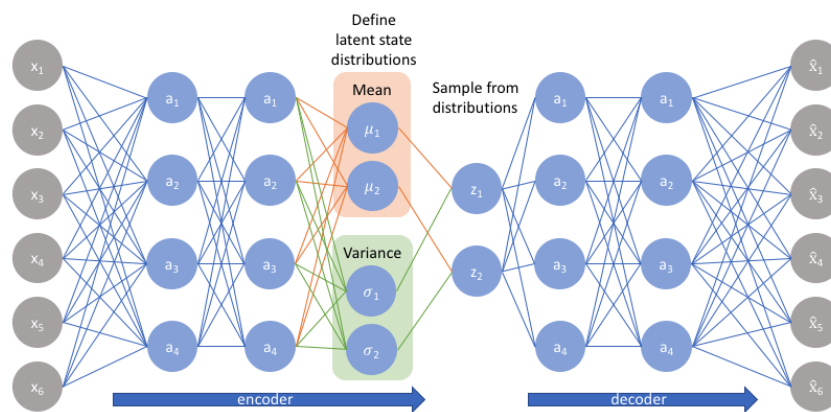


Figura 2.21: Estructura de un VAE. El *encoder* transforma la entrada en distribuciones latentes caracterizadas por una media (μ) y una varianza (σ). A partir de estas distribuciones se muestrean variables latentes (z), que posteriormente son utilizadas por el *decoder* para reconstruir la entrada original. Este enfoque permite un aprendizaje probabilístico de representaciones latentes continuas. [39]

- **Generative Adversarial Networks (GAN):** Los GAN consisten en dos redes neuronales que compiten en un juego de suma cero: un *generador*, que crea datos sintéticos, y un *discriminador*, que intenta distinguir entre datos reales y generados. Durante el entrenamiento, el generador mejora progresivamente su capacidad de producir muestras cada vez más realistas, mientras que el discriminador se vuelve más eficiente en su clasificación. Este mecanismo adversarial permite generar imágenes, audio o texto de alta fidelidad y ha revolucionado el campo de la síntesis de datos, incluyendo aplicaciones como mejora de resolución de imágenes (*super-resolution*), transferencia de estilo y generación de contenido creativo.

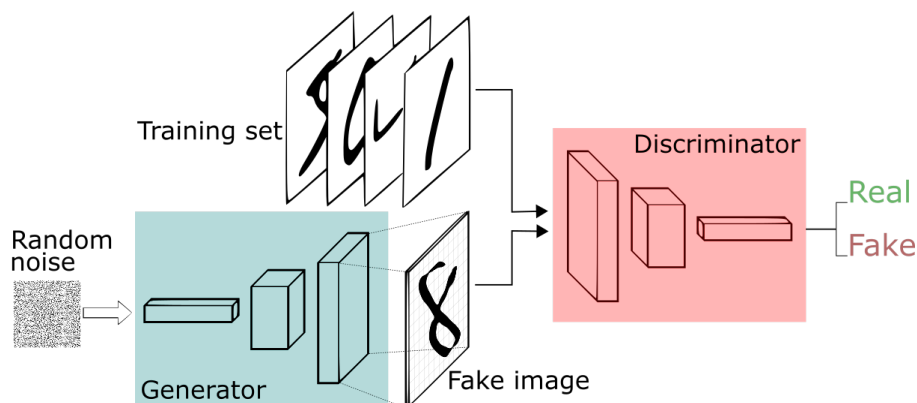


Figura 2.22: Esquema de una GAN. El generador crea datos sintéticos a partir de ruido aleatorio, mientras que el discriminador evalúa si los datos corresponden a muestras reales o falsas. Ambos modelos se entrenan de forma competitiva hasta que el generador produce datos cada vez más realistas. [40]

Modelo	Descripción
MLP	Totalmente conectada con una o más capas ocultas. Se utiliza para tareas generales de clasificación y regresión.
CNN	Diseñada para procesar datos con estructura de cuadrícula, como imágenes. Utiliza convoluciones para extraer características espaciales.
RNN	Diseñada para procesar secuencias, donde las salidas dependen del estado previo. Común en procesamiento de lenguaje natural y series temporales.
LSTM	Variante de RNN que mitiga el problema del desvanecimiento del gradiente. Captura dependencias a largo plazo en secuencias.
GRU	Variante simplificada de LSTM con rendimiento comparable, menor complejidad computacional.
Autoencoder	Red entrenada para reconstruir su entrada. Utilizado para reducción de dimensionalidad, compresión y detección de anomalías.
GAN	Compuesta por un generador y un discriminador que compiten entre sí. Se usa para generación de datos sintéticos realistas.
Transformer	Modelo basado en mecanismos de atención que reemplaza a las RNN en muchas tareas de NLP. Permite procesamiento paralelo y captura de relaciones a largo plazo.

Tabla 2.2: Resumen comparativo de las principales arquitecturas de aprendizaje profundo y sus aplicaciones más comunes.

En el procesamiento de texto, estas arquitecturas presentan limitaciones significativas. Los MLP son adecuados únicamente para representaciones vectoriales simples, pero no capturan dependencias contextuales entre palabras. Las CNN pueden identificar patrones locales, aunque su capacidad para modelar dependencias largas es reducida. Las RNN, junto con variantes como LSTM y GRU, fueron ampliamente utilizadas en secuencias, pero su entrenamiento resulta cos-

toso en tiempo y memoria, además de ser propensas al sobreajuste y difíciles de escalar. Por su parte, los modelos generativos (AE, VAE o GAN) han demostrado utilidad en representación latente y síntesis de datos, pero no están diseñados de manera específica para la clasificación supervisada.

Estas limitaciones impulsaron el desarrollo de nuevas arquitecturas, entre las cuales destacan los **Transformers**, que se han consolidado como el enfoque predominante en procesamiento de lenguaje natural [41].

2.3.3. Modelos Transformer

El **Transformer** es una arquitectura de red neuronal profunda diseñada para procesar secuencias de datos, como texto, audio o series temporales. Su principal innovación es el **mecanismo de atención**, que sustituye a las estrategias tradicionales basadas en recurrencia o convolución. Este mecanismo permite que cada elemento de la secuencia incorpore de manera simultánea información de todos los demás, capturando dependencias tanto locales como de largo alcance de forma eficiente [42].

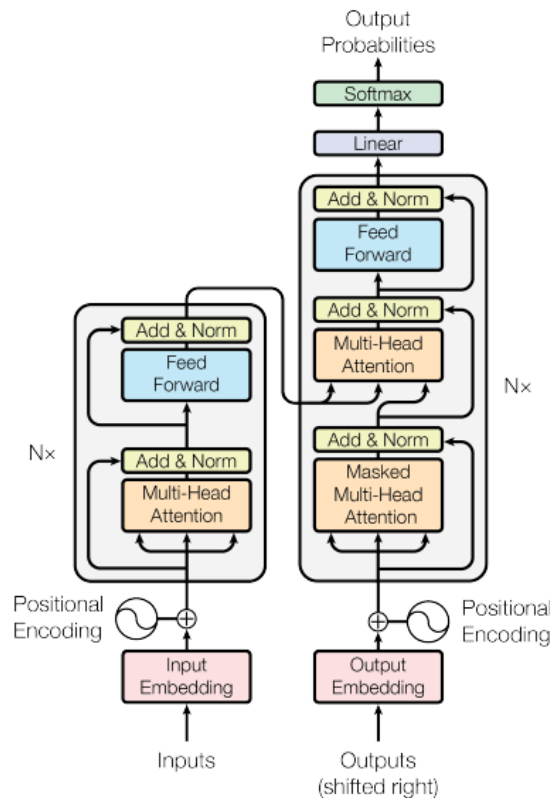


Figura 2.23: Arquitectura general del modelo Transformer con estructura encoder-decoder. A la izquierda se representa el **encoder**, formado por N bloques que incluyen atención multi-cabeza y redes feed-forward, cada uno con normalización y conexiones residuales. A la derecha, el **decoder** incluye atención enmascarada, atención cruzada con las salidas del encoder y componentes similares. Las codificaciones posicionales se suman a los embeddings de entrada para preservar el orden secuencial. [43]

A diferencia de arquitecturas como las RNN, LSTM o CNN, los Transformers **procesan toda**

la secuencia en paralelo, lo que acelera el entrenamiento y facilita el manejo de contextos largos. Además, su diseño modular y escalable los hace especialmente adecuados para el preentrenamiento sobre grandes volúmenes de datos y el ajuste fino en tareas específicas, razón por la cual se han convertido en la base de la mayoría de los modelos actuales de procesamiento de lenguaje natural [44].

En cuanto a su arquitectura, un Transformer se organiza en **bloques apilados**, es decir, capas idénticas dispuestas una sobre otra de forma secuencial. Cada bloque realiza un conjunto específico de operaciones, y al apilarse permiten al modelo construir representaciones progresivamente más complejas de los datos de entrada.

Estos bloques incorporan además **conexiones residuales** y **normalización de capa**, mecanismos fundamentales para entrenar redes profundas de manera estable. Las conexiones residuales facilitan que la información fluya directamente hacia capas posteriores sin degradarse, mientras que la normalización de capa mantiene las activaciones en rangos controlados.

Estos elementos estabilizan el entrenamiento y favorecen un flujo de gradiente más efectivo, lo que garantiza que las señales de error se propaguen de las capas de salida hacia las capas iniciales sin desvanecerse ni amplificarse en exceso.

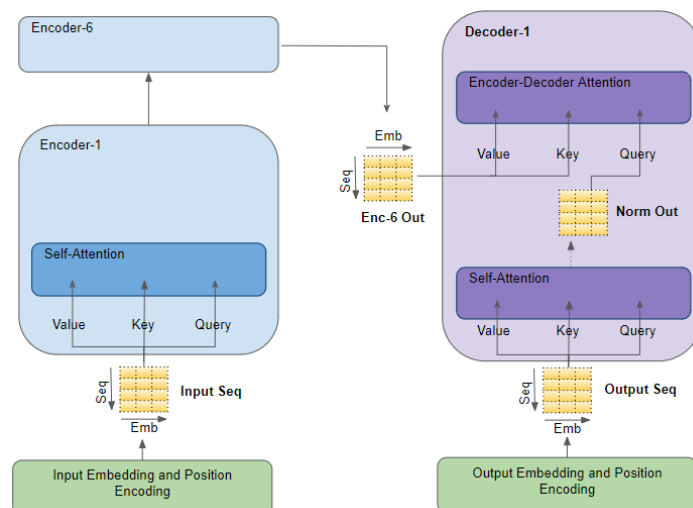


Figura 2.24: Arquitectura básica de un Transformer: la secuencia de entrada se transforma mediante embeddings y codificación posicional, pasa por capas de self-attention en el encoder y se entrega al decoder, donde la atención combinada (self y encoder-decoder attention) genera la secuencia de salida. [45]

- **Bloque de encoder.** Cada capa de encoder contiene varios componentes clave:
 - **Atención multi-cabeza (Self-Attention):** permite que cada posición de la secuencia “atienda” a todas las demás. Se asigna un peso diferente a cada token del contexto al calcular la representación de un token dado. Esto se calcula mediante las consultas (Q), claves (K) y valores (V), según la fórmula:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

Las múltiples cabezas permiten capturar de forma paralela patrones de dependencia diversos (por ejemplo, relaciones sintácticas y semánticas) en diferentes subespacios de representación.

$$\text{softmax} \left(\frac{Q \times K}{\sqrt{d_k}} \right) = \text{Mapa de Atención}$$

Importantly, the sum of each row is 1.

Figura 2.25: Esquema del mecanismo de autoatención en transformadores: los vectores de consulta (Q) y clave (K) se multiplican para generar un mapa de atención, donde cada celda representa la relación entre un token y los demás en la secuencia. [42]

- **Red feed-forward (FFN):** dos capas densas con activación (ReLU o GELU) aplicadas de manera independiente a cada posición de la secuencia.
- **Normalización de capa y conexiones residuales:** estabilizan el entrenamiento y facilitan el flujo de gradiente entre capas.

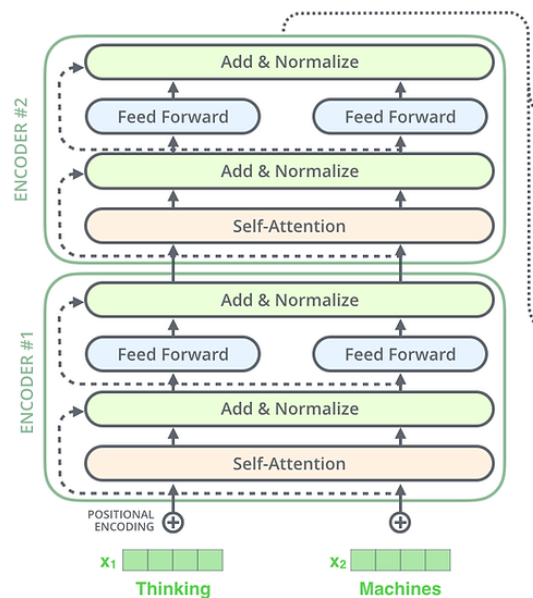


Figura 2.26: Estructura apilada de capas de **encoder** en el modelo Transformer. Cada capa contiene una subcapa de self-attention seguida de una red feed-forward, ambas con normalización y conexiones residuales. La entrada incluye embeddings sumados a codificaciones posicionales. El apilamiento de bloques permite construir representaciones jerárquicas cada vez más abstractas de la secuencia de entrada. [46]

- **Bloque de decoder.** El decoder añade componentes específicos a los del encoder:
 - **Atención enmascarada (Masked Self-Attention):** impide que cada posición vea información de pasos futuros, necesario para la generación autoregresiva.
 - **Atención cruzada (Cross-Attention):** permite que el decoder combine la información del encoder con sus propias representaciones parciales, integrando el contexto de entrada para generar la salida correcta.

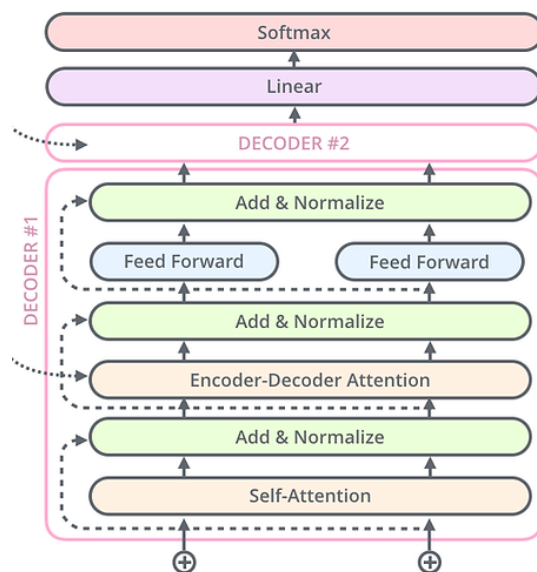


Figura 2.27: Estructura interna de un bloque de **decoder** en la arquitectura Transformer. Cada bloque incluye: una capa de **self-attention enmascarada**, una capa de **atención encoder-decoder** (cross-attention), una red feed-forward, conexiones residuales y normalización de capa. Esta estructura permite generar secuencias autoregresivas basadas tanto en la salida previa del decoder como en las representaciones del encoder. [46]

- **Codificación posicional.** Dado que el mecanismo de atención por sí solo no incorpora información sobre el orden de los elementos en la secuencia, es necesario añadir **codificaciones posicionales** a los embeddings de entrada. Estas codificaciones pueden ser **sinusoidales**, basadas en funciones trigonométricas fijas, o **aprendidas**, donde el modelo ajusta los vectores posicionales durante el entrenamiento. Existen además variantes más avanzadas, como las **codificaciones relativas** o las **codificaciones rotatorias (RoPE)**, que permiten representar relaciones de posición de manera más flexible y eficiente, mejorando la capacidad del Transformer para capturar dependencias relativas a lo largo de la secuencia.

El **funcionamiento general** de un modelo Transformer es el siguiente:

1. Los datos de entrada se tokenizan y transforman en *embeddings*, sumados a codificaciones posicionales.
2. Se procesan mediante una pila de N capas de encoder y/o decoder.
3. La capa de salida proyecta las representaciones al espacio de vocabulario (en NLP) o a la tarea específica.

En generación autoregresiva, se emplean técnicas como *beam search*, muestreo top- k , *nucleus sampling* (top- p) y el uso de cachés de claves y valores para acelerar la inferencia [43].

Según su uso, existen tres **configuraciones principales** de Transformer:

- **Encoder-only** (por ejemplo, BERT): está diseñado para tareas de comprensión de secuencias, como clasificación de texto, extracción de información o *question answering*.

- **Decoder-only** (por ejemplo, GPT): es una **arquitectura autoregresiva**, lo que significa que genera cada elemento de la secuencia de salida tomando como referencia los elementos previamente generados. Esto permite producir texto de manera coherente paso a paso.
- **Encoder-decoder** (por ejemplo, T5, BART): combina encoder y decoder, útil en tareas de transformación de secuencias, como traducción automática, resumen o *sequence-to-sequence*.

2.3.3.1 BERT y DistilBERT

BERT (Bidirectional Encoder Representations from Transformers) es un modelo de lenguaje propuesto por Devlin et al. (2019) que marcó un hito en el procesamiento del lenguaje natural. Su principal innovación fue el **preentrenamiento bidireccional**, gracias al cual es capaz de capturar el contexto de cada palabra considerando simultáneamente lo que aparece antes y después en la secuencia. Esto le permite generar representaciones semánticas más ricas que las obtenidas por modelos unidireccionales previos [47].

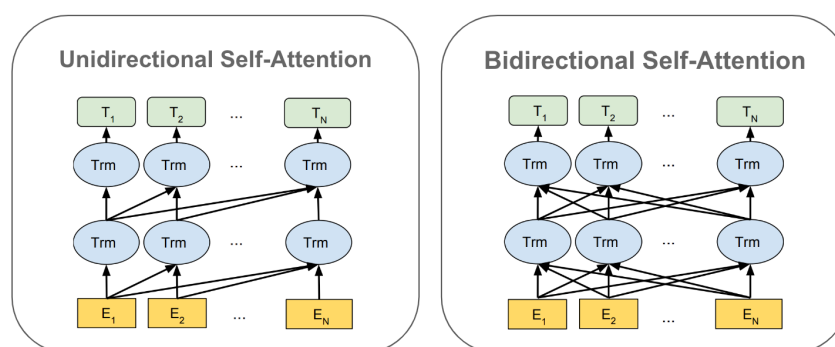


Figura 2.28: Comparación entre atención unidireccional y bidireccional. Mientras que los modelos tradicionales de lenguaje utilizan atención unidireccional (izquierda), BERT se basa en atención bidireccional (derecha), permitiendo que cada token incorpore información tanto del pasado como del futuro en la secuencia. Esto mejora considerablemente la calidad de las representaciones contextuales. [48]

BERT se basa únicamente en el **módulo codificador** del Transformer y se entrena en dos tareas generales:

- **Masked Language Modeling (MLM)**: consiste en ocultar aleatoriamente un porcentaje de las palabras de una oración (habitualmente el 15 %) y entrenar el modelo para predecirlas a partir del resto del contexto.

Esta tarea obliga a BERT a aprender representaciones distribuidas robustas, ya que la predicción se basa tanto en las palabras precedentes como en las siguientes. Una innovación clave es que no todas las palabras seleccionadas se sustituyen directamente por el token **[MASK]**: en un 80 % de los casos se reemplazan por **[MASK]**, en un 10 % se sustituyen por una palabra aleatoria y en el 10 % restante se mantienen sin cambios. Este procedimiento evita que el modelo dependa exclusivamente de la presencia del token **[MASK]** y mejora su capacidad de generalización en escenarios reales donde dicho token no aparece.

- **Next Sentence Prediction (NSP)**: busca modelar la relación semántica y discursiva entre pares de oraciones.

Durante el preentrenamiento, al modelo se le presentan dos segmentos de texto: en un 50 % de los casos, el segundo segmento corresponde efectivamente a la oración que sigue en el corpus, mientras que en el 50 % restante se trata de una oración seleccionada aleatoriamente.

El objetivo de BERT es clasificar si la segunda oración es consecutiva o no. Esta tarea se diseñó para dotar al modelo de nociones de coherencia textual y de relaciones entre frases, lo cual resulta útil en aplicaciones como respuesta a preguntas o inferencia de relaciones lógicas.

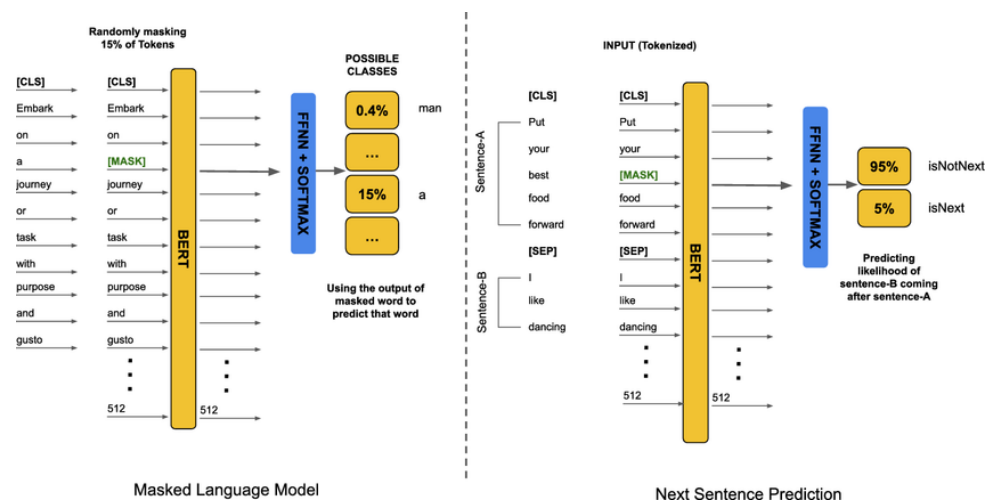


Figura 2.29: Tareas de preentrenamiento de BERT: **Masked Language Modeling (izquierda)** consiste en predecir palabras ocultas a partir del contexto circundante. **Next Sentence Prediction (derecha)** busca determinar si una oración sigue lógicamente a otra, ayudando a capturar relaciones discursivas. Ambas tareas permiten a BERT adquirir un entendimiento profundo del lenguaje antes del ajuste fino. [49]

Este esquema de preentrenamiento permite que, una vez entrenado, el modelo pueda adaptarse fácilmente a tareas específicas mediante un proceso de *fine-tuning*, en el que se añade una capa de salida adecuada y se ajustan los parámetros del modelo con un conjunto de datos anotados. Existen dos versiones habituales: *BERT-base* (12 capas, 110M de parámetros) y *BERT-large* (24 capas, 340M de parámetros).

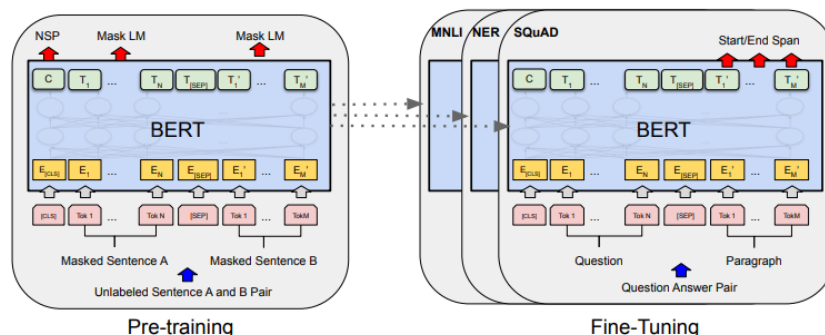


Figura 2.30: Esquema general del proceso de preentrenamiento y fine-tuning en BERT. Durante el preentrenamiento, el modelo aprende con tareas generales (MLM y NSP) sobre datos no etiquetados. Posteriormente, se adapta a tareas específicas (como clasificación, NER o QA) mediante fine-tuning sobre datos anotados. [50]

Si bien BERT logra resultados sobresalientes, su elevado número de parámetros lo hace costoso en términos de entrenamiento e inferencia, lo que limita su uso en entornos con recursos restringidos. Para responder a este problema, Hugging Face desarrolló en 2019 **DistilBERT**, una versión más ligera que conserva la mayor parte de la capacidad de BERT reduciendo de manera significativa su tamaño y coste computacional [44], [51].

DistilBERT se obtiene mediante la técnica de **distillation**, en la cual un modelo más pequeño (*estudiante*) aprende a imitar las salidas de un modelo grande previamente entrenado (*maestro*, en este caso BERT). La arquitectura de DistilBERT mantiene los mismos principios del encoder Transformer, pero con la mitad de capas (6 en lugar de 12) y prescindiendo de la tarea NSP en su entrenamiento. El resultado es un modelo con alrededor de un 40 % menos de parámetros, un 60 % más rápido en inferencia y con un rendimiento cercano al 97 % del alcanzado por BERT en benchmarks estándar.

Característica	BERT-base	DistilBERT
Número de capas	12	6
Parámetros	110M	66M
Entrenamiento	MLM + NSP	MLM + distillation
Rendimiento relativo	100 %	~97 %
Velocidad de inferencia	1x	1.6x más rápida

Tabla 2.3: Comparación entre BERT-base y DistilBERT en términos de arquitectura, entrenamiento y eficiencia. DistilBERT logra una reducción significativa en tamaño y tiempo de inferencia, manteniendo un rendimiento cercano al de BERT.

Así, BERT representa la versión completa y más precisa, especialmente adecuada para investigación y tareas donde la calidad es prioritaria, mientras que DistilBERT ofrece un equilibrio más favorable entre eficiencia y rendimiento, lo que lo convierte en una alternativa idónea para aplicaciones prácticas y despliegues en producción.

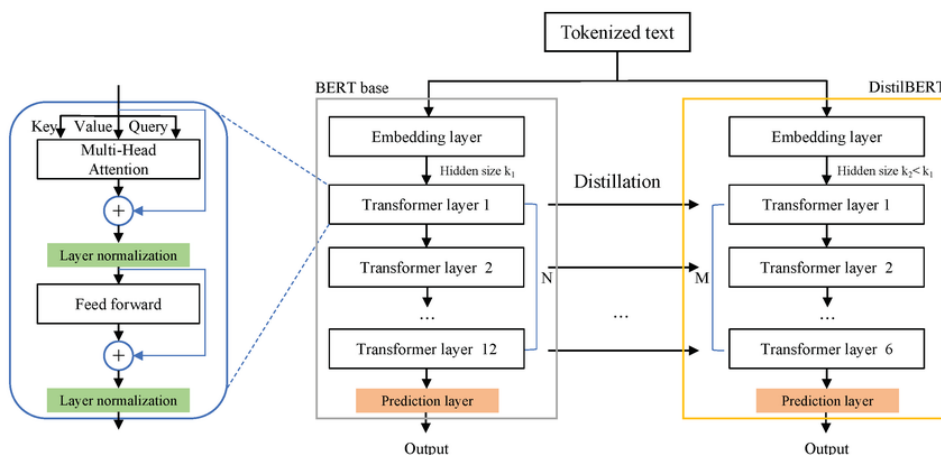


Figura 2.31: Proceso de distilación en DistilBERT. El modelo estudiante (derecha) aprende a imitar el comportamiento del modelo maestro BERT (izquierda), reduciendo el número de capas y eliminando la tarea NSP. Esto resulta en un modelo más ligero y eficiente, manteniendo un rendimiento competitivo. [52]

2.3.4. Entrenamiento, Validación y Evaluación de Modelos de Deep Learning

2.3.4.1 Entrenamiento

El entrenamiento de un modelo de *deep learning* consiste en ajustar sus parámetros internos, denominados pesos, para que las predicciones se aproximen a los valores reales de los datos. Este proceso requiere la definición de **hiperparámetros**, que controlan aspectos clave del aprendizaje [53]:

- **Tasa de aprendizaje (*learning rate*)**: determina el tamaño de los pasos que el modelo da al actualizar sus pesos durante la optimización. Una tasa demasiado alta puede provocar inestabilidad, mientras que una muy baja ralentiza el aprendizaje.

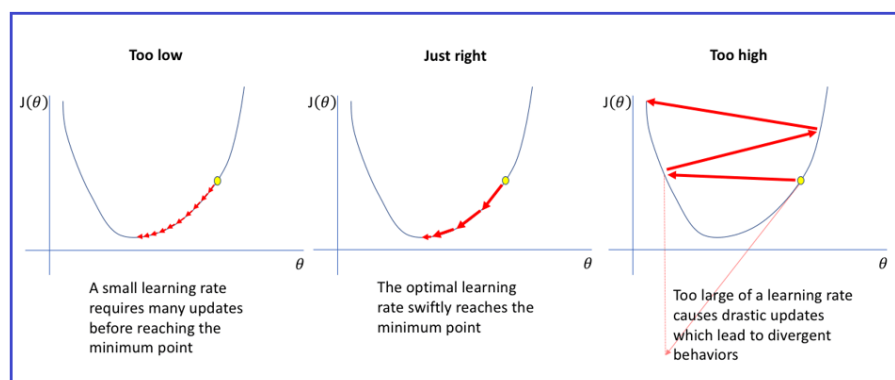


Figura 2.32: Efecto de diferentes tasas de aprendizaje en el descenso de gradiente: una tasa demasiado baja requiere muchas actualizaciones para converger, la tasa óptima alcanza el mínimo eficientemente, y una tasa demasiado alta provoca oscilaciones o divergencia del entrenamiento. [54]

- **Número de épocas (*epochs*)**: indica cuántas veces el modelo recorre todo el conjunto de datos de entrenamiento. Procesar los datos varias veces permite refinar los pesos y capturar patrones más complejos.
- **Tamaño del lote (*batch size*)**: especifica cuántos ejemplos se procesan antes de actualizar los pesos. Dividir los datos en *mini-batches* mejora la eficiencia computacional y la estabilidad del entrenamiento.

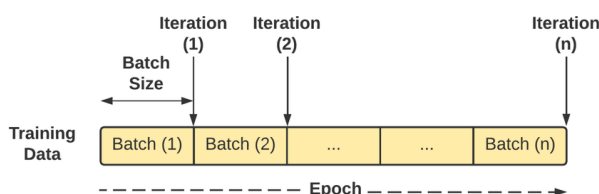


Figura 2.33: Representación del proceso de entrenamiento en redes neuronales: los datos de entrenamiento se dividen en lotes, cada lote corresponde a una iteración, y un ciclo completo sobre todos los lotes constituye una época. [55]

Durante cada mini-batch se calcula la **función de pérdida**, que cuantifica la discrepancia entre las predicciones y los valores reales. La función de pérdida guía la actualización de los pesos me-

diante algoritmos de optimización, permitiendo que el modelo aprenda progresivamente patrones relevantes en los datos.

La función de pérdida se selecciona según la tarea específica [56]:

■ **Regresión:**

• **Error cuadrático medio (MSE):**

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

Penaliza fuertemente errores grandes y fuerza al modelo a aproximarse estrechamente a los valores reales.

• **Error absoluto medio (MAE):**

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |\hat{y}_i - y_i|$$

Más robusto frente a *outliers*, aunque su derivada constante puede ralentizar la convergencia.

• **Huber loss:**

$$L_{\delta}(\hat{y}, y) = \begin{cases} \frac{1}{2}(\hat{y} - y)^2 & \text{si } |\hat{y} - y| \leq \delta \\ \delta \cdot |\hat{y} - y| - \frac{1}{2}\delta^2 & \text{si } |\hat{y} - y| > \delta \end{cases}$$

Combina las ventajas de MSE para errores pequeños y MAE para errores grandes.

■ **Clasificación:**

• **Entropía cruzada (Cross-Entropy) para clasificación multiclase:**

$$L = - \sum_{i=1}^C y_i \log(\hat{y}_i)$$

Penaliza fuertemente predicciones incorrectas con alta confianza.

• **Hinge loss (para SVM):**

$$L = \sum_{i=1}^n \max(0, 1 - y_i \hat{y}_i)$$

Penaliza errores en función de la distancia al margen de decisión.

• **Focal loss:**

$$L = - \sum_{i=1}^n (1 - \hat{y}_i)^{\gamma} y_i \log(\hat{y}_i)$$

Con $\gamma > 0$ aumenta el peso de ejemplos difíciles y reduce la influencia de ejemplos ya correctamente clasificados.

■ **Otras funciones:**

• **Kullback-Leibler Divergence (KL Divergence):**

$$D_{\text{KL}}(P||Q) = \sum_i P(i) \log \frac{P(i)}{Q(i)}$$

Mide la divergencia entre la distribución de probabilidad P y Q .

- **Cosine similarity loss:**

$$L = 1 - \frac{\hat{y} \cdot y}{\|\hat{y}\| \|y\|}$$

Evalúa la similitud angular entre vectores, útil en NLP o sistemas de recomendación.

El ajuste de los pesos se realiza mediante algoritmos de optimización [53]:

- **Gradient Descent y variantes:**

- *Batch Gradient Descent*: calcula el gradiente usando todo el conjunto de entrenamiento antes de actualizar los pesos. Es preciso, pero computacionalmente costoso y poco práctico para grandes conjuntos de datos.
- *Stochastic Gradient Descent (SGD)*: actualiza los pesos tras cada ejemplo, introduciendo ruido que puede ayudar al modelo a escapar de mínimos locales, pero haciendo la convergencia más oscilante.
- *Mini-batch SGD*: actualiza los pesos tras procesar pequeños lotes de datos. Combina estabilidad y eficiencia, siendo el enfoque más común en deep learning.

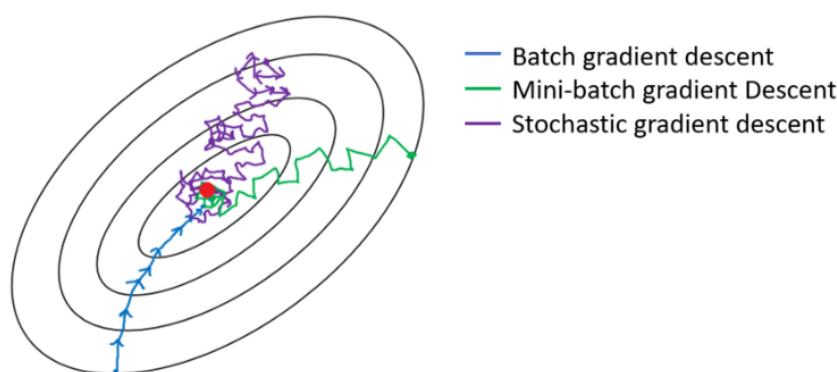


Figura 2.34: Trayectorias de optimización hacia el mínimo en el espacio de parámetros: Batch Gradient Descent (azul) sigue un camino estable y directo, Mini-batch Gradient Descent (verde) combina estabilidad con algo de variabilidad, y Stochastic Gradient Descent (morado) muestra un recorrido más errático debido a su alta varianza en cada actualización. [57]

- **Optimizadores avanzados:**

- *Momentum*: acumula un promedio ponderado de gradientes anteriores para suavizar la dirección de actualización, acelerando la convergencia y evitando oscilaciones.
- *Nesterov Accelerated Gradient (NAG)*: mejora Momentum calculando el gradiente en la posición anticipada, ofreciendo convergencia más precisa y rápida.
- *AdaGrad*: adapta la tasa de aprendizaje para cada parámetro según la frecuencia de actualización. Esto permite a parámetros poco frecuentes tener pasos de actualización mayores, útil en datos dispersos.
- *RMSProp*: corrige AdaGrad evitando que la tasa de aprendizaje disminuya demasiado rápido, estabilizando el entrenamiento en redes profundas y recurrentes.
- *Adam (Adaptive Moment Estimation)*: combina Momentum y RMSProp, adaptando la tasa de aprendizaje para cada parámetro y acumulando el promedio de gradientes y de su cuadrado. Es ampliamente usado por su estabilidad, rapidez y eficacia en gran variedad de problemas.

2.3.4.2 Overfitting

Cuando un modelo aprende los datos de entrenamiento demasiado bien, perdiendo su capacidad generalizadora y mostrando un desempeño reducido en datos nuevos, se da un fenómeno denominado overfitting. Este se debe al ruido en los datos, datasets de entrenamiento demasiado reducidos y modelos demasiado complejos.

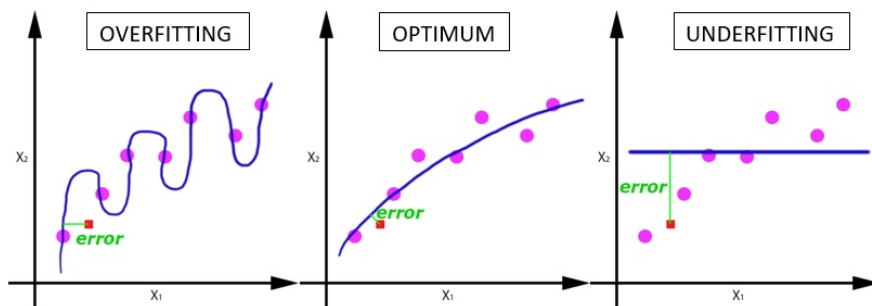


Figura 2.35: Ilustración de tres escenarios de ajuste de un modelo: sobreajuste (overfitting), donde el modelo se adapta excesivamente al ruido de los datos; ajuste óptimo, que capta correctamente la tendencia subyacente; y subajuste (underfitting), donde el modelo es demasiado simple para representar la relación entre variables. [58]

Existen diversas técnicas cuyo objetivo es mitigar los efectos del overfitting y evitar que el modelo se ajuste en exceso a los datos de entrenamiento [59] .

- **Early-Stopping:** Detener el entrenamiento en el momento en el que el desempeño en un set de validación comienza a degradarse. Para ello se monitoriza el desempeño de la validación tras cada época de entrenamiento. Cuando el error en el set de validación deja de reducirse, conviene interrumpir el entrenamiento.
- **Network Reduction:** Consiste en limitar o reducir la complejidad del modelo, reduciendo el número de parámetros (p. ej. disminuyendo el número de capas o neuronas) para evitar capturar el ruido. Esto puede implicar técnicas como la poda de neuronas (pruning), la congelación de capas innecesarias (freezing) o el uso de arquitecturas más sencillas.
- **Data Augmentation:** Aumentar el tamaño y la diversidad del conjunto de datos de entrenamiento permite que el modelo aprenda a generalizar mejor. En el caso de imágenes, esto puede lograrse mediante técnicas de data augmentation, como el volteo (flipping), rotación (rotating) o recorte (cropping), con el objetivo de sintetizar nuevos ejemplos a partir de los datos existentes.

Otras estrategias incluyen la incorporación de ruido aleatorio a los datos originales, la reobtención de muestras mediante procesamientos específicos, o la generación de nuevos datos siguiendo la distribución estadística del conjunto original.

- **Regularización:**

Consiste en añadir términos de penalización a la función de pérdida con el objetivo de evitar que el modelo adquiriera una complejidad excesiva y se sobreajuste a los datos de entrenamiento.

- **Regularización L1 (Lasso Regression):** Añade la suma de los valores absolutos de los pesos a la función de pérdida, fomentando la esparsidad (es decir, algunos pesos se reducen a cero).

La función de pérdida regularizada mediante L1 se define como:

$$\mathcal{L}_{\text{reg}}(\omega) = \mathcal{L}_{\text{original}}(\omega) + \lambda \sum_j |\omega_j| \quad (2.10)$$

donde:

- $\mathcal{L}_{\text{reg}}(\omega)$ es la nueva función de pérdida regularizada,
- $\mathcal{L}_{\text{original}}(\omega)$ es la función de pérdida original del modelo (por ejemplo, entropía cruzada en clasificación),
- λ es un hiperparámetro que controla la intensidad de la regularización,
- ω_j representa los pesos del modelo; el término $\sum_j |\omega_j|$ corresponde a la norma L1 de los pesos.

- **Regularización L2 (Ridge Regression o Weight Decay):** Añade la suma de los cuadrados de los pesos a la función de pérdida, penalizando los pesos grandes y favoreciendo soluciones con valores más pequeños y distribuidos.

La función de pérdida con regularización L2 se expresa como:

$$\mathcal{L}_{\text{reg}}(\omega) = \mathcal{L}_{\text{original}}(\omega) + \lambda \sum_j \omega_j^2 \quad (2.11)$$

donde:

- $\mathcal{L}_{\text{reg}}(\omega)$ es la función de pérdida regularizada,
- $\mathcal{L}_{\text{original}}(\omega)$ representa la función de pérdida original,
- λ es un hiperparámetro que controla el grado de penalización,
- ω_j son los pesos del modelo, y el término $\sum_j \omega_j^2$ corresponde a la norma L2 o norma Euclídea.

A diferencia de L1, L2 no fuerza los pesos a cero, sino que tiende a suavizarlos. Esto ayuda a reducir la varianza del modelo y mejora la generalización, manteniendo todos los parámetros activos.

- **Elastic Net:** Combina las penalizaciones L1 y L2, aprovechando las ventajas de ambas. Es especialmente útil cuando hay multicolinealidad entre variables o cuando el número de características supera al número de observaciones.

La función de pérdida regularizada con Elastic Net se define como:

$$\mathcal{L}_{\text{reg}}(\omega) = \mathcal{L}_{\text{original}}(\omega) + \lambda_1 \sum_j |\omega_j| + \lambda_2 \sum_j \omega_j^2 \quad (2.12)$$

donde:

- λ_1 controla la penalización L1,
- λ_2 controla la penalización L2,
- Los demás términos tienen el mismo significado que en los casos anteriores.

- **Dropout:** Es una técnica de regularización específica para redes neuronales, que consiste en desactivar aleatoriamente un subconjunto de neuronas durante el entrenamiento. Esto evita que las neuronas se co-adapten excesivamente, promoviendo una representación más robusta y generalizable.

La salida modificada de cada neurona i se define como:

$$\tilde{h}_i = h_i \cdot z_i, \quad z_i \sim \text{Bernoulli}(p) \quad (2.13)$$

donde:

- h_i es la salida original de la neurona,
- $\tilde{h}_i$ es la salida tras aplicar *dropout*,
- z_i es una variable aleatoria con distribución Bernoulli de parámetro p .

En la fase de inferencia (evaluación), no se aplica *dropout*; en su lugar, las salidas de las neuronas se escalan por el factor p para mantener la coherencia con la fase de entrenamiento.

2.3.4.3 Validación

La validación permite estimar la capacidad de generalización del modelo y ajustar hiperparámetros sin afectar sobre el conjunto de prueba. Un enfoque básico es la **validación simple (hold-out)**, que separa los datos en conjuntos de entrenamiento, validación y prueba. Sin embargo, este método puede ser sensible a la partición de los datos, especialmente en conjuntos pequeños.

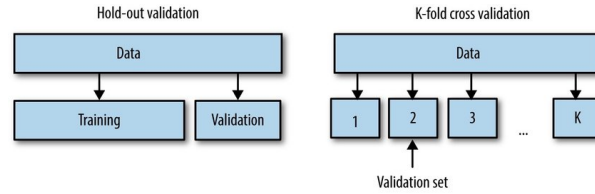


Figura 2.36: Comparación de dos estrategias de validación de modelos: en Hold-out se divide el conjunto de datos en entrenamiento y validación una sola vez; en K-fold Cross Validation los datos se particionan en K subconjuntos, rotando el subconjunto de validación en cada iteración. [60]

Para mayor robustez se emplea la **validación cruzada (cross-validation)**, donde el conjunto de datos se divide en k particiones (*folds*). El modelo se entrena k veces, utilizando $k - 1$ folds para entrenamiento y uno diferente para validación en cada iteración. Se pueden utilizar variantes como la *Stratified K-fold* para mantener la proporción de clases, o *Leave-One-Out* para evaluar cada ejemplo individualmente [61].

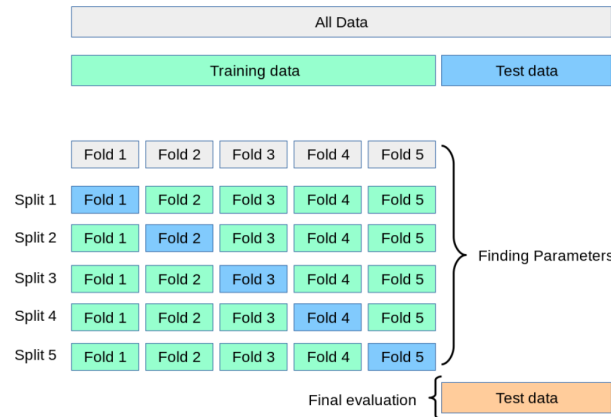


Figura 2.37: Esquema de validación cruzada anidada (Nested Cross-Validation): el conjunto total de datos se divide en subconjuntos (folds). En cada división, un fold actúa como validación mientras los demás se usan para entrenar, repitiendo el proceso en cada iteración para ajustar parámetros. Finalmente, el rendimiento del modelo se evalúa con un conjunto de prueba independiente. [62]

En series temporales, se aplican técnicas de **validación temporal**, que respetan el orden de los datos mediante ventanas deslizantes (*sliding window*) o expansivas (*expanding window*), entrenando con datos pasados y validando con datos futuros [63].

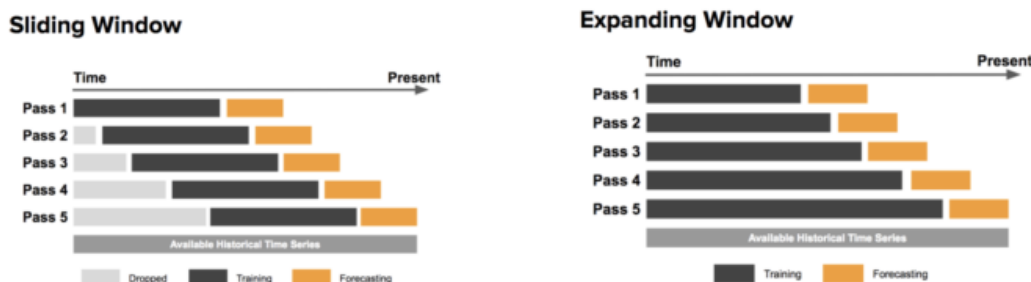


Figura 2.38: Comparación de estrategias de validación temporal: en Sliding Window el tamaño de la ventana de entrenamiento se mantiene constante desplazándose en el tiempo, mientras que en Expanding Window la ventana de entrenamiento crece incorporando nuevos datos históricos en cada iteración, antes de realizar la predicción. [64]

2.3.4.4 Evaluación

Tras las fases de entrenamiento y validación, el modelo se somete a evaluación sobre un **conjunto de prueba** independiente. Su propósito es estimar el rendimiento final y, en particular, la capacidad de generalización frente a datos no observados previamente. La elección de métricas depende de la naturaleza de la tarea.

Por un lado, en problemas de **regresión** se busca cuantificar la discrepancia entre los valores predichos ($\hat{y}_i$) y los reales (y_i), siendo n el número de observaciones. Las métricas más comunes son [65]:

- **Error cuadrático medio (MSE):**

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Evalúa la magnitud media del error al elevar al cuadrado las diferencias, lo que penaliza con mayor severidad los errores grandes. Es útil cuando es crítico evitar desviaciones extremas, pero puede ser poco robusto ante valores atípicos.

- **Raíz del error cuadrático medio (RMSE):**

$$RMSE = \sqrt{MSE}$$

Tiene la misma interpretación que el MSE, pero expresada en las **mismas unidades** que la variable objetivo, lo que facilita la comprensión práctica. Permite comparar directamente el error con el rango de los valores reales.

- **Error absoluto medio (MAE):**

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Calcula el error promedio sin elevarlo al cuadrado, por lo que es **menos sensible a valores atípicos** que el MSE. Es una métrica más robusta cuando la distribución de errores es asimétrica.

- **Coefficiente de determinación (R^2):**

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

Representa la proporción de varianza de la variable objetivo explicada por el modelo. Un R^2 cercano a 1 indica un modelo con buen ajuste, valores próximos a 0 indican que el modelo apenas mejora sobre predecir la media, y valores negativos reflejan que el modelo es peor que un predictor trivial.

- **Error porcentual absoluto medio (MAPE):**

$$MAPE = \frac{100}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right|$$

Expresa el error relativo en porcentaje, lo que lo hace fácil de interpretar. Sin embargo, puede ser **inestable** cuando y_i se aproxima a cero, ya que el denominador puede inflar el valor.

- **Error absoluto mediano (MedAE):**

$$MedAE = \text{mediana}(|y_i - \hat{y}_i|)$$

Resume la **mediana** de los errores absolutos, lo que lo hace altamente **robusto frente a valores extremos** y más representativo de la tendencia central del error.

Por otro lado, en **clasificación** se emplean métricas derivadas de la **matriz de confusión**, formada por los verdaderos positivos (TP), falsos positivos (FP), verdaderos negativos (TN) y falsos negativos (FN) [66].

- **Exactitud (accuracy):**

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Representa la proporción de predicciones correctas sobre el total. Es sencilla, pero puede ser **engañosa en datasets desbalanceados**.

- **Precisión (precision):**

$$Precision = \frac{TP}{TP + FP}$$

Indica cuántas de las predicciones positivas realmente lo son. Es clave cuando el **costo de los falsos positivos es alto**, como en detección de fraudes.

- **Exhaustividad (recall):**

$$Recall = \frac{TP}{TP + FN}$$

Mide la capacidad del modelo para detectar correctamente todas las instancias positivas. Es crucial cuando el **costo de los falsos negativos es elevado**, como en diagnósticos médicos.

- **F1-score:**

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

Combina precisión y exhaustividad en una sola métrica. Es particularmente útil en **problemas con clases desbalanceadas**, donde accuracy puede resultar poco representativa.

- **Exactitud balanceada (balanced accuracy):**

$$\text{Balanced Accuracy} = \frac{1}{2} \left(\frac{TP}{TP + FN} + \frac{TN}{TN + FP} \right)$$

Ajusta la exactitud clásica considerando el **tamaño relativo de cada clase**, evitando sesgos hacia la clase mayoritaria.

- **Kappa de Cohen:**

$$\kappa = \frac{p_o - p_e}{1 - p_e}$$

Compara el rendimiento real del modelo (p_o) con el esperado por azar (p_e). Un $\kappa = 1$ indica acuerdo perfecto, $\kappa = 0$ indica acuerdo aleatorio y valores negativos reflejan desacuerdo sistemático.

- **Coefficiente de correlación de Matthews (MCC):**

$$MCC = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}$$

Es considerada una de las métricas más **robustas** para clasificación binaria, pues tiene en cuenta todos los elementos de la matriz de confusión y funciona bien incluso con clases muy desbalanceadas.

- **Pérdida logarítmica (log-loss):**

$$\text{LogLoss} = -\frac{1}{n} \sum_{i=1}^n \sum_{c=1}^C y_{i,c} \log(\hat{p}_{i,c})$$

Evalúa la calidad de las **predicciones probabilísticas** del modelo. Penaliza más severamente las predicciones erróneas con alta confianza.

En contextos **multietiqueta**, cada instancia i puede tener múltiples etiquetas verdaderas Y_i y un conjunto predicho $\hat{Y}_i$. Las métricas específicas incluyen:

- **Exact Match Ratio (EMR):**

$$EMR = \frac{1}{n} \sum_{i=1}^n \mathbb{I}[Y_i = \hat{Y}_i]$$

Calcula el porcentaje de instancias cuyas etiquetas se predicen **exactamente**. Es una métrica muy estricta.

- **Pérdida de Hamming:**

$$\text{Hamming Loss} = \frac{1}{n \cdot L} \sum_{i=1}^n \sum_{j=1}^L \mathbb{I}[y_{ij} \neq \hat{y}_{ij}]$$

Mide el **error promedio por etiqueta**, lo que la hace más flexible que EMR.

- **Label Ranking Loss:**

$$LR \text{ Loss} = \frac{1}{n} \sum_{i=1}^n \frac{1}{|Y_i| \cdot |\bar{Y}_i|} \left| \{(y^+, y^-) : \hat{f}_i(y^+) \leq \hat{f}_i(y^-)\} \right|$$

Evalúa cuántas veces el modelo ordena incorrectamente etiquetas relevantes e irrelevantes. Es esencial en tareas de **recomendación**.

- **Error de cobertura:**

$$Coverage\ Error = \frac{1}{n} \sum_{i=1}^n \left(\max_{y \in Y_i} rank_i(y) - 1 \right)$$

Indica cuántas etiquetas del ranking deben revisarse para incluir todas las verdaderas. Cuanto menor, mejor.

- **Precisión media (Average Precision, AP):**

$$AP = \frac{1}{|Y_i|} \sum_{k=1}^L P(k) \cdot rel(k)$$

Resume la **precisión acumulada** en distintas posiciones del ranking y es muy usada en motores de búsqueda y recuperación de información.

Sin embargo, la evaluación del desempeño de un modelo no solo puede realizarse mediante métricas numéricas, sino también mediante representaciones gráficas que facilitan la interpretación de los resultados y permiten detectar patrones que las cifras por sí solas no muestran. Entre las más utilizadas se incluyen:

- **Matriz de confusión:** visualiza la relación entre las predicciones del modelo y las clases reales, mostrando los aciertos y errores. Las celdas diagonales representan las predicciones correctas (verdaderos positivos y verdaderos negativos), mientras que las celdas fuera de la diagonal indican errores (falsos positivos y falsos negativos). Este gráfico permite identificar qué clases se confunden con mayor frecuencia y detectar posibles problemas de desbalance en los datos.

		Ground truth		
		+	-	
Predicted	+	True positive (TP)	False positive (FP)	Precision = TP / (TP + FP)
	-	False negative (FN)	True negative (TN)	
		Recall = TP / (TP + FN)		Accuracy = (TP + TN) / (TP + FP + TN + FN)

Figura 2.39: Matriz de confusión para la evaluación de modelos de clasificación binaria. Se muestran las categorías de verdaderos positivos (TP), falsos positivos (FP), falsos negativos (FN) y verdaderos negativos (TN), junto con las fórmulas de métricas clave: precisión, exhaustividad (recall) y exactitud (accuracy). [67]

- **Curvas ROC (Receiver Operating Characteristic) y AUC (Área Bajo la Curva):** la curva ROC grafica la tasa de verdaderos positivos frente a la tasa de falsos positivos para distintos umbrales de decisión. Una curva cercana a la esquina superior izquierda indica buena capacidad de discriminación.

El área bajo la curva (AUC) resume esta capacidad en un solo valor: un AUC cercano a 1 refleja un excelente desempeño, mientras que un valor próximo a 0.5 indica que el modelo discrimina de manera similar al azar.

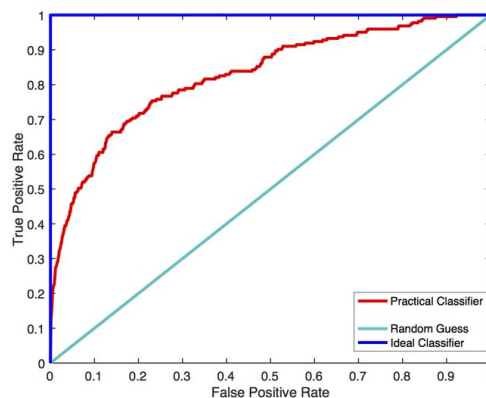


Figura 2.40: Curva ROC (Receiver Operating Characteristic) para evaluar el rendimiento de un clasificador. El eje X representa la tasa de falsos positivos (False Positive Rate) y el eje Y la tasa de verdaderos positivos (True Positive Rate). La línea azul muestra el clasificador ideal, la línea gris indica el comportamiento de un clasificador aleatorio, y la curva roja corresponde al clasificador práctico evaluado. [68]

- **Curvas precisión–recall (PR curve):** muestran cómo varía la precisión del modelo en función de la exhaustividad (recall) para distintos umbrales. Son especialmente útiles en problemas con clases desbalanceadas, ya que permiten evaluar el compromiso entre detectar correctamente todos los positivos y minimizar los falsos positivos.
- **Diagramas de errores (residual plots):** comparan los valores predichos frente a los observados, siendo particularmente útiles en tareas de regresión. Permiten identificar sesgos o patrones sistemáticos en los errores, como tendencias lineales o curvaturas, que pueden indicar que el modelo no captura adecuadamente la relación entre variables.
- **Gráficos de calibración (reliability diagrams):** contrastan las probabilidades predichas por el modelo con las frecuencias observadas de ocurrencia de la clase positiva. Un modelo bien calibrado asigna probabilidades coherentes con la realidad; por ejemplo, si predice un 80 % de probabilidad, aproximadamente el 80 % de esos casos deberían pertenecer efectivamente a la clase positiva. Esta representación es útil para evaluar modelos probabilísticos y para ajustar la confianza de las predicciones.

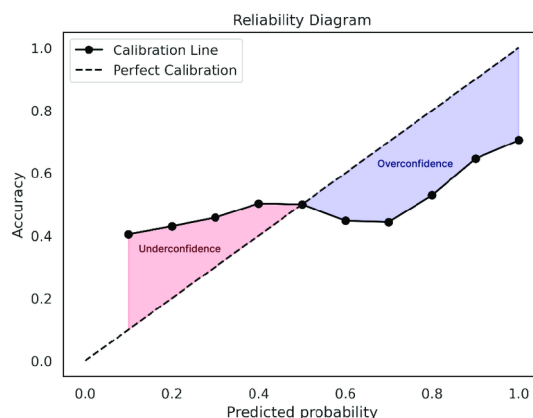


Figura 2.41: Diagrama de fiabilidad utilizado para evaluar la calibración de un modelo de clasificación. La línea discontinua representa la calibración perfecta, mientras que la línea negra muestra la calibración real del modelo. Las áreas sombreadas indican zonas de subconfianza (underconfidence) y sobreconfianza (overconfidence) según la discrepancia entre la probabilidad predicha y la precisión observada. [69]

2.3.5. Explicabilidad

Uno de los principales desafíos éticos de la inteligencia artificial IA es su falta de transparencia, especialmente en modelos complejos que funcionan como una caja negra. La explicabilidad hace referencia a la capacidad de comprender, interpretar y justificar las decisiones específicas que toma un sistema de IA, de forma que sea accesible también para personas no expertas.

- **LIME (Local Interpretable Model-agnostic Explanations)**: LIME es un método diseñado para explicar predicciones individuales de modelos complejos tratándolos como una *caja negra*. Su principio fundamental consiste en aproximar el comportamiento del modelo original f en una **vecindad local** alrededor de una instancia específica mediante un modelo interpretable y sencillo g , como una regresión lineal o un clasificador de baja complejidad [70].

Para lograrlo, LIME genera múltiples perturbaciones del dato original, obtiene las predicciones del modelo para estas variantes y ajusta un modelo interpretable ponderando las muestras en función de su cercanía al punto de interés. Los coeficientes de este modelo simplificado indican la contribución relativa de cada característica a la predicción local. Aunque es flexible y agnóstico al tipo de modelo, LIME puede presentar **inestabilidad** si se cambia la forma de generar perturbaciones o la definición de vecindad, y sus explicaciones pueden variar entre ejecuciones. Es más útil para entender **predicciones individuales** que para describir el comportamiento global del sistema.

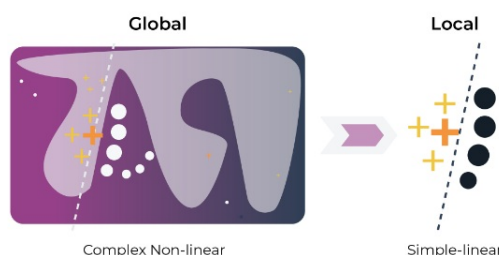


Figura 2.42: Ilustración conceptual del método LIME. El modelo global, complejo y no lineal, se aproxima localmente mediante un modelo lineal interpretable en torno a una instancia concreta. De esta manera, se obtiene una explicación sencilla y comprensible de cómo el modelo toma decisiones para un caso específico. [71]

- **SHAP (SHapley Additive exPlanations)**: SHAP se basa en los principios de la **teoría de juegos cooperativos** para asignar un valor de importancia a cada característica de entrada en la predicción de un modelo. Cada característica se considera un “jugador” que contribuye a un “juego” cuyo resultado es la predicción, y la contribución de cada jugador se calcula mediante los *valores de Shapley*, que representan la ganancia marginal promedio de incluir dicha característica en todas las combinaciones posibles [72].

Este enfoque permite obtener explicaciones **locales** (para predicciones individuales) y **globales** (promediando contribuciones a nivel de todo el dataset). Sin embargo, el cálculo exacto de los valores de Shapley puede ser computacionalmente costoso, especialmente con muchas características, por lo que se suelen utilizar aproximaciones como *Kernel SHAP* (agnóstica al modelo) o *TreeSHAP* (eficiente para modelos basados en árboles). SHAP es especialmente valioso en contextos donde la **interpretabilidad robusta** es crítica, ya que produce explicaciones coherentes, aditivas y comparables entre diferentes instancias [73].

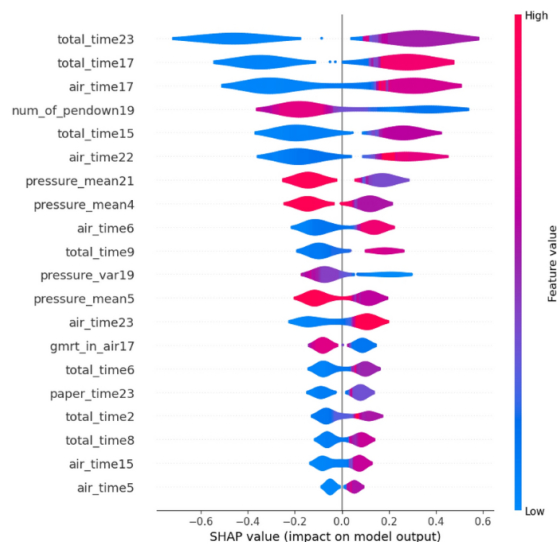


Figura 2.43: Gráfico de resumen SHAP que muestra la contribución de cada variable al resultado del modelo. El eje horizontal representa el impacto de cada característica sobre la predicción, mientras que el color indica el valor de la variable (azul = bajo, rojo = alto). Las variables más influyentes aparecen en la parte superior, permitiendo identificar de manera visual qué factores condicionan más las decisiones del modelo. [74]

- **Visualización de mapas de atención:** En modelos basados en *Transformers* los mecanismos de atención permiten que cada token de entrada “observe” a otros tokens y asigne pesos que determinan cuánta información recibe de ellos al generar representaciones internas o predicciones. Los **mapas de atención** visualizan estos pesos, mostrando sobre qué partes de la entrada se concentra el modelo en cada paso del procesamiento.

Esta técnica ofrece una perspectiva intuitiva sobre el **razonamiento interno** del modelo, ayudando a detectar patrones de alineamiento, dependencias semánticas y posibles sesgos. Sin embargo, los mapas de atención deben interpretarse con cautela: la magnitud de un peso de atención no siempre equivale a la **importancia causal** de un token en la predicción final, ya que capas posteriores pueden transformar la información de manera no lineal. Pese a estas limitaciones, la visualización de atención es una herramienta muy útil para explorar el funcionamiento de los Transformers y analizar cómo distribuyen su “foco” al procesar secuencias complejas.

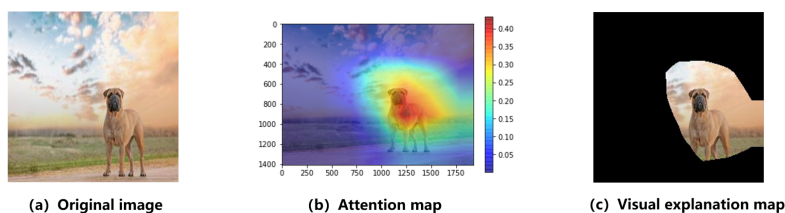


Figura 2.44: Visualización de mapas de atención generados por un modelo basado en transformers: (a) imagen original, (b) mapa de atención que resalta las regiones de la imagen más relevantes para la predicción, y (c) mapa de explicación visual que muestra explícitamente la zona de interés identificada por el modelo. [75]

- **Captum:** Captum es una biblioteca de interpretabilidad diseñada para modelos basados en PyTorch que proporciona un conjunto de herramientas para analizar y explicar el comportamiento de redes neuronales profundas. Su objetivo es estimar la **contribución de**

cada componente de la entrada (por ejemplo, cada token o característica) a la salida del modelo, permitiendo comprender qué elementos influyen más en las predicciones.

Entre las técnicas más destacadas que ofrece Captum se encuentran **Saliency** e **Integrated Gradients**, que utilizan información derivada de los gradientes del modelo para cuantificar la importancia de cada entrada.

- **Saliency**: Esta técnica calcula el gradiente de la salida del modelo respecto a cada entrada, evaluando cómo cambios infinitesimales en un componente particular afectan la predicción. El resultado es un **mapa de sensibilidad**, donde los valores absolutos más altos indican que la predicción es especialmente sensible a esa característica. En tareas de procesamiento de lenguaje natural, esto permite identificar qué palabras o subpalabras tienen mayor influencia en la decisión del modelo para una clase determinada. Saliency es eficiente de calcular y proporciona una visión inmediata de los factores más relevantes, pero puede ser ruidosa o saturarse en redes profundas, por lo que a veces se combina con técnicas de suavizado como SmoothGrad.
- **Integrated Gradients**: Integrated Gradients busca superar algunas limitaciones de Saliency, como la saturación de gradientes, proporcionando atribuciones más estables y coherentes. La idea es comparar la entrada real con un *baseline* de referencia (por ejemplo, un vector nulo o promedio) y acumular los gradientes a lo largo de un camino lineal que conecta ambos puntos. Esto genera una medida de importancia que refleja la contribución acumulada de cada característica desde un estado de referencia hasta la entrada real. Integrated Gradients cumple propiedades teóricas deseables, como sensibilidad y consistencia, y suele ser más robusta frente a redes profundas y activaciones no lineales.

Legend: ■ Negative □ Neutral ■ Positive

True Label	Predicted Label	Attribution Label	Attribution Score	Word Importance
pos	pos (0.96)	pos	1.29	it was a fantastic performance ! #pad
pos	pos (0.87)	pos	1.56	best film ever #pad #pad #pad
pos	pos (0.92)	pos	1.14	such a great show ! #pad #pad
neg	neg (0.29)	pos	-1.11	it was a horrible movie #pad #pad
neg	neg (0.22)	pos	-1.03	i 've never watched something as bad
neg	neg (0.07)	pos	-0.84	that is a terrible movie . #pad

Figura 2.45: Ejemplo de interpretabilidad local mediante el método Integrated Gradients, implementado en Captum. Se representan las frases con etiquetas reales y predichas junto con la puntuación de atribución de cada palabra. Los términos que contribuyen positivamente a la predicción se resaltan en verde, mientras que los que influyen negativamente aparecen en rojo, lo que permite comprender qué fragmentos del texto guían la decisión del modelo. [76]

2.4. Tecnologías web

El desarrollo de aplicaciones web se suele apoyar en una arquitectura de tipo cliente-servidor, que separa la interfaz de usuario y la lógica del sistema.

2.4.1. Arquitectura cliente-servidor

La arquitectura cliente-servidor es un modelo ampliamente adoptado en el ámbito del desarrollo web. En este esquema, el cliente actúa como interfaz de interacción con el usuario, realizando peticiones al servidor, que se encarga de procesar las peticiones, ejecutar la lógica de negocio y gestionar el acceso a fuentes de datos o servicios inteligentes, como modelos de Machine Learning.

Este modelo de comunicación estructurada se basa en protocolos como HTTP y promueve la reutilización de recursos, el desacoplamiento funcional y una gestión más segura de la información sensible, ya que esta permanece en el entorno del servidor.

2.4.2. Frontend

El frontend, o capa de presentación, es la parte visible de la aplicación con la que interactúa directamente el usuario. Se construye con tecnologías como HTML, CSS y JavaScript, y se ejecuta en el navegador del cliente. Su función principal es proporcionar una interfaz gráfica intuitiva, enviar solicitudes al servidor y mostrar las respuestas de forma comprensible.

2.4.3. Backend

El backend, o capa de lógica de negocio, opera en el servidor y es responsable de procesar las solicitudes provenientes del frontend. Incluye tareas como el acceso a bases de datos, la ejecución de algoritmos, el control de flujos de información y la gestión de seguridad. En este proyecto, el backend ha sido implementado utilizando el framework Flask, lo que permite una integración eficiente con modelos de aprendizaje automático.

2.4.4. APIs RESTful

Las APIs RESTful constituyen un estilo arquitectónico para la construcción de servicios web que facilitan la comunicación entre sistemas distribuidos. Basadas en el protocolo HTTP, permiten la manipulación de recursos mediante métodos estandarizados como GET, POST, PUT y DELETE, identificando cada recurso a través de una URL única.

Estas interfaces son, por diseño, sin estado (stateless), lo que implica que cada petición contiene toda la información necesaria para ser procesada. Este enfoque mejora la escalabilidad y la interoperabilidad entre distintos sistemas. En el contexto de aplicaciones web, las APIs RESTful permiten desacoplar el frontend del backend y facilitar la integración con servicios externos.

2.5. Tumblr

Tumblr es una plataforma de microblogueo y red social que permite a los usuarios publicar y compartir diversos tipos de contenido, como texto, imágenes, vídeos y enlaces, en un formato altamente personalizable. Su estructura se basa en el reblogueo, lo que fomenta la difusión de contenido dentro de comunidades creativas y subculturas digitales. Ha sido especialmente relevante como espacio de expresión juvenil, artística y activista. Además, Tumblr ofrece una API pública que facilita el acceso programático a sus datos, habilitando aplicaciones de análisis y desarrollo externo.

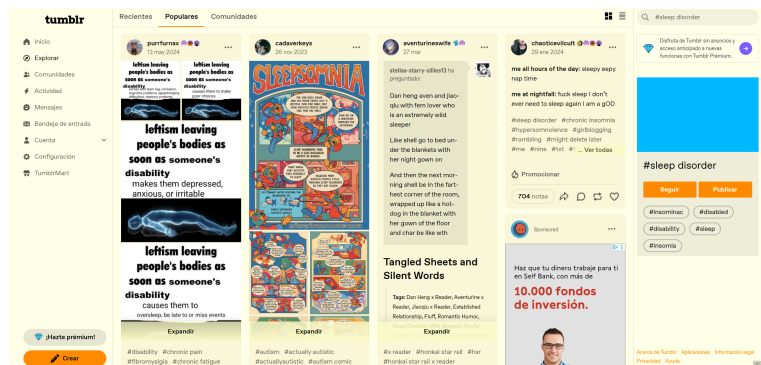


Figura 2.46: Ejemplo de búsqueda por etiqueta en Tumblr

2.5.1. API de Tumblr

La API de Tumblr es una interfaz que permite a los desarrolladores acceder y gestionar contenido en la plataforma de forma programática. Se basa en el protocolo HTTP y utiliza el formato JSON para el intercambio de datos, mientras que la autenticación se realiza mediante OAuth para garantizar la seguridad.

A través de sus *endpoints*, es posible obtener publicaciones, filtrarlas por tipo o etiquetas y realizar búsquedas por etiquetas o comunidades.

2.6. Reddit

Reddit es una plataforma de agregación de contenidos y discusión social que funciona como un conjunto de foros organizados en comunidades temáticas llamadas *subreddits*. Cada *subreddit* está dedicado a un tema específico, lo que permite a los usuarios publicar enlaces, imágenes, texto o encuestas, y participar en discusiones. Reddit se ha consolidado como un espacio diverso y altamente dinámico para el intercambio de información, noticias y experiencias personales.

2.6.1. API de Reddit

La API de Reddit proporciona acceso programático a los datos de la plataforma, facilitando la recolección y el análisis de información generada por los usuarios. Basada en el protocolo HTTP y con soporte para respuestas en formato JSON, esta API permite interactuar con publicaciones, comentarios, usuarios y subreddits. La autenticación se realiza a través de OAuth 2.0, lo que garantiza la seguridad y el control en el acceso a los recursos.

Entre sus principales funcionalidades, la API permite:

- Recuperar publicaciones y comentarios en función de distintos criterios, como popularidad, relevancia temporal o palabras clave.
- Acceder a información específica de subreddits, incluyendo tendencias, reglas de la comunidad y estadísticas de actividad.
- Realizar búsquedas avanzadas que combinan filtros por título, contenido o etiquetas.
- Publicar, votar o comentar de manera programática, lo que habilita la creación de aplicaciones interactivas o bots.

A diferencia de la API de Tumblr, que restringe la búsqueda principalmente a etiquetas y comunidades, la API de Reddit ofrece mayor flexibilidad al permitir filtros por palabras clave en títulos y contenidos. Esta capacidad de búsqueda más amplia y precisa resultó fundamental para este trabajo, ya que permitió construir el conjunto de datos a partir de publicaciones del subreddit `r/depression`.

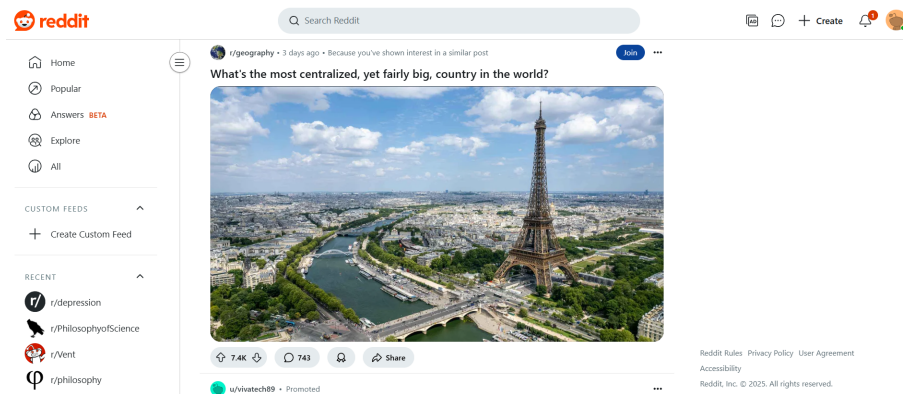


Figura 2.47: Ejemplo de publicación en Reddit dentro del subreddit `r/geography`, mostrando una discusión sobre ciudades europeas.

3. Aspectos socio-sanitarios relevantes para el proyecto

3.1. IA en Salud Mental

En el ámbito de la salud mental, la inteligencia artificial ha impulsado el desarrollo de diversas herramientas, tales como chatbots, sistemas de monitoreo mediante dispositivos wearables, tecnologías de reconocimiento facial y aplicaciones de realidad virtual orientadas a la promoción de la autocompasión. Para la detección de trastornos mentales, se emplean técnicas basadas en el análisis de señales fisiológicas, como electrocardiogramas (ECG), análisis de voz, procesamiento de lenguaje natural (NLP) aplicado a entrevistas clínicas, así como el estudio de biomarcadores sanguíneos y la minería de datos en redes sociales [77], [78].

En particular, la detección de trastornos mentales mediante algoritmos de aprendizaje automático ha sido objeto de múltiples estudios comparativos, evaluando la eficacia y precisión de diversos modelos predictivos, especialmente en el diagnóstico de la depresión [79].

3.2. Modelos Clásicos vs. Deep Learning

Los modelos de aprendizaje automático más comúnmente utilizados incluyen regresión logística, Random Forest, LGBM y SVM. Sin embargo, el desempeño de estos modelos depende en gran medida del proceso de ingeniería de características, y generalmente presentan un rendimiento inferior al de los modelos de aprendizaje profundo, ya que su capacidad para captar matices complejos del lenguaje y dependencias a largo plazo en los datos es muy limitada [79].

En cuanto a las redes neuronales, las CNN optimizan la extracción automática de características relevantes, mientras que las RNN y sus variantes bidireccionales como BiLSTM, están mejor preparadas para modelar datos secuenciales. Estas últimas son capaces de capturar dependencias temporales y contextuales que resultan cruciales en el procesamiento de lenguaje natural.

3.3. Modelos basados en Transformer

Específicamente, los modelos basados en la arquitectura Transformer han demostrado un desempeño superior en tareas de procesamiento de lenguaje natural, debido a su capacidad para comprender de manera más efectiva el contexto y las relaciones a largo plazo dentro del texto [80]. Ejemplos destacados incluyen MentalBERT y PHSBERT, que han sido diseñados y ajustados para aplicaciones en salud mental [81], [82].

La combinación de múltiples modelos preentrenados mediante técnicas de selección y ensamblado permite aumentar tanto la precisión en la clasificación como la interpretabilidad de los resultados, lo que contribuye a una mayor capacidad de generalización frente a nuevos datos [83]. Además, el preprocesamiento adaptado al dominio específico de la aplicación mejora significativamente la calidad de las representaciones aprendidas [84].

Finalmente, las mejoras mediante modelos ensamblados aprovechan la diversidad de predictores para reducir errores y aumentar la robustez del sistema, consolidando un enfoque efectivo para abordar la complejidad inherente en el análisis del lenguaje en contextos clínicos [85].

3.4. Uso del DSM-V para etiquetado clínico

El Manual Diagnóstico y Estadístico de los Trastornos Mentales, quinta edición (DSM-5), constituye un estándar ampliamente reconocido para la clasificación y diagnóstico de trastornos psiquiátricos. En el ámbito clínico y de investigación, el DSM-5 se emplea como referencia para el etiquetado y categorización precisa de síntomas y diagnósticos, facilitando la homogeneización de criterios diagnósticos [86]. Su uso en el etiquetado clínico permite una definición sistemática y estructurada de los trastornos mentales, lo cual es fundamental para el desarrollo de modelos predictivos basados en datos clínicos y la comparación entre estudios.

Según el Manual Diagnóstico y Estadístico de los Trastornos Mentales, quinta edición (DSM-5), el trastorno depresivo mayor se caracteriza por la presencia de al menos cinco de los siguientes síntomas durante un período mínimo de dos semanas. Entre estos síntomas se incluyen [87]:

- **Estado de ánimo deprimido:** Presencia de tristeza, vacío o desesperanza durante la mayor parte del día, casi todos los días, evidenciado por observación directa o reporte subjetivo.
- **Disminución notable del interés o placer:** Pérdida significativa de interés o disfrute en todas o casi todas las actividades cotidianas, afectando la motivación y participación social.
- **Cambios significativos en el peso o apetito:** Pérdida o aumento de peso sin intención, o disminución/aumento del apetito, que reflejan alteraciones en los hábitos alimenticios.
- **Alteraciones del sueño:** Insomnio (dificultad para conciliar o mantener el sueño) o hipersomnia (exceso de sueño), lo cual impacta negativamente en el estado de alerta y energía.
- **Agitación o retraso psicomotor:** Inquietud motora o movimientos lentos y reducidos, observables por otros y no simplemente sentimientos subjetivos.
- **Fatiga o pérdida de energía:** Sensación persistente de cansancio y falta de energía que limita la realización de actividades diarias.
- **Sentimientos de inutilidad o culpa excesiva:** Autoevaluación negativa exagerada o inapropiada, con sentimientos de culpa excesiva que pueden no estar justificados por la realidad.
- **Dificultad para concentrarse o tomar decisiones:** Problemas para mantener la atención, pensar con claridad o decidir, afectando la capacidad funcional.
- **Pensamientos recurrentes de muerte o ideación suicida:** Pensamientos persistentes sobre la muerte, ideas suicidas o intentos de suicidio, que requieren evaluación clínica inmediata.

Estos criterios diagnósticos deben ser evaluados considerando la interferencia significativa en el funcionamiento social, laboral u otras áreas importantes, y que los síntomas no sean atribuibles a efectos fisiológicos de sustancias o a otra condición médica. La definición rigurosa del DSM-5 permite un diagnóstico estructurado y estandarizado, facilitando la investigación y el tratamiento clínico del trastorno depresivo.

4. Descripción de la solución propuesta

El presente trabajo se articula en torno a dos componentes principales y complementarios. Por un lado, se desarrolló un **modelo de predicción** orientado a la detección automática de síntomas depresivos en textos de redes sociales. Por otro, se implementó una **aplicación web** que integra dicho modelo y permite recolectar y procesar en tiempo real publicaciones de la plataforma *Tumblr*.

La exposición sigue el mismo orden seguido en el desarrollo: en primer lugar, se describe el modelo de clasificación, que constituye el núcleo metodológico de la propuesta; a continuación, se presenta la aplicación web, en la que este modelo se incorpora como motor de análisis y clasificación.

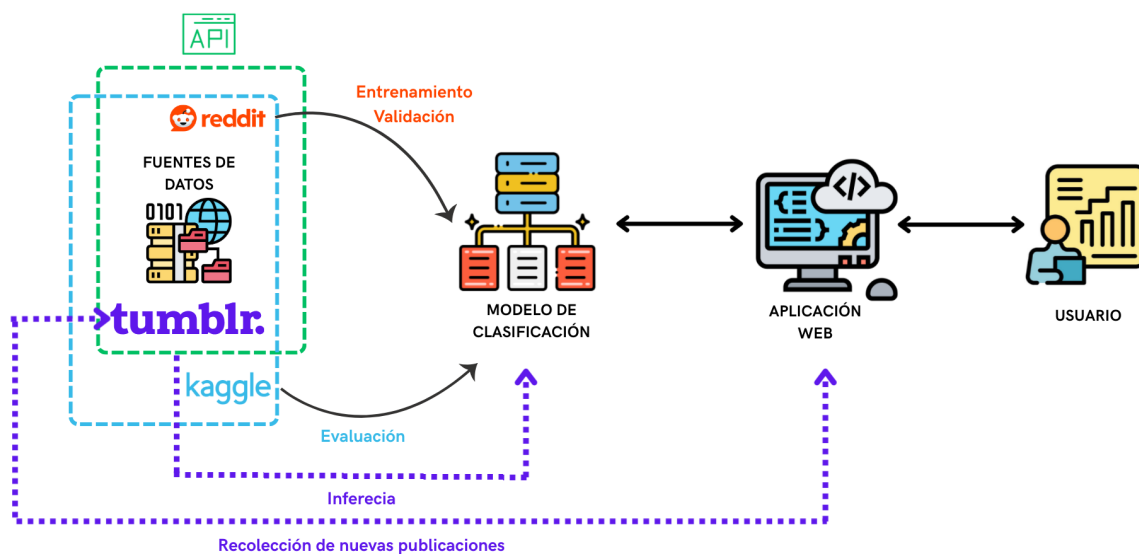


Figura 4.1: Esquema general de la solución propuesta. Se muestran las fuentes de datos empleadas para el entrenamiento, validación y evaluación del modelo de clasificación, así como su integración en la aplicación web.

4.1. Modelo de predicción

4.1.1. Definición del problema

El objetivo central de esta investigación es diseñar un modelo que identifique de manera automática la presencia de síntomas asociados al trastorno depresivo mayor en textos provenientes de redes sociales. En lugar de optar por una clasificación binaria del tipo “depresión/no-depresión”, se adopta un enfoque de **predicción sintomática**, que ofrece mayor granularidad en el análisis y se ajusta mejor al razonamiento clínico, al centrarse en la manifestación específica de los síntomas.

El problema se enmarca en un contexto de **procesamiento de lenguaje natural** aplicado a publicaciones digitales, caracterizadas por su variabilidad en extensión, estructura y contenido. Ante esta situación, se plantea la necesidad de un modelo capaz de realizar **predicciones multietiqueta**, de modo que un mismo texto pueda estar asociado simultáneamente a varios síntomas.

4.2. Construcción del conjunto de datos de Reddit y Tumblr

El desarrollo del modelo requiere un **conjunto de datos etiquetado a nivel de síntoma**, el cual no está disponible en los repositorios públicos existentes. Dado que el etiquetado manual por parte de expertos clínicos resultaba inviable por limitaciones de tiempo y recursos, se implementó una estrategia de etiquetado semi-automático a partir de datos recolectados en redes sociales.

Para llevar a cabo este proceso, se recurrió a dos plataformas descritas en el marco tecnológico. Por un lado, la API de *Tumblr*, que únicamente permite realizar búsquedas de publicaciones mediante etiquetas y comunidades. Por otro lado, la API de *Reddit* ofrece funcionalidades más avanzadas, como el filtrado por palabras clave en títulos y contenidos. Debido a esta mayor flexibilidad, se utilizó *Reddit* como fuente principal de datos para la construcción del modelo, concentrando la recolección principalmente en el subreddit *r/depression*. El conjunto, mucho menor, proveniente de *Tumblr* se utilizó en la fase de evaluación del modelo.

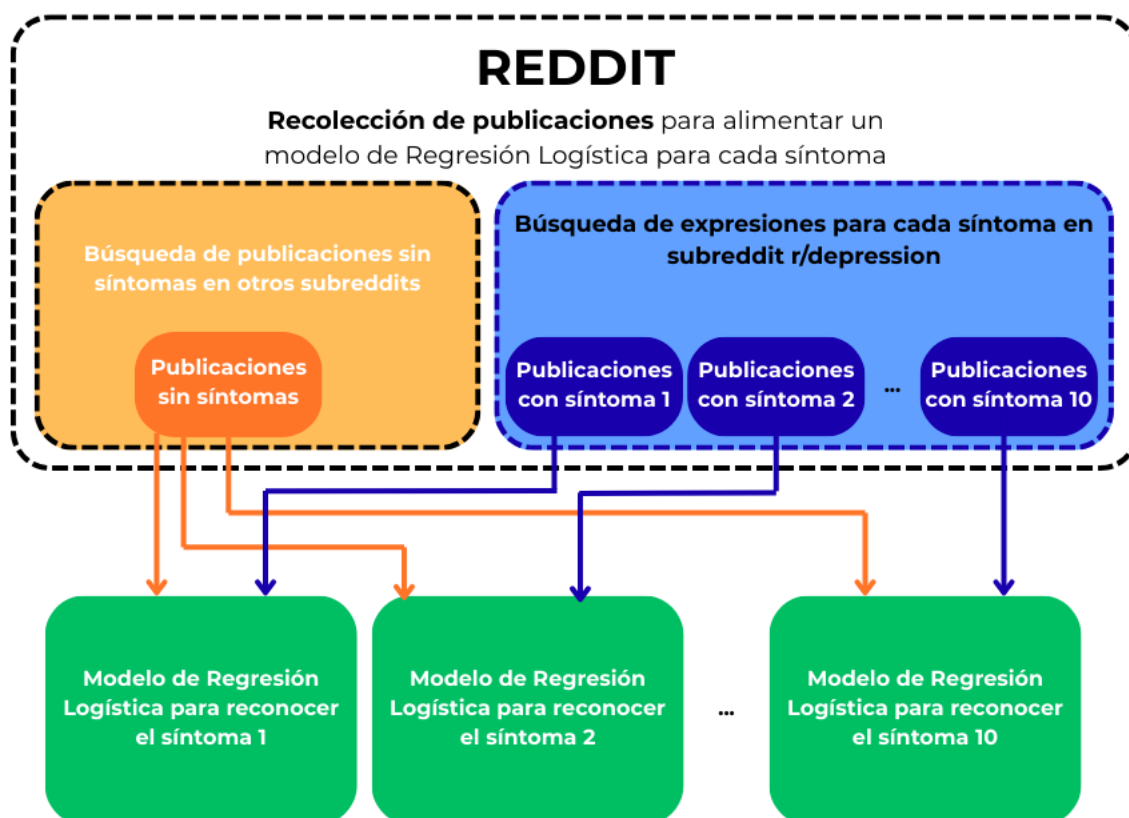


Figura 4.2: Flujo de recolección y etiquetado de publicaciones en Reddit. Se seleccionan publicaciones con síntomas en *r/depression* y textos sin síntomas en otros subreddits para entrenar modelos de regresión logística específicos por síntoma.

Para cada uno de los diez síntomas principales del trastorno depresivo mayor, definidos en el DSM-5, se elaboró una **lista de expresiones representativas**. Estas expresiones consistieron en términos y frases mediante los cuales los usuarios podrían describir cada síntoma (por ejemplo, “no puedo dormir” en el caso del insomnio). Dichos términos se utilizaron como **criterios de búsqueda** para identificar publicaciones potencialmente relacionadas.

Adicionalmente, se recopilaron **publicaciones neutrales** en subreddits no vinculados con la

salud mental (como `r/funny` o `r/technology`), que se emplearon como ejemplos negativos (sin síntomas).

Los textos correspondientes a cada síntoma se organizaron en subconjuntos independientes, generando diez conjuntos en total. Cada publicación fue etiquetada como positiva para el síntoma correspondiente, y a cada conjunto se añadieron ejemplos negativos extraídos de forma balanceada del grupo neutral. De esta manera, se garantizó un número equitativo de ejemplos positivos y negativos.

El resultado fue un **conjunto de datos por síntoma**, compuesto por publicaciones etiquetadas como positivas (con el síntoma) o negativas (sin él), lo que permitió el **entrenamiento de un modelo binario independiente** para cada manifestación clínica.



Figura 4.3: Ilustración del proceso de generación de subconjuntos balanceados para el síntoma 4.

Para seleccionar el modelo más adecuado, se comparó el desempeño de la **regresión logística** (*Logistic Regression*) y de las **máquinas de vectores de soporte** (SVM) utilizando el conjunto correspondiente al primer síntoma. Si bien no se observaron diferencias significativas en las métricas de clasificación, la regresión logística mostró tiempos de entrenamiento considerablemente menores debido a su menor complejidad computacional, motivo por el cual fue el modelo elegido.

Clase	Logistic Regression			SVM		
	Precisión	Recall	F1-score	Precisión	Recall	F1-score
0	0.9770	0.9575	0.9672	0.9847	0.9625	0.9735
1	0.9518	0.9739	0.9628	0.9576	0.9826	0.9700
Exactitud		0.9651			0.9718	
Promedio macro	0.9644	0.9657	0.9650	0.9711	0.9726	0.9717
Promedio ponderado	0.9654	0.9651	0.9651	0.9721	0.9718	0.9718

Tabla 4.1: Comparación de métricas de clasificación entre regresión logística y SVM para el primer síntoma. Los valores de precisión, recall y F1-score por clase (0 = sin síntoma, 1 = con síntoma), así como las métricas globales de exactitud y promedios, muestran diferencias mínimas (menores al 1 %) entre ambos modelos.

	Predicha 0	Predicha 1
Real 0	383	17
Real 1	9	336

Tabla 4.2: Matriz de confusión del modelo de regresión logística. Cada celda indica el número de publicaciones clasificadas en cada categoría. La diagonal (383 y 336) corresponde a aciertos: textos sin el síntoma identificados correctamente como “0” y textos con el síntoma como “1”. Los valores fuera de la diagonal (17 y 9) son errores: textos mal clasificados. Por lo tanto, el modelo logró clasificar correctamente la gran mayoría de los casos.

	Predicha 0	Predicha 1
Real 0	385	15
Real 1	6	339

Tabla 4.3: Matriz de confusión del modelo SVM. Al igual que en la regresión logística, la diagonal (385 y 339) indica aciertos y los valores fuera de ella (15 y 6) representan errores. El SVM cometió ligeramente menos errores que la regresión logística, aunque con un coste de entrenamiento mayor.

Posteriormente, se entrenó un **modelo de regresión logística para cada uno de los síntomas restantes**. Estos clasificadores se aplicaron de forma recursiva sobre los subconjuntos recolectados, de modo que cada publicación fue evaluada por los diez modelos, generando una **etiqueta binaria** (presencia o ausencia del síntoma) acompañada de una **probabilidad asociada**. Este procedimiento permitió construir un conjunto final de datos con publicaciones etiquetadas para todos los síntomas, lo que posibilitó el entrenamiento de modelos más complejos en etapas posteriores.

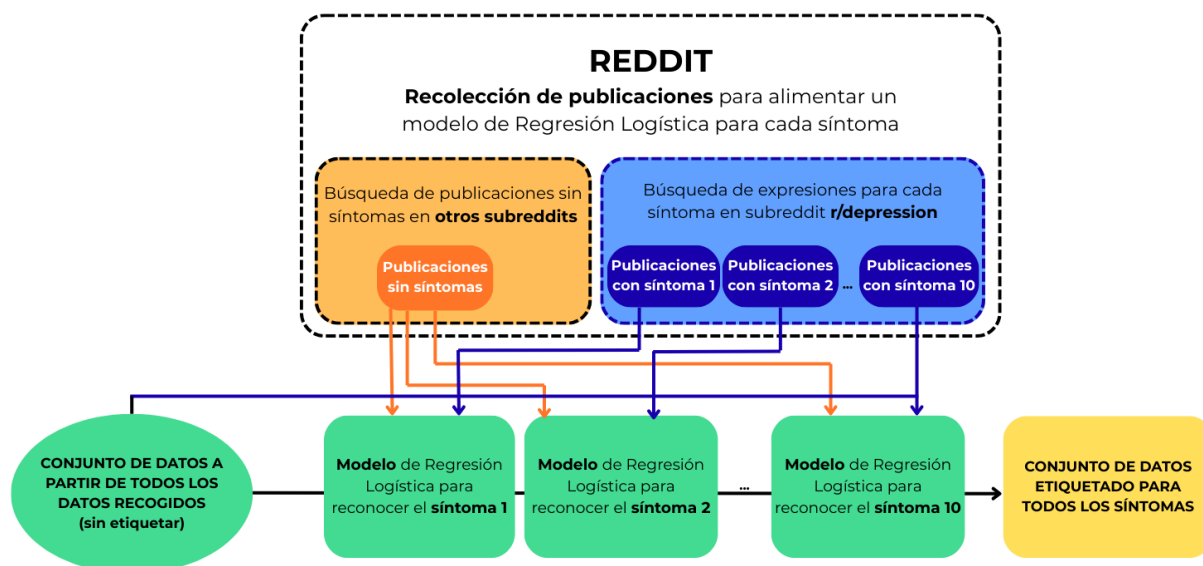


Figura 4.4: Integración del proceso de recolección y modelado. Cada modelo de regresión logística entrenado por síntoma asigna etiquetas al conjunto total de publicaciones, produciendo un dataset final con anotaciones para todos los síntomas.

4.2.1. Conjuntos de datos utilizados

Para el desarrollo y validación del modelo se emplearon tres conjuntos de datos con características complementarias, procedentes de diferentes fuentes y con distintos niveles de granularidad en el etiquetado:

- **Conjunto de datos de Reddit.** Contiene 34.864 entradas y un total de 26 columnas. Incluye el identificador de cada publicación, el texto y diez variables binarias que representan la presencia o ausencia de síntomas (*síntoma_1* a *síntoma_10*). Adicionalmente incorpora columnas de probabilidad, generadas como salida de modelos de regresión logística entrenados para cada síntoma, que permanecen vacías cuando el texto ha sido etiquetado manualmente. La distribución es relativamente balanceada, aunque con ligera predominancia de la clase negativa en todos los síntomas. Este conjunto se utilizó para el entrenamiento,

la validación y la evaluación del modelo.

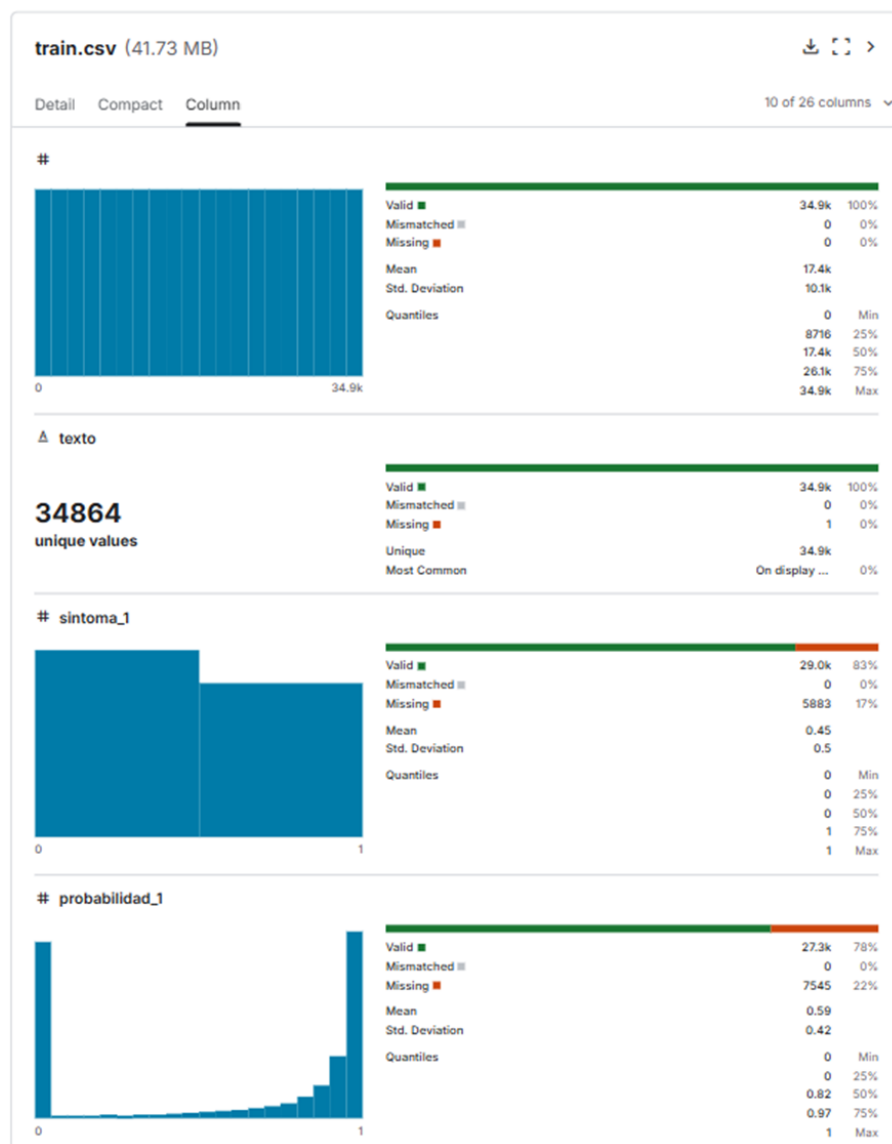


Figura 4.5: Visualización de las estadísticas del conjunto *train.csv*, que contiene las publicaciones recopiladas de Reddit. Se muestran algunas columnas: el identificador (*#*), *texto*, que incluye el contenido de la publicación; *sintoma_1*, correspondiente al síntoma "estado de ánimo deprimido"; y *probabilidad_1*, que indica la probabilidad asociada a la categoría 1 para este síntoma.

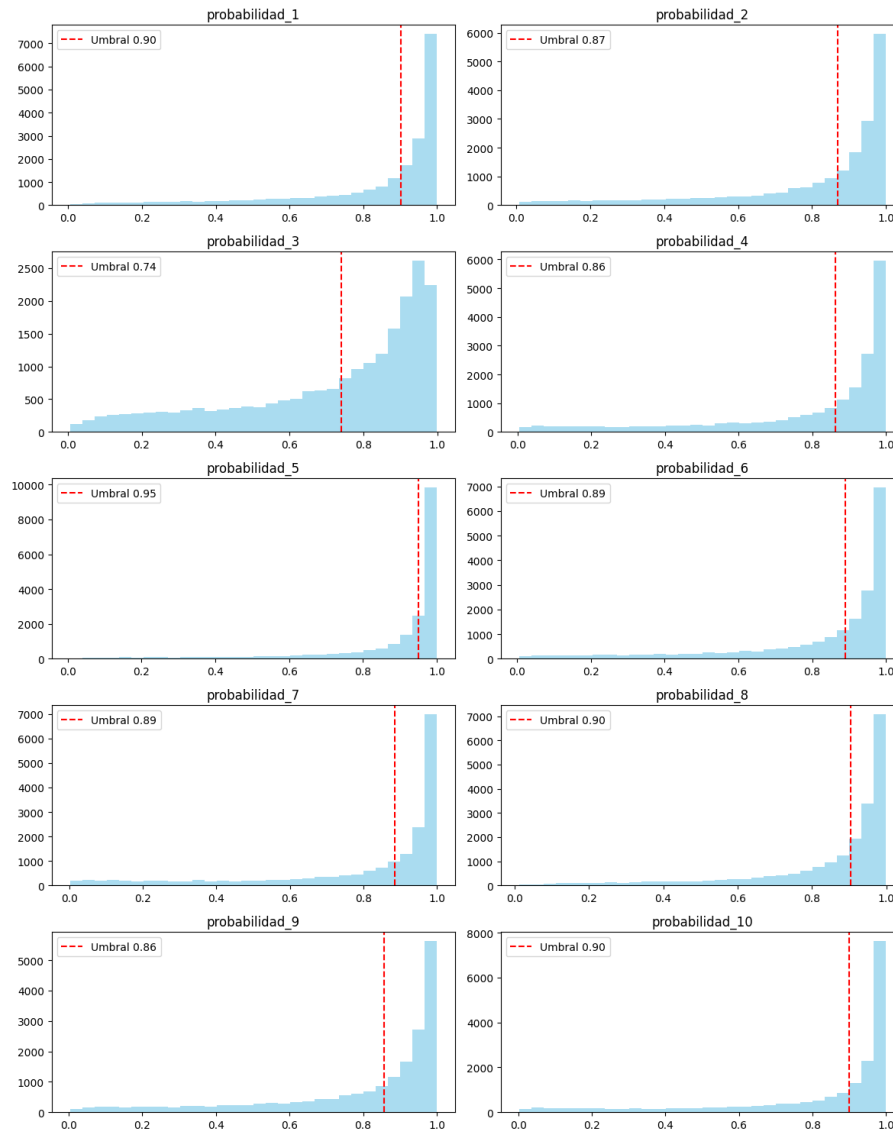


Figura 4.6: Histograma de las variables *probabilidad_1* a *probabilidad_10*. Cada gráfico incluye una línea roja punteada que indica el umbral de decisión utilizado para clasificar una observación como positiva.

- **Conjunto de datos de Tumblr.** Se compone de 324 publicaciones, etiquetadas manualmente. Incluye once columnas: el texto y las variables binarias correspondientes a los diez síntomas. Presenta un notable desbalance en todas las etiquetas, con predominio de valores negativos. Aunque reducido en tamaño, resulta esencial para evaluar el rendimiento del modelo en el contexto específico de *Tumblr*, red social en la que se centra la fase de inferencia en tiempo real.

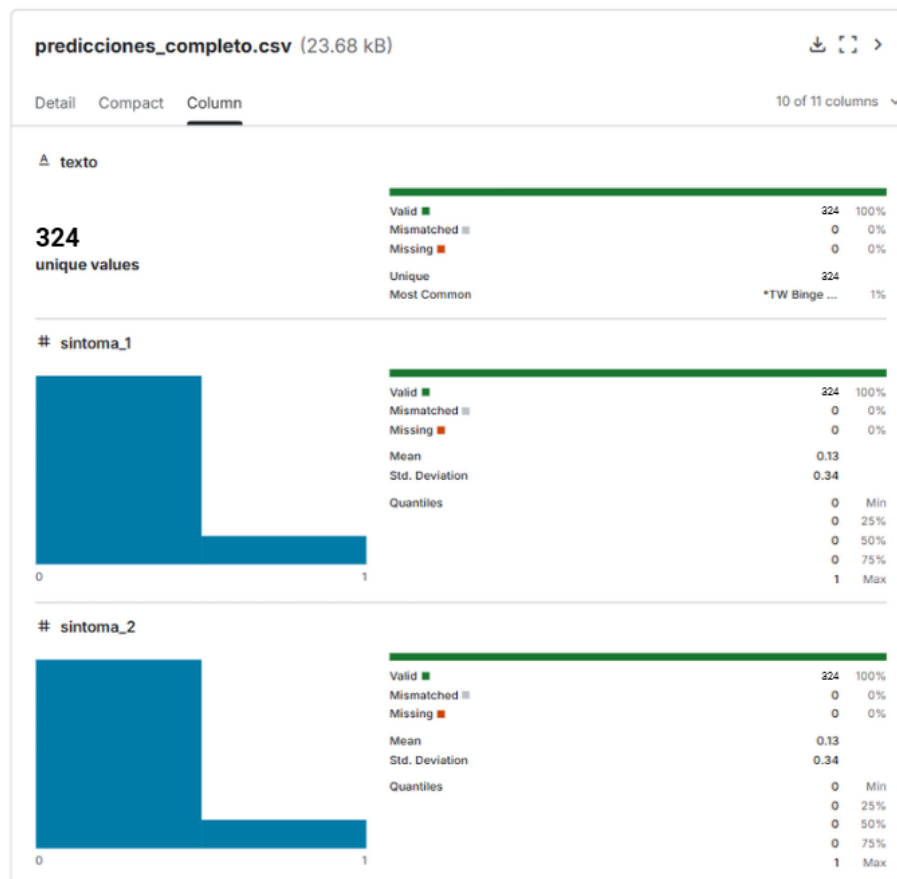


Figura 4.7: Visualización de las estadísticas del conjunto predicciones_completo.csv, que contiene las publicaciones recopiladas de Tumblr. Se muestran las columnas *texto* (contenido de la publicación), *sintoma_1* (estado de ánimo deprimido) y *sintoma_2* (disminución del interés).

- **Conjunto de datos “Depression: Reddit Dataset (Cleaned)” [88].** Obtenido de la plataforma Kaggle, este corpus contiene 7.650 publicaciones extraídas mediante *scraping* de subreddits relacionados con la depresión. Consta de dos columnas: el texto limpio (*clean_text*) y una etiqueta binaria (*is_depression*) que indica si la publicación está asociada o no a la depresión. La distribución de clases es equilibrada. Este conjunto se utilizó para evaluar el rendimiento del modelo en una tarea de clasificación binaria, complementando los resultados obtenidos con la predicción sintomática.

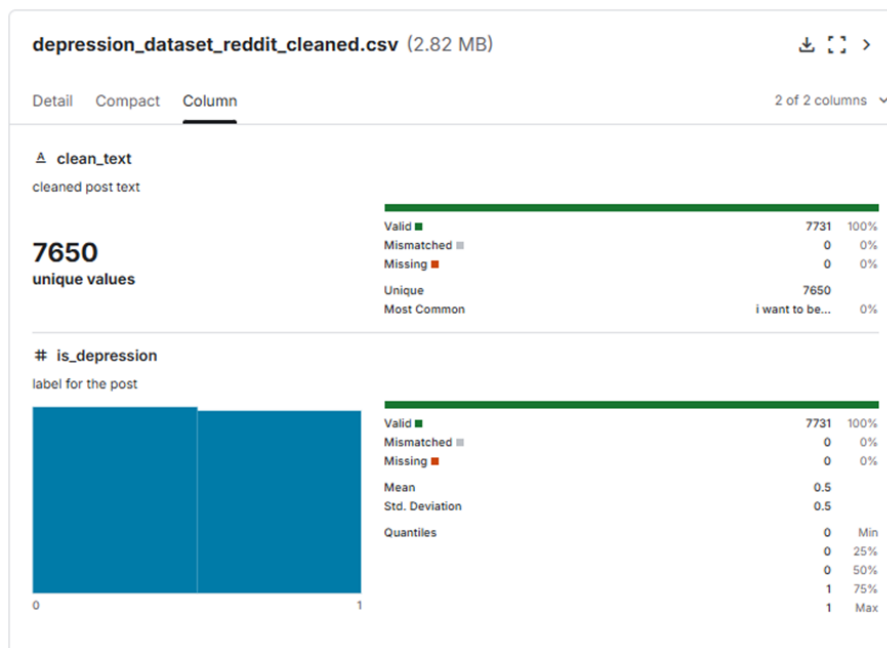


Figura 4.8: Visualización de las estadísticas del conjunto de datos *Depression: Reddit Dataset (Cleaned)*, almacenado en `depression_dataset_reddit_cleaned.csv`. Se observan las columnas `clean_text` y `is_depression`

4.2.1.1 Pasos previos al entrenamiento

Antes de ajustar el modelo se aplicaron procedimientos de **preprocesamiento** con el fin de preparar el corpus de manera adecuada para el entrenamiento. Estos incluyeron la **limpieza y normalización textual** (eliminación de caracteres no estándar, estandarización de signos de puntuación y ajuste de espacios), así como la **curación del corpus** mediante la eliminación de duplicados y organización de los textos en una estructura tabular que vincula cada documento con sus etiquetas correspondientes. Este proceso se aplicó tanto a los datos provenientes de *Tumblr* como a los de *Reddit*.

En el caso de *Reddit*, fuente de los datos de entrenamiento, para mitigar el **desbalance de clases** se aplicaron técnicas de muestreo entre ejemplos positivos y negativos, y se estableció un **límite máximo de longitud** por documento con el fin de optimizar la utilización de recursos computacionales. Los conjuntos procesados se exportaron siguiendo el formato descrito en la sección anterior.

Posteriormente, los textos de este corpus se transformaron en **representaciones densas** (*embeddings*) mediante el modelo *all-MiniLM-L6-v2* de *Sentence-Transformers*, que proyecta cada documento en un espacio vectorial de baja dimensión preservando la información semántica. Estas representaciones fueron **normalizadas** con la norma ℓ_2 :

$$\tilde{\mathbf{e}} = \frac{\mathbf{e}}{\|\mathbf{e}\|_2}.$$

Los embeddings resultantes se utilizaron como entrada para los clasificadores supervisados y se almacenaron para su posterior uso y trazabilidad experimental.

Finalmente, el conjunto de datos se dividió en **entrenamiento, validación y prueba**.

4.2.1.2 Arquitectura del modelo

El modelo implementa una arquitectura de fine-tuning del encoder preentrenado (DistilBERT) seguido de capas densas para clasificación multietiqueta.

1. Entrada Cada texto se convierte en una **secuencia de tokens**, es decir, fragmentos de palabras o palabras completas que el modelo puede procesar. Este proceso se conoce como tokenización. Sea una secuencia de texto tokenizada:

- $\mathbf{x} \in \mathbb{Z}^L$: secuencia de tokens
- $\mathbf{m} \in \{0,1\}^L$: *attention mask*, donde $m_i = 1$ si el token es válido, 0 si es *padding*

La *attention mask* es una herramienta que le indica al modelo qué partes del texto son reales y cuáles son relleno (*padding*) añadido para igualar la longitud de las secuencias.

2. Capa de Embedding (DistilBERT) DistilBERT convierte cada token en un vector que captura su significado dentro del contexto de toda la frase. Esto permite al modelo “entender” palabras según su uso, no solo su forma aislada.

El modelo base produce una representación contextualizada:

$$\mathbf{H} = \text{DistilBERT}(\mathbf{x}, \mathbf{m}) \in \mathbb{R}^{L \times d}$$

donde $d = 768$ es la dimensión del *embedding* y $\mathbf{H} = [\mathbf{h}_1, \mathbf{h}_2, \dots, \mathbf{h}_L]$.

3. Vector CLS y proyección lineal El modelo inserta un **token especial** llamado [CLS] al **inicio de cada secuencia**. Su representación es usada como resumen de todo el texto y se emplea como entrada para la etapa de clasificación.

Se extrae la representación del token especial [CLS]:

$$\mathbf{h}_{\text{CLS}} = \mathbf{h}_1 \in \mathbb{R}^{768}$$

Luego se proyecta mediante una **capa lineal** y una **activación no lineal**:

$$\begin{aligned} \mathbf{z}_1 &= \mathbf{W}_1 \mathbf{h}_{\text{CLS}} + \mathbf{b}_1, \quad \mathbf{W}_1 \in \mathbb{R}^{768 \times 768} \\ \mathbf{a}_1 &= \tanh(\mathbf{z}_1) \end{aligned}$$

4. Regularización Para evitar que el modelo se adapte demasiado a los datos de entrenamiento (sobreajuste), se introduce una técnica llamada *dropout*, que desactiva aleatoriamente una parte de las neuronas durante el entrenamiento.

Se aplica **dropout** para prevenir overfitting:

$$\mathbf{a}'_1 = \text{Dropout}(\mathbf{a}_1)$$

5. Capa de Clasificación Una vez que DistilBERT ha procesado el texto, se obtiene un vector resumen que condensa la información más relevante del texto completo en una representación compacta.

La **capa de clasificación** transforma este vector resumen en una salida con C valores (en este caso, 10), uno para cada clase o síntoma que deseamos predecir. Estos valores, llamados **logits**, son números reales que reflejan la **afinidad entre el texto y cada clase** antes de convertirlos en probabilidades.

Proyección final hacia C clases:

$$\hat{\mathbf{y}} = \mathbf{W}_2 \mathbf{a}'_1 + \mathbf{b}_2, \quad \mathbf{W}_2 \in \mathbb{R}^{C \times 768}$$

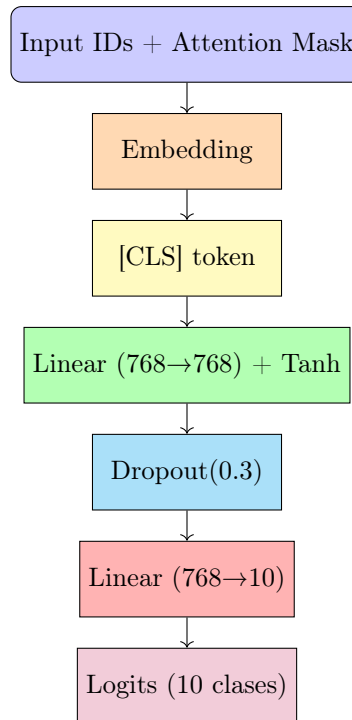
6. Función de pérdida La **función de pérdida** cuantifica la **diferencia entre las predicciones del modelo y las etiquetas reales**. En este caso, se utiliza una variante de la **entropía cruzada**, una función estándar para problemas de clasificación.

Para tareas multietiqueta, se aplica una versión por clase que combina la función sigmoide con la entropía cruzada binaria.

Se utiliza entropía cruzada sobre los logits $\hat{\mathbf{y}} \in \mathbb{R}^C$, con una etiqueta verdadera $\mathbf{y} \in \{0, 1\}^C$ codificada en one-hot:

$$\mathcal{L} = - \sum_{i=1}^C y_i \log \left(\frac{e^{\hat{y}_i}}{\sum_{j=1}^C e^{\hat{y}_j}} \right)$$

Este proceso permite ajustar los parámetros del modelo.



4.2.1.3 Entrenamiento

En la configuración estándar de DistilBERT, se recomienda emplear un **tamaño de lote** de 16 ejemplos y una **tasa de aprendizaje** de 3×10^{-5} , siendo esta última especialmente determinante para el desempeño del modelo. Estudios previos han evidenciado una correlación negativa significativa entre la tasa de aprendizaje y la capacidad de generalización: tasas más bajas tienden a mejorar el rendimiento en el conjunto de validación y a reducir el sobreajuste [89].

Considerando que DistilBERT está diseñado para entrenarse eficientemente con un número reducido de **épocas**, se decidió comenzar con 10 épocas de entrenamiento. Por tanto, la configuración inicial adoptada incluyó una longitud máxima de secuencia de 128 tokens, un tamaño de lote de 16, diez épocas de entrenamiento y una tasa de aprendizaje de 3×10^{-5} .

Para la optimización de los parámetros del modelo se utilizó el algoritmo **Adam**, que combina los beneficios del descenso de gradiente estocástico con la adaptación automática de la tasa de aprendizaje para cada parámetro. Adam es ampliamente empleado en entrenamiento de modelos de lenguaje basados en Transformers debido a su rapidez de convergencia y estabilidad frente a gradientes ruidosos.

La evaluación del modelo durante el entrenamiento se realizó mediante un esquema de validación **hold-out**, separando un subconjunto de los datos de entrenamiento como conjunto de validación. Este enfoque permite monitorear el desempeño del modelo en datos no vistos y ajustar hiperparámetros, evitando el sobreajuste, sin necesidad de realizar una validación cruzada completa que incrementaría el costo computacional.

A partir de esta base, se exploraron diferentes configuraciones, evaluando el rendimiento en cada época mediante las siguientes métricas [90]:

- **Pérdida (Loss):** Se utilizó la función de pérdida definida por $\mathcal{L}(\hat{y}, y)$, donde $\hat{y}$ representa las predicciones del modelo y y las etiquetas verdaderas. La pérdida promedio se calcula

como:

$$\text{Loss} = \frac{1}{N} \sum_{i=1}^N \ell(\hat{y}_i, y_i)$$

donde N es el número de muestras, y ℓ es la función de pérdida por muestra, en este caso, la entropía binaria cruzada).

- **Exactitud (Accuracy):** Mide la proporción de predicciones correctas sobre todas las muestras, calculada como:

$$\text{Accuracy} = \frac{\text{Número de predicciones correctas}}{N} = \frac{\sum_{i=1}^N \text{coincidencia}(\hat{y}_i, y_i)}{N},$$

donde N es el número de muestras, y

$$\text{coincidencia}(\hat{y}_i, y_i) = \begin{cases} 1, & \text{si } \hat{y}_i = y_i, \\ 0, & \text{si } \hat{y}_i \neq y_i. \end{cases}$$

- **F1 Score Micro ($F1_{\text{micro}}$):** Combina precisión y exhaustividad considerando todos los elementos predichos a nivel global:

$$F1_{\text{micro}} = \frac{2 \times P_{\text{micro}} \times R_{\text{micro}}}{P_{\text{micro}} + R_{\text{micro}}}$$

donde

$$P_{\text{micro}} = \frac{\sum_{j=1}^L TP_j}{\sum_{j=1}^L (TP_j + FP_j)}, \quad R_{\text{micro}} = \frac{\sum_{j=1}^L TP_j}{\sum_{j=1}^L (TP_j + FN_j)}$$

con TP_j , FP_j , y FN_j los verdaderos positivos, falsos positivos y falsos negativos para la etiqueta j .

- **F1 Score Macro ($F1_{\text{macro}}$):** Promedia el F1 Score de cada etiqueta, otorgando igual peso a cada clase:

$$F1_{\text{macro}} = \frac{1}{L} \sum_{j=1}^L F1_j$$

donde $F1_j$ es el F1 Score para la etiqueta j .

- **Pérdida de Hamming (Hamming Loss):** Indica la fracción promedio de etiquetas incorrectamente clasificadas, definido como:

$$\text{Hamming Loss} = \frac{1}{N \times L} \sum_{i=1}^N \sum_{j=1}^L \mathbf{1}(\hat{y}_{ij} \neq y_{ij})$$

Durante el entrenamiento se realizaron las siguientes modificaciones:

1. **Dropout:** Durante las primeras pruebas, se aplicó un valor de **dropout** de 0.1. Sin embargo, este valor resultó insuficiente para mitigar el **sobreaajuste**, por lo que se incrementó a 0.3. Este ajuste produjo una mejora notable en el rendimiento del modelo.

Época	Loss		Accuracy		F1 Micro		F1 Macro		Hamming Loss	
	Entren.	Val.	Entren.	Val.	Entren.	Val.	Entren.	Val.	Entren.	Val.
1	0.2668	0.2197	0.8802	0.9006	0.8459	0.8715	0.8458	0.8712	0.1198	0.0994
2	0.1918	0.2113	0.9170	0.9058	0.8919	0.8792	0.8918	0.8788	0.0830	0.0942
3	0.1415	0.2114	0.9412	0.9083	0.9229	0.8788	0.9229	0.8785	0.0588	0.0917
4	0.1111	0.2366	0.9550	0.9078	0.9409	0.8785	0.9409	0.8781	0.0450	0.0922
5	0.0910	0.2578	0.9640	0.9038	0.9527	0.8765	0.9526	0.8763	0.0360	0.0962

Tabla 4.4: Métricas para entrenamiento y validación por época con dropout de 0.3 (se destacan los mejores valores para cada métrica)

2. **Congelación de capas:** También se experimentó con la **congelación de capas** para reducir la complejidad del modelo y mejorar la estabilidad. Inicialmente se congelaron las dos primeras capas de DistilBERT, dado que las capas inferiores suelen capturar **representaciones lingüísticas generales** (como patrones morfosintácticos o léxicos básicos) que son transferibles entre tareas. Esta estrategia permite conservar dicho conocimiento adquirido durante el preentrenamiento, reduciendo el riesgo de sobreajuste y el coste computacional.

No obstante, congelar las primeras capas podría limitar la capacidad del modelo para adaptarse a las sutiles diferencias gramaticales en el uso del lenguaje por parte de personas con depresión. Por tanto, en configuraciones posteriores, se optó por descongelarlas.

3. **Weight decay:** Una vez descongeladas, se introdujo un término de regularización adicional mediante **weight decay**, lo cual tuvo un impacto positivo en el rendimiento. En particular, se observó una mejora significativa al establecer su valor en 0.01.

Época	Loss		Accuracy		F1 Micro		F1 Macro		Hamming Loss	
	Entren.	Val.	Entren.	Val.	Entren.	Val.	Entren.	Val.	Entren.	Val.
1	0.2667	0.2199	0.8801	0.8999	0.8461	0.8731	0.8460	0.8731	0.1199	0.1001
2	0.1937	0.2074	0.9163	0.9074	0.8911	0.8803	0.8910	0.8801	0.0837	0.0926
3	0.1435	0.2148	0.9404	0.9077	0.9220	0.8786	0.9220	0.8783	0.0596	0.0923
4	0.1117	0.2284	0.9547	0.9060	0.9406	0.8775	0.9406	0.8774	0.0453	0.0940
5	0.0929	0.2524	0.9635	0.9063	0.9520	0.8789	0.9520	0.8787	0.0365	0.0937

Tabla 4.5: Métricas para entrenamiento y validación por época con weight decay de 0.01

4. **Tamaño de lote:** Asimismo, se incrementó el **tamaño de lote** a 32, observándose un mejor desempeño sin comprometer la estabilidad del entrenamiento. Tal como se muestra en la Tabla 4.6, con esta configuración el modelo alcanzó valores de **F1 Micro** y **F1 Macro** más altos en validación (épocas 3 y 4), junto con una reducción del **Hamming Loss**, lo que refleja una mejor capacidad de generalización. No obstante, con un tamaño de lote de 64 no se observaron mejoras adicionales, e incluso se detectaron signos de degradación del rendimiento.

Época	Loss		Accuracy		F1 Micro		F1 Macro		Hamming Loss	
	Entren.	Val.	Entren.	Val.	Entren.	Val.	Entren.	Val.	Entren.	Val.
1	0.2758	0.2167	0.8765	0.9032	0.8412	0.8740	0.8411	0.8737	0.1235	0.0968
2	0.1975	0.2052	0.9146	0.9077	0.8887	0.8811	0.8886	0.8808	0.0854	0.0923
3	0.1508	0.2258	0.9370	0.9069	0.9175	0.8813	0.9174	0.8811	0.0630	0.0931
4	0.1180	0.2383	0.9527	0.9086	0.9378	0.8829	0.9378	0.8828	0.0473	0.0914
5	0.0988	0.2475	0.9610	0.9054	0.9487	0.8785	0.9487	0.8783	0.0390	0.0946
6	0.0832	0.2612	0.9672	0.9067	0.9569	0.8804	0.9569	0.8804	0.0328	0.0933

Tabla 4.6: Resultados de entrenamiento y validación por época con un lote de tamaño 32 y una tasa de aprendizaje de $3e-5$.

Dado que la tarea abordada corresponde a una clasificación **multietiqueta**, la selección de la época óptima para interrumpir el entrenamiento se basó principalmente en las métricas **F1 Micro**, **F1 Macro** y el **Hamming Loss** en el conjunto de validación.

El **F1 Micro** resulta especialmente relevante en este contexto, ya que tiene en cuenta el rendimiento global del modelo sobre todas las etiquetas, ponderando por frecuencia, lo que permite evaluar su capacidad general de predicción.

En paralelo, el **F1 Macro** proporciona una perspectiva equilibrada al promediar el rendimiento sobre cada etiqueta por igual, siendo útil para detectar posibles sesgos del modelo hacia clases más frecuentes.

Finalmente, el **Hamming Loss**, al medir la proporción de etiquetas mal clasificadas respecto al total, ofrece una visión directa del error cometido en cada instancia multietiqueta.

Estas métricas fueron priorizadas frente a la mera precisión, ya que esta última puede resultar engañosa en escenarios con múltiples etiquetas por muestra, especialmente si las distintas etiquetas se presentan de manera desbalanceada. Con base en estos criterios, se aplicó **early-stopping** y se seleccionó la cuarta época, en la que se alcanzó un equilibrio entre rendimiento y generalización, evitando el sobreajuste.

Los resultados obtenidos con esta configuración final del modelo sobre el conjunto de validación se presentarán en el capítulo de resultados.

4.3. Desarrollo de la aplicación web

4.3.1. Arquitectura del diseño

La aplicación web se ha desarrollado siguiendo un modelo cliente-servidor, lo que permite separar claramente la lógica de presentación de la lógica de negocio y facilita la integración de servicios externos. Esta arquitectura modular aporta escalabilidad, mantenibilidad y flexibilidad, características clave para un sistema que combina procesamiento de datos en tiempo real y análisis mediante aprendizaje automático.

La solución se organiza en tres capas principales:

- **Backend:** Gestiona la lógica de negocio, la adquisición y procesamiento de datos, y la ejecución del modelo de predicción. Se encarga de recibir solicitudes del frontend y devolver resultados estructurados en formato JSON.
- **Frontend:** Proporciona la interfaz de usuario, permitiendo la introducción de parámetros, la visualización de resultados y la interacción con los datos de manera dinámica e intuitiva.
- **API externa (Tumblr API):** Fuente de datos en tiempo real que alimenta la aplicación, ofreciendo publicaciones etiquetadas según criterios definidos por el usuario.

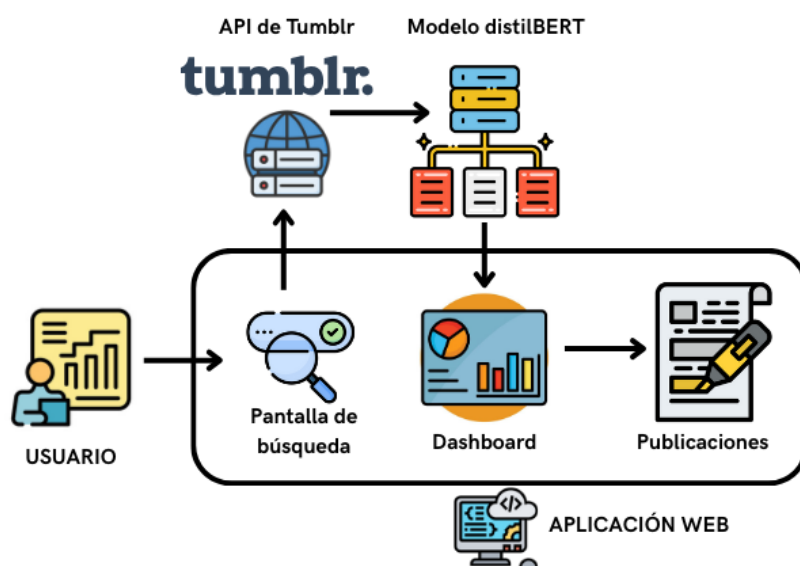


Figura 4.9: Arquitectura general de la aplicación web. La pantalla de búsqueda, el dashboard y la visualización de las publicaciones se muestran en el frontend. El backend se encarga del procesamiento de los datos introducidos por el usuario, de la adquisición de información mediante la API de Tumblr y de la clasificación de los resultados utilizando el modelo distilBERT.

4.3.2. Backend

El backend de la aplicación web ha sido desarrollado utilizando el microframework **Flask**, con el objetivo de ofrecer una interfaz RESTful que integre funcionalidades de adquisición de datos desde la API pública de Tumblr, procesamiento de texto y clasificación automática mediante un modelo de aprendizaje profundo.

4.3.2.1 Arquitectura modular

El backend se estructura modularmente en los siguientes componentes principales:

- **app.py**: punto de entrada de la aplicación, donde se configura la instancia de Flask, se habilita CORS para permitir peticiones desde el frontend y se registran los blueprints correspondientes.
- **routes/fetch_posts.py**: define la ruta `/api/fetch-publicaciones`, que recibe solicitudes POST y orquesta la lógica del flujo completo: adquisición de publicaciones, procesamiento textual y predicción de etiquetas.
- **routes/get_importance.py**: define la ruta `/api/get-importance`, que permite obtener la importancia de cada palabra en la predicción de una etiqueta específica mediante mapas de saliencia.
- **utils/tumblr_api.py**: módulo auxiliar que encapsula la lógica de acceso a la API de Tumblr.
- **ml/predict.py**: módulo de inferencia que emplea el modelo preentrenado y fine-tuneado `DistilBertForSequenceClassification` para realizar predicción multi-etiqueta sobre textos extraídos de publicaciones. Además, incorpora interpretabilidad mediante mapas de saliencia para identificar qué palabras contribuyen más a cada predicción.

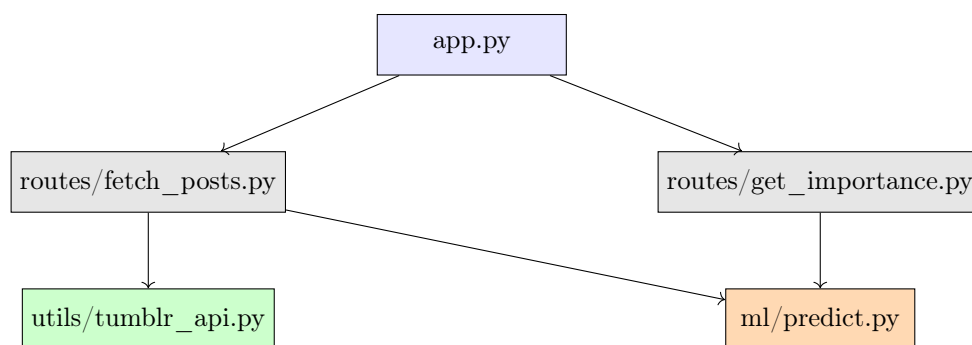


Figura 4.10: Arquitectura modular del backend con sus componentes y dependencias.

4.3.2.2 Flujo funcional

1. Extracción y etiquetado de publicaciones

1. El cliente realiza una solicitud POST al endpoint `/api/fetch-publicaciones`, proporcionando parámetros como el tipo de fuente (`sourceType`), la palabra clave (`inputValue`) y el número de publicaciones deseado (`nrPosts`).
2. El backend invoca la función `get_tagged_posts()` del módulo `utils/tumblr_api.py` para recuperar publicaciones desde Tumblr.
3. Los textos relevantes se extraen y limpian utilizando BeautifulSoup, almacenándose en un Dataframe de Pandas.

4. Los textos procesados se pasan al modelo distilBERT mediante la función `predict_posts()`, la cual retorna etiquetas binarias y sus probabilidades asociadas.
5. Finalmente, se devuelve al frontend un objeto JSON con los textos y las predicciones correspondientes.

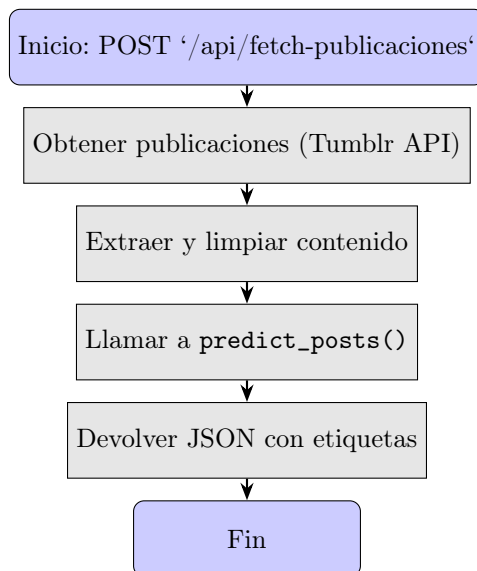


Figura 4.11: Flujo funcional para la extracción y clasificación de publicaciones en `/api/fetch-publicaciones`.

2. Obtención de mapas de saliencia

Los **mapas de saliencia**, generados con la librería **Captum**, permiten identificar qué tokens del texto contribuyen en mayor medida a la predicción del modelo, facilitando así la interpretación de sus decisiones.

Para su cálculo dentro del sistema, se sigue el procedimiento técnico que se detalla a continuación:

1. El cliente envía un POST a `/api/get-importance` con el texto (`text`) y la etiqueta objetivo (`target_label`).
2. El backend ejecuta `interpret_prediction()` en `ml/predict.py` para generar saliency maps.
3. Se calculan las puntuaciones de importancia por token con `importancia_palabras()`.
4. El backend responde con un JSON que contiene el array `tokensWithScores`.

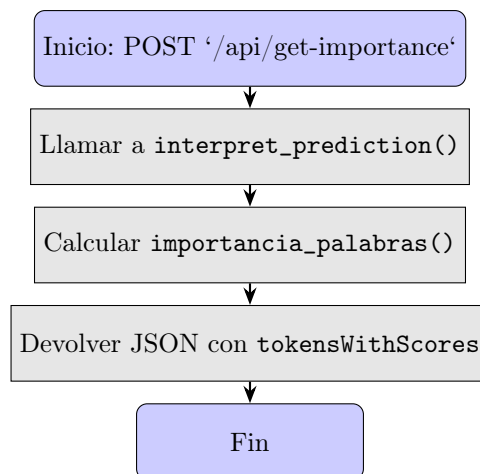


Figura 4.12: Flujo funcional para la generación de mapas de saliencia en `/api/get-importance`.

4.3.3. Frontend

El frontend de la aplicación ha sido desarrollado utilizando la biblioteca `React.js`, orientada a la construcción de interfaces de usuario reactivas mediante componentes funcionales. Su propósito principal es ofrecer una plataforma visual para la exploración y análisis de los síntomas depresivos detectados en las publicaciones de texto.

4.3.3.1 Arquitectura modular

La arquitectura del frontend se estructura en módulos principales, con una clara separación entre componentes contenedores (gestión de estado y lógica de flujo) y componentes presentacionales (visualización de datos):

- `App.js`: punto de entrada principal del frontend. Renderiza el componente `Fetchpublicaciones`.
- `Fetchpublicaciones.js`: componente raíz del flujo funcional. Gestiona los parámetros de entrada del usuario (palabra clave, fuente, cantidad de publicaciones), realiza solicitudes POST al backend y almacena los resultados para su visualización.
- `Dashboard.js`: componente de análisis general. Presenta visualizaciones agregadas (frecuencia de síntomas, distribución de etiquetas, heatmap de correlaciones) y permite la navegación hacia publicaciones individuales.
- `HeatmapCorrelacion.js`: módulo dedicado a la visualización de correlaciones entre síntomas en el Dashboard, utilizando matrices de calor (heatmaps) calculadas a partir de coeficientes de Pearson.
- `PostViewer.js`: componente detallado de visualización individual. Muestra el texto de la publicación con resaltado de síntomas, una barra de probabilidad de depresión y navegación secuencial entre publicaciones.
- `utils/HighlightText.js`: componente utilitario para resaltar palabras dentro de un texto según su puntuación de relevancia sintomática. Usa colores semitransparentes para codificar la intensidad de cada token en función del síntoma especificado.

- `utils/processData.js`: módulo auxiliar que contiene constantes para la asignación de descripciones y colores a los síntomas.

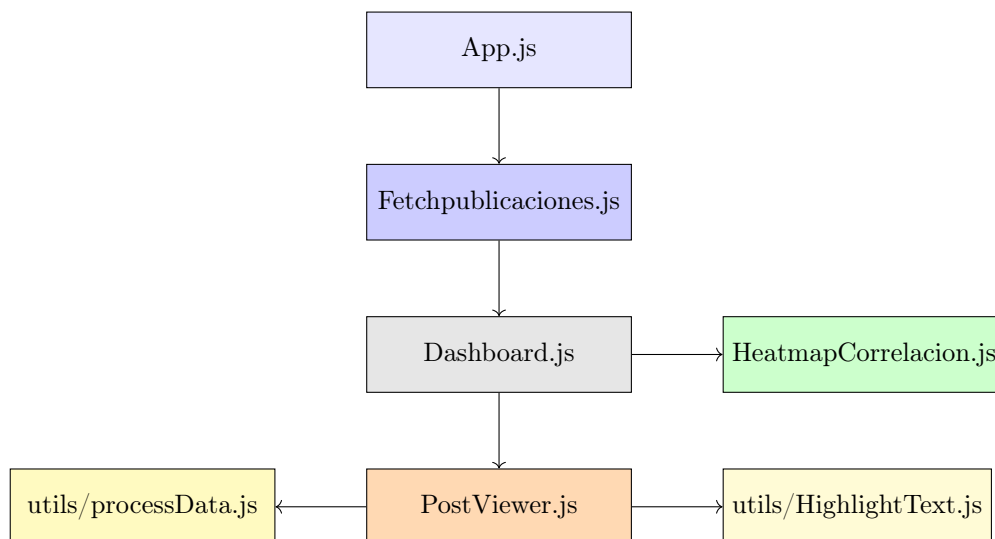


Figura 4.13: Arquitectura modular del frontend y sus dependencias.

4.3.3.2 Flujo funcional

1. El flujo comienza con la interacción del usuario, quien introduce parámetros como la palabra clave, la fuente de datos (Tumblr o archivo CSV) y el número de publicaciones deseado. Estos parámetros son gestionados por el componente `Fetchpublicaciones.js`, que realiza una solicitud POST al backend. La respuesta contiene una lista de publicaciones procesadas y etiquetadas.
2. Una vez recibidos los datos, `Fetchpublicaciones.js` los almacena y activa la renderización condicional del componente `Dashboard.js`. Este componente genera visualizaciones agregadas, como gráficos de frecuencia de síntomas, distribución de publicaciones con y sin indicios de depresión, y una matriz de correlación de síntomas representada como mapa de calor, mediante el módulo `HeatmapCorrelacion.js`.
3. El usuario puede interactuar con estas visualizaciones y seleccionar una publicación individual para abrir el componente `PostViewer.js`. Este muestra un análisis de la publicación seleccionada, con los síntomas resaltados dinámicamente usando el componente auxiliar `HighlightText.js`, que aplica una codificación de color cuya opacidad es proporcional a la relevancia del síntoma en cada palabra. La codificación y las descripciones de síntomas provienen del módulo `processData.js`. También se presenta una barra con la probabilidad estimada de depresión y una leyenda interactiva que permite explorar los síntomas resaltados.

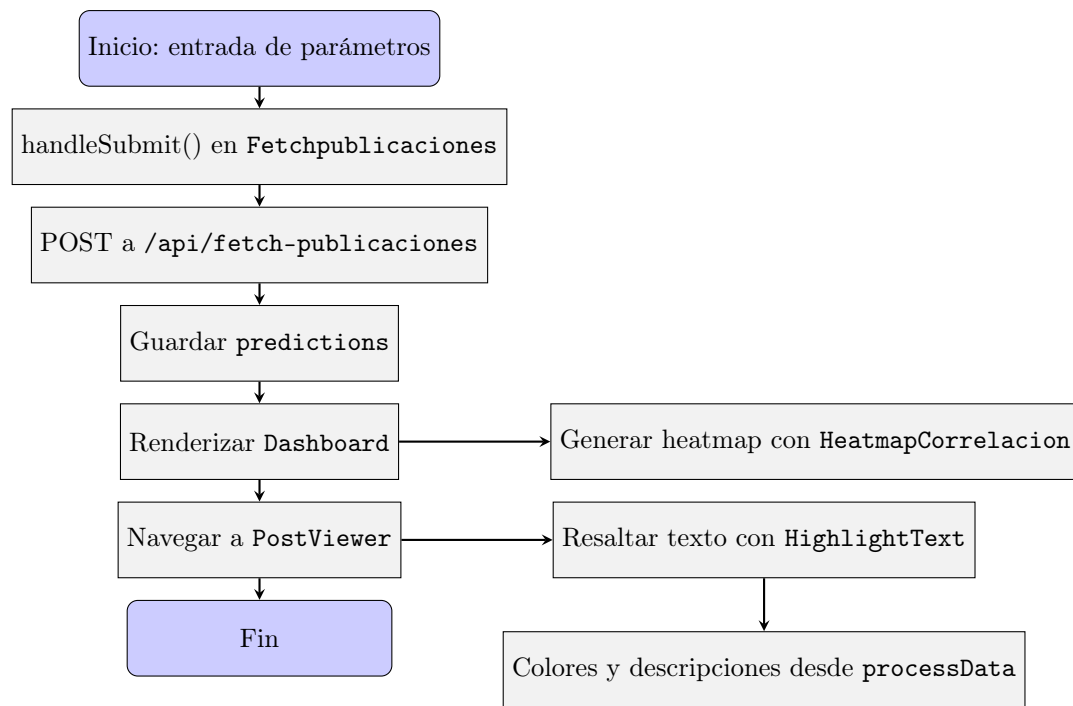


Figura 4.14: Diagrama de flujo funcional del frontend.

5. Resultados

5.1. Evaluación del modelo

En esta sección se presentan los resultados de la evaluación del modelo propuesto. Primero se examina el desempeño de la configuración final durante las fases de entrenamiento y validación; posteriormente, se analiza su capacidad de generalización en un conjunto de prueba independiente y en un escenario real con publicaciones de Tumblr. Finalmente, se evalúa su potencial como detector binario de casos con alta probabilidad de depresión.

5.1.1. Evaluación de predicción de Síntomas con dataset de Reddit

Durante la validación del modelo, se había seleccionado la época cuya evaluación en validación era la siguiente:

Conjunto	Loss	Accuracy	F1 Micro	F1 Macro	Hamming
Entrenamiento	0.1180	0.9527	0.9378	0.9378	0.0473
Validación	0.2383	0.9086	0.8829	0.8828	0.0914

Tabla 5.1: Resultados de entrenamiento y validación para la época 4

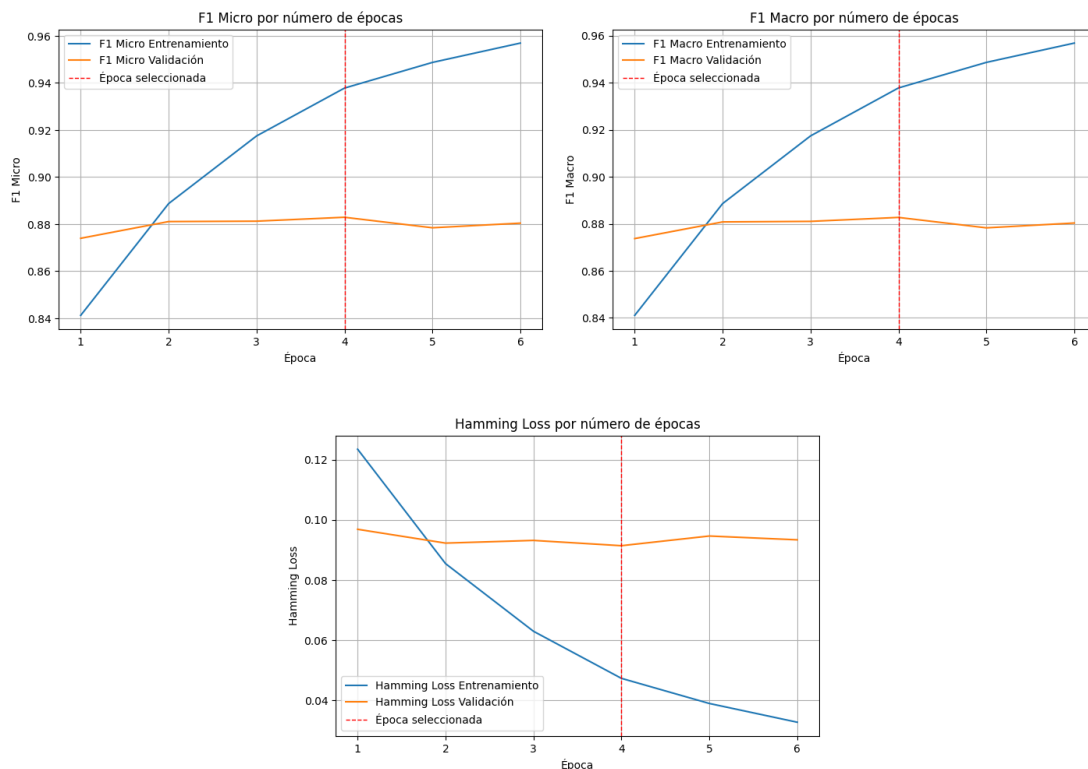


Figura 5.1: F1 Micro, F1 Macro y Hamming Loss en entrenamiento y validación para cada época.

Estos valores reflejan un rendimiento robusto del modelo, especialmente en términos de **F1**, lo cual es indicativo de una buena capacidad de detección de múltiples etiquetas por instancia. Asimismo, el bajo valor de **Hamming Loss** confirma un reducido número de errores por etiqueta.

Una vez finalizado el proceso de entrenamiento y validación, el modelo seleccionado fue evaluado sobre un **conjunto de test independiente**, el cual no fue utilizado en ninguna etapa previa del desarrollo. Esta evaluación permite estimar de manera objetiva la capacidad de generalización del modelo ante datos no vistos.

Interpretación de métricas en test

Las **métricas** obtenidas por el modelo en el conjunto de test, ordenadas según su relevancia práctica para tareas de clasificación multilabel son las siguientes:

- **F1 micro: 0.8792.** Representa el balance entre precisión y exhaustividad, considerando todas las etiquetas de forma agregada. Es la métrica más confiable en escenarios multilabel con clases desbalanceadas.
- **Recall: 0.9047.** Indica que el modelo recupera correctamente el 90.47 % de las etiquetas verdaderas, en promedio por clase. Una alta sensibilidad garantiza que pocas etiquetas relevantes se omiten.
- **Precisión: 0.8561.** Refleja que el 85.61 % de las etiquetas predichas como positivas son efectivamente correctas, en promedio por clase. Evalúa la fidelidad del modelo al predecir etiquetas positivas.
- **F1 macro: 0.8793.** Promedia el equilibrio entre precisión y recall para cada clase, siendo útil cuando se busca un rendimiento equitativo entre etiquetas frecuentes y raras.
- **Hamming Loss: 0.0945.** El modelo comete errores en un 9.45 % de las etiquetas por muestra, lo que indica un bajo nivel de equivocación a nivel etiqueta.
- **Precisión media: 0.9489.** Representa el área bajo la curva precisión-recall por clase. Es robusta ante clases desbalanceadas y útil cuando el modelo produce salidas probabilísticas.
- **ROC AUC: 0.9706.** Indica una capacidad de discriminación promedio del 97.06 % entre etiquetas relevantes e irrelevantes. Cuanto más alto, mejor separación entre clases.
- **Exact Match Ratio: 0.6539.** El 65.39 % de las instancias fueron clasificadas con todas sus etiquetas correctamente. Aunque muy exigente, es útil como referencia de máxima precisión.
- **Error de cobertura: 4.1629.** En promedio, se requieren 4.16 etiquetas ordenadas por probabilidad para cubrir todas las verdaderas. Valores bajos indican buena priorización de etiquetas relevantes.
- **Label Ranking Loss: 0.0331.** Solo el 3.31 % de los pares de etiquetas están mal ordenados, es decir, el modelo prioriza correctamente las etiquetas relevantes en la mayoría de los casos.

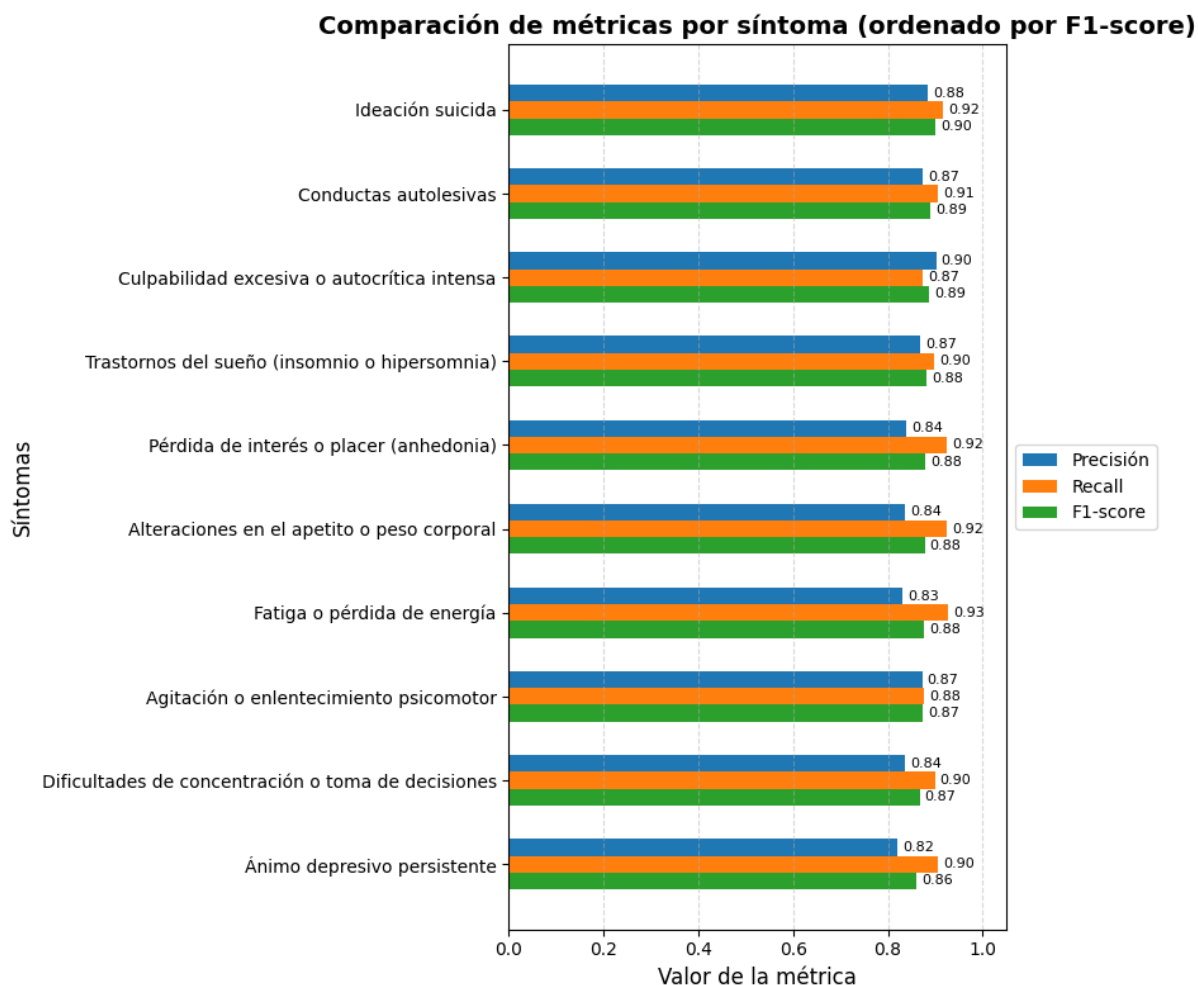


Figura 5.2: Distribución de Precision, Recall y F1-score en los distintos síntomas.

La comparación entre precisión, recall y F1-score para los distintos síntomas, mostrada en la 5.2, indica que el modelo mantiene un **desempeño homogéneo en todas las clases**, con valores de F1 que oscilan entre 0.86 y 0.90. Esto sugiere una capacidad de generalización equilibrada, sin un deterioro marcado en síntomas específicos.

En la mayoría de los síntomas el recall supera levemente a la precisión, lo que indica una inclinación del modelo hacia una **mayor sensibilidad diagnóstica** (prioriza evitar falsos negativos), un aspecto especialmente relevante en aplicaciones clínicas, donde los falsos negativos pueden tener consecuencias más graves que los falsos positivos.

En consecuencia, el análisis por clase confirma que el modelo no solo alcanza métricas globales satisfactorias, sino que también demuestra solidez en la identificación precisa de cada síntoma por separado.

5.1.2. Evaluación con publicaciones recogidas de Tumblr

Con el objetivo de estimar el rendimiento del modelo en su dominio de aplicación real, se recogió una muestra de 324 publicaciones de Tumblr mediante la aplicación desarrollada, volviendo a etiquetarse cada una de ellas manualmente. Estos datos se pueden interpretar como una aproximación

al desempeño real del modelo:

- **Exact Match Ratio: 0.493:** Aproximadamente el 49.3% de las publicaciones fueron etiquetadas con todas las etiquetas correctamente.
- **F1 micro: 0.664, Precisión micro: 0.737, Recall micro: 0.605:** Indicadores aceptables que sugieren un rendimiento moderado.
- **F1 macro: 0.671, Precisión macro: 0.727, Recall macro: 0.660:** Valores estables entre clases, aunque afectados por etiquetas con escasa representación.
- **Hamming Loss: 0.091:** La tasa de error por etiqueta sigue siendo baja, lo cual refleja una buena generalización a nivel individual de etiquetas.
- **F1 weighted: 0.658:** Promedio ponderado que refleja el impacto de las clases más frecuentes.

Los resultados muestran una reducción en el rendimiento con respecto al conjunto de test original, lo cual se debe a distintas causas.

- **Diferencias estilísticas entre plataformas:** En ambos conjuntos se recogen publicaciones de la temática de la depresión; no obstante, el **estilo y formato de las publicaciones** varía ligeramente. En Reddit, los usuarios redactan sus publicaciones de manera más estructurada y explícita, mientras que en Tumblr el estilo tiende a ser más informal, emocional y fragmentado. Estas diferencias pueden afectar al modelo, ya que los patrones a los que se ajusta durante el entrenamiento difieren de la forma lingüística presente en el caso real. Además, en Tumblr se recurre con mayor frecuencia a jerga juvenil y expresiones coloquiales, así como a abreviaturas y eufemismos para evitar ser explícito en temas delicados, lo que representa un desafío adicional para una clasificación precisa.

I can't stop feeling restless, please help me. It's my day off, there's a lot of things going on in my life and in my head that I don't wanna talk about, so please without any context, how do I stop feeling so restless. My heart's racing thinking about bad things, and I can't stop it. There's reasons to be restless but I'm not taking any actions to make things better and even the thought of doing anything is making my head hurt. I'm making a post because I'm feeling lonely as well, and don't wanna just google. I want a real person to guide me, i mean, if you get it then you get it.

Publicación de Reddit

TW!! Suic*de Rant [[MORE]] It's impressive how some people don't immediately result to suic*de as soon as something goes slightly wrong. To me, it just seems like the only logical solution that can fix all my problems!! (•̀ಠ_ಠ)

Publicación de Tumblr

Figura 5.3: Ejemplos comparativos de publicaciones sobre salud mental en Reddit y Tumblr. En Reddit, el texto se presenta de forma estructurada y explícita, con una narrativa clara y una solicitud directa de ayuda. En contraste, la publicación de Tumblr emplea un estilo más fragmentado y emocional, utilizando eufemismos ("suic*de") y emoticones para expresar malestar, lo que refleja el uso frecuente de jerga juvenil en esta plataforma.

- **Diferencias en la distribución de clases:** El conjunto de Tumblr presenta un **menor número de ejemplos** y una **distribución distinta de los síntomas**, con etiquetas

subrepresentadas y frecuencias que difieren respecto al conjunto de entrenamiento. Esta discrepancia dificulta que el modelo generalice adecuadamente, dado que DistilBERT aprende patrones y umbrales basados en la distribución observada durante el entrenamiento. Cambios en la frecuencia o co-ocurrencia de etiquetas en la inferencia pueden conducir a la subestimación de etiquetas menos frecuentes y a la sobrerrepresentación de las más comunes, afectando negativamente métricas como precisión y F1. Además, el modelo no cuenta con mecanismos intrínsecos para adaptarse automáticamente a estas variaciones sin un proceso de reentrenamiento.

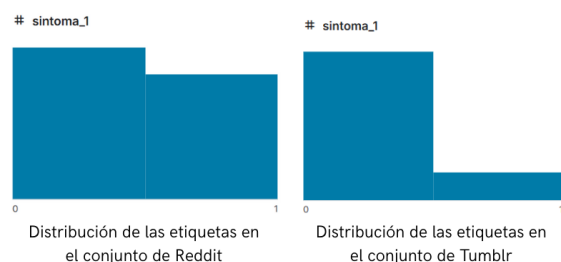


Figura 5.4: Distribución de etiquetas para un síntoma específico en los conjuntos de Reddit y Tumblr. Se observa un desequilibrio más marcado en el conjunto de Tumblr, con una menor proporción de ejemplos positivos. Esta diferencia en la distribución puede afectar la capacidad del modelo para generalizar correctamente, especialmente al enfrentarse a clases subrepresentadas durante la inferencia.

- **Tamaño del conjunto de evaluación:** El conjunto de Tumblr cuenta con únicamente 324 ejemplos, lo cual restringe el poder estadístico del análisis y **aumenta la sensibilidad del rendimiento a pequeñas variaciones**, ocasionadas por un reducido número de errores o aciertos, por lo que el rendimiento fluctúa más fácilmente.
- **Calidad del etiquetado manual vs. semi-automático:** El conjunto de entrenamiento original se etiquetó mediante un proceso semi-automático, el cual, aunque eficiente para grandes volúmenes de datos, puede introducir **ruido**. Este tipo de etiquetado puede no reflejar con precisión la realidad semántica o clínica de las muestras, afectando la calidad del aprendizaje del modelo. Por otro lado, el etiquetado manual realizado en la muestra de Tumblr, aunque potencialmente más preciso, puede resultar poco confiable si no es realizado por expertos en la materia.

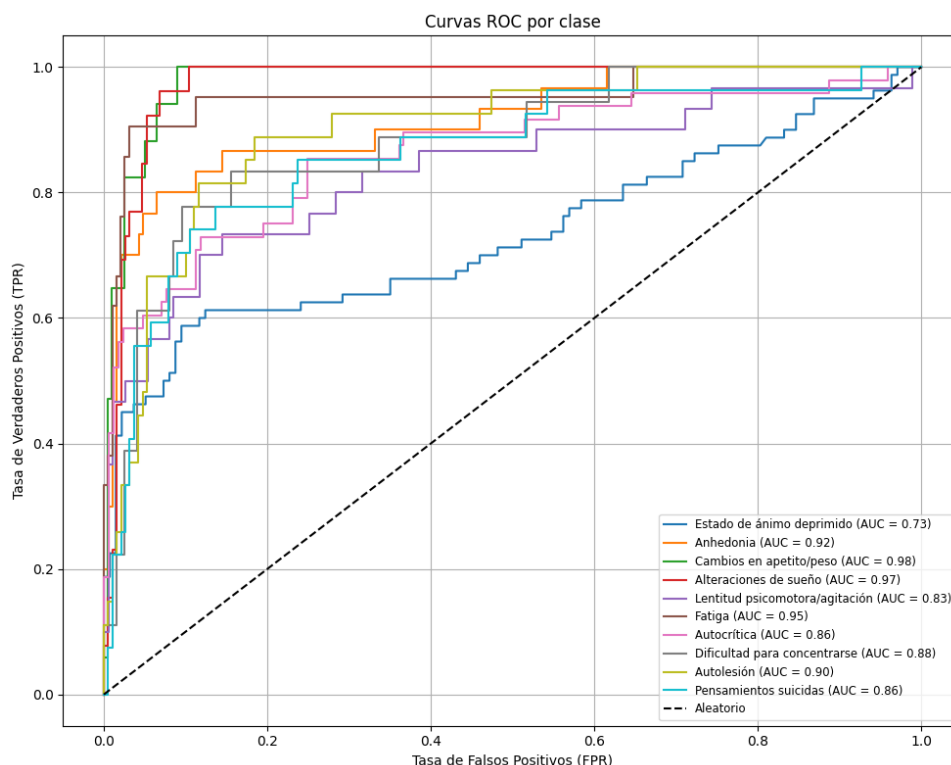


Figura 5.5: Curvas ROC por clase para el conjunto de publicaciones de Tumblr.

La Figura 5.5 muestra las curvas ROC (Receiver Operating Characteristic) para cada una de las etiquetas sintomáticas en el conjunto de evaluación de Tumblr. En general, se observa un rendimiento sólido en la mayoría de las clases, con valores de AUC superiores a 0.85 en ocho de las diez etiquetas, lo cual indica una buena capacidad discriminativa. Etiquetas como *cambios en apetito/peso* (AUC = 0.98), *alteraciones de sueño* (AUC = 0.97) y *fatiga* (AUC = 0.95) destacan por su alta separabilidad.

Por el contrario, *estado de ánimo deprimido* presenta un valor de AUC más bajo (0.73), lo cual sugiere mayores dificultades para distinguir correctamente entre casos positivos y negativos en esta categoría. Esto podría deberse a la ambigüedad o sutileza con la que este síntoma se manifiesta en los textos de Tumblr.

La línea diagonal representa el clasificador aleatorio (AUC = 0.5), y permite visualizar que todas las curvas están claramente por encima de dicho umbral, evidenciando que el modelo supera ampliamente un comportamiento al azar. Estos resultados apoyan la validez del modelo en la tarea de detección por clase, aunque también ponen de manifiesto variaciones importantes en el rendimiento entre etiquetas.

5.1.3. Evaluación de la predicción de la etiqueta de depresión

Con el fin de explorar la capacidad del modelo como **detector binario** de casos con alta probabilidad de depresión, se empleó el dataset **reddit-depression-dataset** de Kaggle. Partiendo del modelo de clasificación multietiqueta y en concordancia con los criterios diagnósticos del DSM-5, se asumió que una **instancia con cinco o más síntomas** corresponde a un caso probable de depresión clínica.

- **Accuracy: 0.772:** Nivel aceptable de aciertos generales.
- **Precisión: 0.969:** Alta proporción de verdaderos positivos entre los casos detectados.
- **Recall: 0.609:** Se recupera aproximadamente un 61 % de los casos depresivos.
- **F1-score: 0.748:** Buen balance entre precisión y cobertura.

Estos resultados sugieren que el modelo es capaz de distinguir publicaciones con indicios de depresión cuando se aproxima mediante el número de síntomas detectados. No obstante, el umbral elegido para definir la presencia de depresión podría ajustarse con el fin de incrementar la sensibilidad, dado que el modelo tiende a ser conservador y prioriza reducir los falsos positivos.

- **Umbral clásico** (conteo de etiquetas positivas): A partir de las predicciones binarizadas para los 10 síntomas ($\hat{y}_i \in \{0, 1\}$), se calcula el total de síntomas presentes:

$$S = \sum_{i=1}^{10} \hat{y}_i$$

La instancia se clasifica como caso depresivo si al menos cinco etiquetas son positivas:

$$\text{Depresión} = \begin{cases} 1 & \text{si } S \geq 5 \\ 0 & \text{si } S < 5 \end{cases}$$

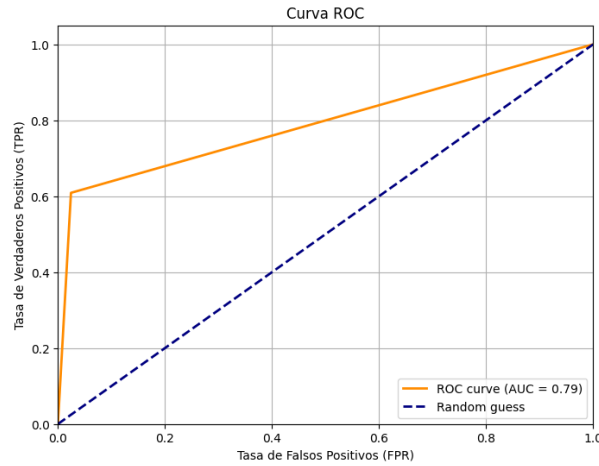


Figura 5.6: Curva ROC con la aproximación binaria basada en ≥ 5 síntomas.

Con el propósito de mejorar esta discriminación, se evaluaron dos variantes basadas en **valores de probabilidad continuos**, derivados directamente de las predicciones de cada síntoma ($p_i \in [0, 1]$):

- **Suma ponderada de probabilidades:**

Se suman las probabilidades de cada síntoma, ponderadas de forma uniforme por 0.1:

$$P_1 = \sum_{i=1}^{10} 0,1 \cdot p_i = 0,1 \cdot \sum_{i=1}^{10} p_i$$

La clasificación final depende de un umbral ajustable t_1 :

$$\text{Depresión} = \begin{cases} 1 & \text{si } P_1 \geq t_1 \\ 0 & \text{si } P_1 < t_1 \end{cases}$$

■ **Escalado de probabilidades con truncamiento:**

Se amplifican las probabilidades multiplicándolas por 2 y limitándolas a un máximo de 1.0:

$$p'_i = \min(2 \cdot p_i, 1,0)$$

Posteriormente, se suman los valores transformados:

$$P_2 = \sum_{i=1}^{10} p'_i$$

Y se aplica el umbral de decisión t_2 :

$$\text{Depresión} = \begin{cases} 1 & \text{si } P_2 \geq t_2 \\ 0 & \text{si } P_2 < t_2 \end{cases}$$

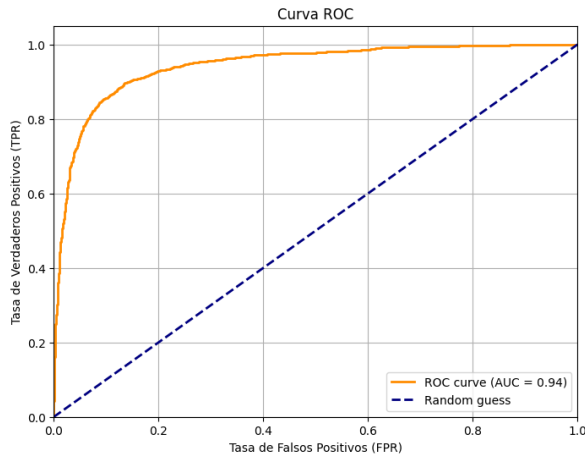


Figura 5.7: Curva ROC del método de suma ponderada de probabilidades (AUC = 0.94).

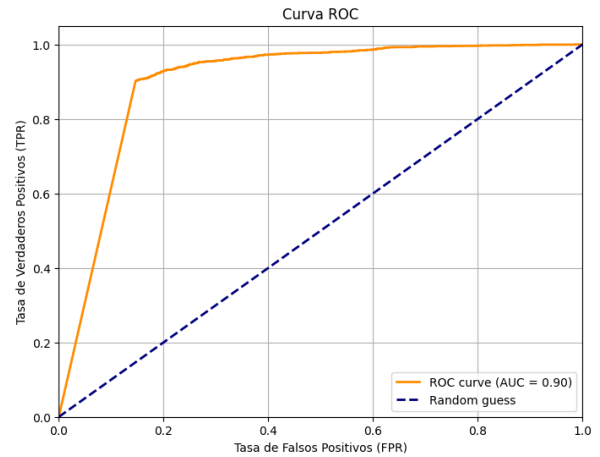


Figura 5.8: Curva ROC del método de escalado y truncamiento de probabilidades (AUC = 0.90).

Los resultados obtenidos muestran que, tomando como referencia la etiqueta binaria de depresión, los métodos basados en valores continuos alcanzan un AUC de 0.94 y 0.90, respectivamente, superando de manera clara el AUC de 0.79 del umbral clásico de cinco síntomas. Esto evidencia que la incorporación de la información probabilística en la decisión final permite ajustar con mayor flexibilidad los umbrales, mejorando la sensibilidad del sistema sin comprometer en exceso la precisión. En consecuencia, los enfoques basados en probabilidades constituyen una alternativa más robusta para la detección automática de casos con alta probabilidad de depresión.

5.2. Evaluación de la aplicación web

En esta sección se describe una prueba práctica que permite evaluar el funcionamiento de la aplicación web desarrollada. A través de un ejemplo completo de búsqueda, análisis predictivo y visualización de resultados, se demuestra la efectividad y usabilidad del sistema.

1. Pantalla de búsqueda

La pantalla inicial permite al usuario introducir los parámetros de búsqueda o cargar sus propios datos:

- **Campo de texto:** Requiere una palabra clave (etiqueta o comunidad) para realizar la búsqueda en Tumblr. Si se deja vacío, se muestra un mensaje de error. Este campo es obligatorio.
- **Campo numérico:** Permite seleccionar la cantidad de publicaciones a analizar (entre 1 y 1000), respetando los límites de la API.
- **Botones de selección:** Permiten elegir entre búsqueda por etiqueta o por comunidad. Actualmente, la opción de comunidad está deshabilitada por problemas de autenticación.
- **Botones de acción:**
 - **Buscar:** Inicia la búsqueda con los parámetros especificados.
 - **Subir mi documento:** Permite al usuario cargar un archivo CSV con publicaciones propias.

2. Proceso de análisis y barra de progreso

Al iniciar la búsqueda, se muestra una barra de progreso dividida en dos fases:

1. **Recolección de publicaciones:** Avanza rápidamente mientras se recuperan publicaciones desde la API de Tumblr.
2. **Análisis de textos:** Avanza más lentamente a medida que los textos son analizados mediante el modelo.

La barra de progreso es simulada, basada en estimaciones de tiempos promedio para mantener una experiencia de usuario fluida.

$$t_{\text{recolección}}(n) = 0,000175n^2 + 0,01775n + 0,475$$

$$t_{\text{predicción}}(n) = 0,00025n^2 + 0,0825n + 5,25$$

Si el backend termina antes, se avanza automáticamente a la pantalla de resultados. Si tarda más, la barra se detiene en 99 % hasta finalizar. Un mensaje sobre la barra indica la etapa actual (Recolectando publicaciones... o Analizando publicaciones...).

3. Filtrado de publicaciones

Durante la recolección, se aplican filtros automáticos para asegurar la calidad de los datos analizados:

- Se descartan publicaciones vacías o que contienen solo contenido multimedia.
- Se eliminan publicaciones en idiomas distintos al inglés, ya que el modelo está entrenado únicamente en ese idioma.

El sistema intenta recuperar el número solicitado de publicaciones mediante múltiples llamadas a la API. Si, tras varios intentos, no se logra obtener la cantidad deseada, se procesa la cantidad recuperada. En caso de no recuperarse ninguna publicación válida, se notifica al usuario y se regresa a la pantalla de búsqueda.

4. Pantalla de resultados y visualizaciones

Una vez completado el análisis, se despliega un Dashboard interactivo que permite explorar los datos de forma global e individual:

- **Proporción de publicaciones con indicios de depresión:** Gráfico de pastel que muestra el porcentaje de publicaciones clasificadas como indicativas de depresión.
- **Distribución de síntomas:** Gráfico de barras con la frecuencia de aparición de cada síntoma.
- **Matriz de correlación de síntomas:** Heatmap que muestra la coocurrencia de síntomas en las publicaciones, revelando posibles patrones de comorbilidad.

También se presenta una lista desplazable de publicaciones, que puede filtrarse por síntomas detectados. Cada publicación puede abrirse con un botón **ver detalles**, además de incluir opciones para **volver a la búsqueda** o **descargar los datos en CSV**.

5. Exploración individual de publicaciones

Al seleccionar una publicación, se despliega una vista detallada con la siguiente información:

- Texto original con **palabras resaltadas** según su contribución a la predicción.
- Barra de progreso que representa visualmente la **probabilidad estimada de depresión**.
- Leyenda interactiva para activar o desactivar el resaltado por síntoma.

6. Visualización mediante mapas de saliencia

La aplicación emplea mapas de saliencia para identificar y resaltar las palabras que más han influido en la predicción de cada síntoma. Estos se basan en el gradiente de la salida del modelo con respecto a las representaciones internas del texto.

Las palabras con mayor impacto se muestran con una mayor intensidad de color, lo que favorece la transparencia y comprensión del razonamiento del modelo. El usuario puede activar o desactivar el resaltado para cada síntoma.

5.2.1. Prueba práctica del sistema

Para validar la funcionalidad, se realiza una búsqueda real con la etiqueta #mentally ill y se analiza:

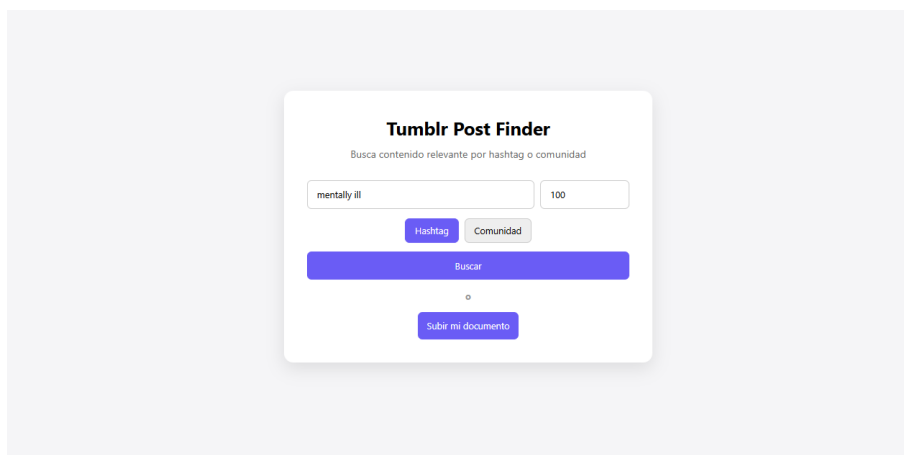


Figura 5.9: Interfaz del sistema antes de una búsqueda con la etiqueta #mentally ill.

Para 100 publicaciones, el tiempo estimado de recolección se calcula como $0,000175 \cdot 100^2 + 0,01775 \cdot 100 + 0,475 = 4$ segundos, y el tiempo estimado de predicción como $0,00025 \cdot 100^2 + 0,0825 \cdot 100 + 5,25 = 16$ segundos. En la captura de pantalla de la consola, el tiempo real de espera es de 6.11 segundos para la recolección de publicaciones y 18.22 segundos para la predicción. Se puede observar que, en cada llamada a la API, casi todas las publicaciones son válidas (la etiqueta está en inglés, por lo que la mayoría de las publicaciones también están en inglés y contienen pocas imágenes). Dado que el tiempo de espera real es ligeramente mayor que el estimado, el proceso se detiene brevemente al alcanzar el 99 % de avance.

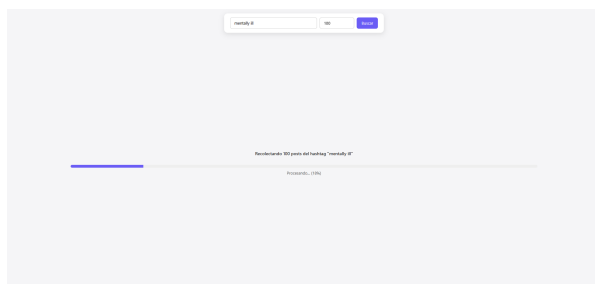


Figura 5.10: Barra de progreso durante la recolección de publicaciones.

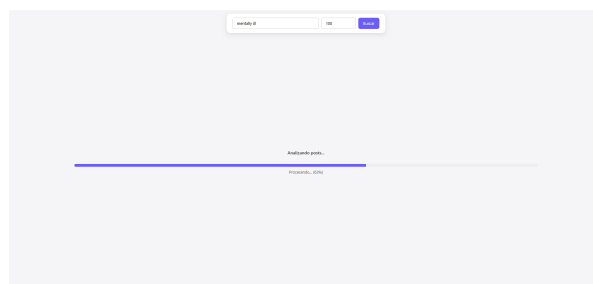


Figura 5.11: Barra de progreso durante la predicción de síntomas.

```

>> Se llamó al endpoint /api/fetch-posts
Datos recibidos: {'sourceType': 'tag', 'inputValue': 'mentally ill', 'nrPosts': 100}
+ Inicializando recolección. nr_posts requeridos: 100, ya hay 0 válidos.
API_KEY cargada: 1C1C2B9DDg9kex6r7mR0f3gMdn1AvMfg01AsouHPiUNFuC5PB
Faltan 80 textos válidos, recolectando más...
API_KEY cargada: 1C1C2B9DDg9kex6r7mR0f3gMdn1AvMfg01AsouHPiUNFuC5PB
Faltan 62 textos válidos, recolectando más...
API_KEY cargada: 1C1C2B9DDg9kex6r7mR0f3gMdn1AvMfg01AsouHPiUNFuC5PB
Faltan 44 textos válidos, recolectando más...
API_KEY cargada: 1C1C2B9DDg9kex6r7mR0f3gMdn1AvMfg01AsouHPiUNFuC5PB
Faltan 24 textos válidos, recolectando más...
API_KEY cargada: 1C1C2B9DDg9kex6r7mR0f3gMdn1AvMfg01AsouHPiUNFuC5PB
Faltan 5 textos válidos, recolectando más...
API_KEY cargada: 1C1C2B9DDg9kex6r7mR0f3gMdn1AvMfg01AsouHPiUNFuC5PB
Total textos válidos: 100
Tiempo de recolección para 100 posts: 6.11 segundos
[Importancia calculada: min=51.191956, max=569.841492, range=518.649536
[Importancia calculada: min=43.462288, max=379.877258, range=327.414970
[Importancia calculada: min=24.714873, max=721.288762, range=696.566689
[Importancia calculada: min=6.489577, max=31.740147, range=25.330569
[Importancia calculada: min=5.279493, max=170.584595, range=165.305101
[Importancia calculada: min=75.768562, max=1352.503906, range=1276.735344
[Importancia calculada: min=29.451494, max=308.832184, range=279.380690
[Importancia calculada: min=103.839020, max=766.484924, range=662.645905
[Importancia calculada: min=159.467926, max=1397.961304, range=1238.493378
[Importancia calculada: min=139.262970, max=671.278076, range=532.015106
[Importancia calculada: min=12.362532, max=508.383636, range=496.021105
[Importancia calculada: min=34.552040, max=604.448547, range=569.896507
Tiempo de predicción para 100 posts: 18.22 segundos
127.0.0.1 - - [30/Jun/2025 11:49:54] "POST /api/fetch-posts HTTP/1.1" 200 -

```

Figura 5.12: Resultados mostrados en consola.

A continuación se muestra el dashboard, donde todos los graficos muestran la información.

Además, es posible descargar los resultados de la predicción en formato CSV.

texto	Síntomas (síntoma)									
	1	2	3	4	5	6	7	8	9	10
tw: sh hey guys! just wanted to tell everyon...	0	0	0	0	0	0	0	0	1	0
SELF H4RM WARNING! - - - [[MORE]] **so**	0	0	0	0	0	0	0	0	1	0
tw self harm - - - i cut myself 60 times yeste...	0	0	0	0	0	0	0	0	1	0
i hate being clean from cutting just for summe...	0	0	0	0	0	0	0	0	1	0
oh, the way i feel the need to relapse and dle...	0	0	1	0	0	0	0	0	0	0
tw sh talk [[MORE]] recently I've been limitin...	0	0	0	0	0	0	0	0	1	0
tw sh [[MORE]] I actually hate my scars so muc...	0	0	0	0	0	0	0	0	1	0
Anyone else just reuse old bandages until they...	0	0	0	0	0	0	0	0	1	0
Waking up after cutting session that you were ...	0	0	0	1	0	0	0	0	1	0
We are at wound 31 right now. 31 sh wounds sin...	0	0	0	0	0	0	0	0	1	0

Tabla 5.2: Estructura del archivo CSV generado con las publicaciones y sus predicciones por síntoma.

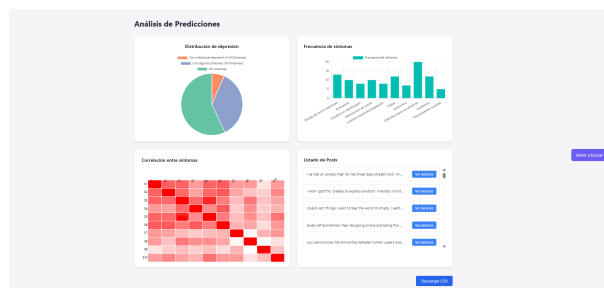


Figura 5.13: Panel de visualización con resultados y gráficos de análisis.

El diagrama de tarta indica que más de la mitad de las publicaciones analizadas no presentan síntomas asociados a la depresión. Entre las publicaciones restantes, la mayoría contiene al menos un síntoma, aunque sin alcanzar los criterios para considerar depresión clínica; únicamente un pequeño porcentaje presenta cinco o más síntomas, lo cual sugiere un posible riesgo.

En relación con la distribución de los síntomas detectados, la mayoría se encuentran en torno a los 10 casos, con excepción de los pensamientos suicidas, que se detectan en solo cinco publicaciones, y la dificultad para concentrarse, presente en 20 casos.

El análisis de correlación muestra que los primeros seis síntomas presentan una correlación relativamente alta entre sí, mientras que el síntoma número ocho exhibe una correlación considerablemente baja con los demás.

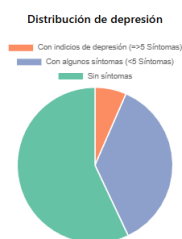


Figura 5.14: Distribución del nivel de depresión detectado.

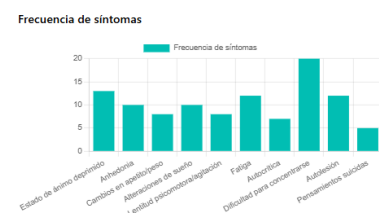


Figura 5.15: Frecuencia de aparición de cada síntoma.

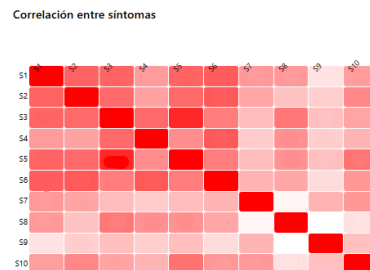


Figura 5.16: Correlación entre síntomas detectados.

Para un análisis más detallado, se pueden examinar publicaciones individuales.

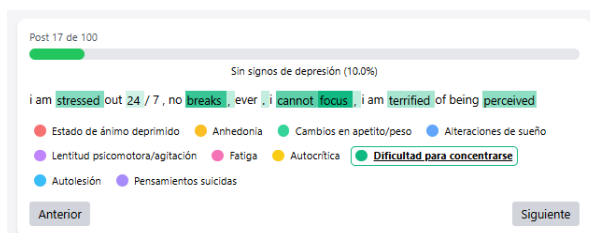


Figura 5.17: Publicación con predicción positiva para el síntoma "dificultad para concentrarse".

Se evidencia que, en numerosos casos, el modelo no detecta la totalidad de los síntomas presentes en una publicación. Por ejemplo, en el caso 5.14, aunque el modelo identifica correctamente la dificultad para concentrarse, podrían inferirse otros síntomas adicionales, como agitación, a partir de expresiones tales como "estoy estresado" o "me aterroriza ser percibido".

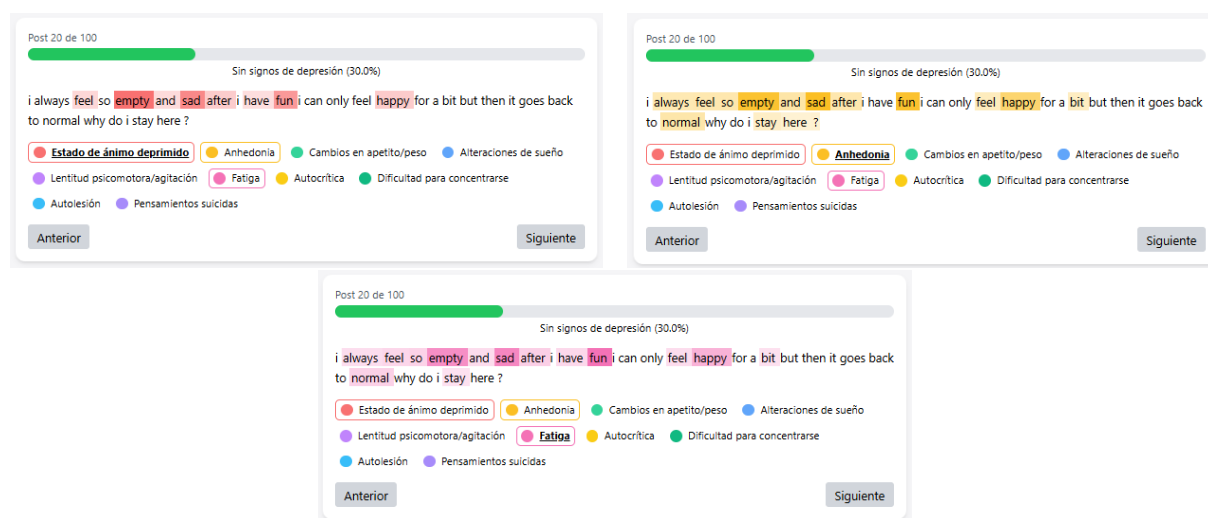


Figura 5.18: Publicación con predicción positiva para los síntomas "estado de ánimo deprimido", "anhedonia", erróneamente, "fatiga".

Asimismo, se presentan casos en los que el modelo predice síntomas que no parecen estar explícitamente manifestados en el texto, como en las figuras 5.15 y 5.16, donde se asigna la etiqueta “fatiga” sin una expresión textual clara que la respalde.

Se aprecia que las palabras resaltadas por el modelo están relacionadas con la depresión en términos generales y, en algunos casos, son específicas para el síntoma analizado. Sin embargo, en la mayoría de los casos, no existe una diferenciación significativa entre las palabras destacadas para los distintos síntomas, lo que sugiere una limitada capacidad del modelo para discriminar y detectar cada síntoma de forma particular.

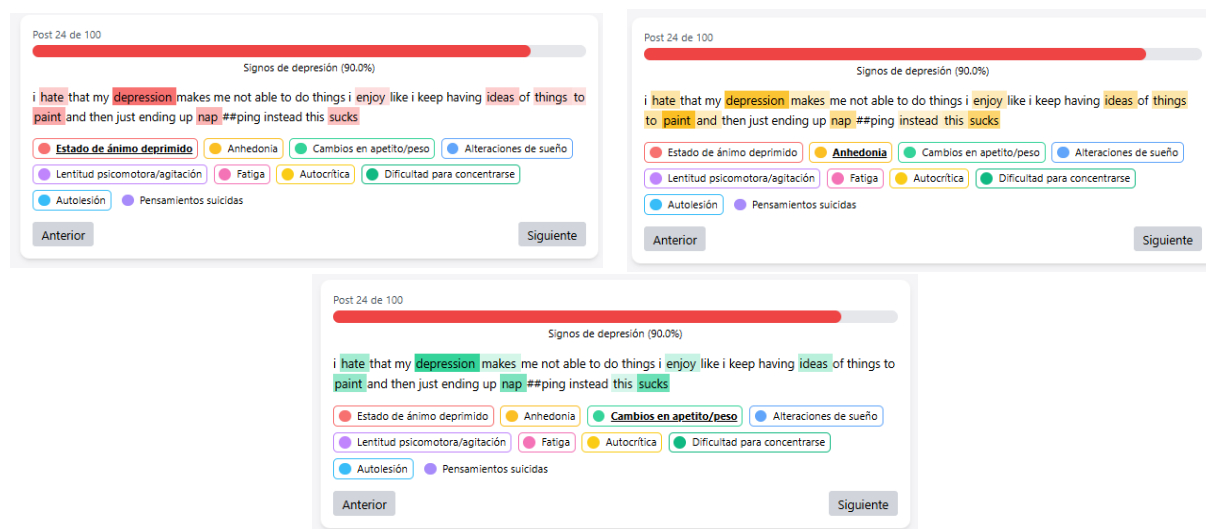


Figura 5.19: Activación de múltiples síntomas a partir de la presencia de la palabra “depresión”.

Gracias a la interpretabilidad proporcionada por los *saliency maps*, se ha identificado un patrón relevante: el modelo tiende a activar numerosas etiquetas cuando detecta la palabra “depresión”. Esto indica que el modelo no distingue adecuadamente entre síntomas específicos, sino que responde a indicadores generales asociados con la depresión, activando etiquetas porque ciertos síntomas suelen co-ocurrir en un mismo texto.

Este comportamiento representa una limitación, ya que, aunque el análisis de la co-ocurrencia de síntomas es clínicamente relevante, en este contexto dificulta que el modelo aprenda a reconocer de forma precisa las características lingüísticas propias de cada síntoma individual.

Una posible estrategia para superar esta limitación sería recopilar ejemplos adicionales de publicaciones que expresen síntomas de manera aislada, sin contexto emocional amplio ni referencias a otros síntomas, lo cual podría favorecer el aprendizaje de patrones más específicos y mejorar la precisión en la detección individual.

6. Impacto del proyecto

El presente proyecto, al igual que el desarrollo de sistemas basados en IA aplicados a la salud mental, conlleva una serie de **implicaciones sociales, éticas, tecnológicas y sanitarias** que deben ser consideradas durante su diseño e implementación. Dada la sensibilidad del ámbito, se ha prestado especial atención al **tratamiento ético de los datos**, a la protección de la **privacidad** y a **transparencia** del modelo, que debe operar de manera responsable.

En primer lugar, se ha priorizado la **protección de la identidad y privacidad** de las personas usuarias. El sistema no almacena ni expone información que permita identificar a los autores de las publicaciones analizadas, tales como nombres de usuario o enlaces. El procesamiento se limita estrictamente al contenido textual, y no se establece ninguna relación entre publicaciones y usuarios concretos. Este enfoque reduce significativamente el riesgo de reidentificación y está alineado con las buenas prácticas y normativas en materia de protección de datos.

Por otro lado, si bien no es posible eliminar por completo los posibles **sesgos** que el modelo pueda heredar de los datos de entrenamiento o del lenguaje característico de la plataforma Tumblr, es fundamental tenerlos presentes al interpretar los resultados, especialmente en contextos sensibles como la salud mental. Para fomentar la **transparencia** y la **confianza** en el sistema, se han incorporado **mapas de saliencia** que permiten visualizar qué fragmentos del texto han influido en las decisiones del modelo, facilitando así la comprensión del razonamiento detrás de cada predicción y permitiendo detectar errores o sesgos involuntarios.

El sistema ha sido concebido como una herramienta de **apoyo a profesionales** de la salud mental, no como un reemplazo de la evaluación clínica. Su función principal es ofrecer análisis y visualizaciones que sirvan como complemento en procesos de diagnóstico o seguimiento, siempre en manos de personas expertas. De este modo, se promueve un uso ético y consciente de la inteligencia artificial, orientado a reforzar la labor profesional y facilitar la toma de decisiones informadas.

Más allá del ámbito clínico, el proyecto tiene un impacto positivo en términos de **accesibilidad tecnológica**. Gracias a su diseño intuitivo, permite que personas sin formación técnica puedan realizar análisis complejos sobre datos textuales relacionados con la salud mental.

Asimismo, este trabajo representa un avance relevante en la aplicación práctica de modelos de lenguaje natural como BERT para el análisis de indicadores emocionales y psicológicos en textos no estructurados, lo que puede sentar las bases para desarrollos más sofisticados como sistemas de alerta temprana, personalización del análisis o integración con otras plataformas.

Además de su aplicabilidad práctica, el sistema podría tener valor como fuente de datos anónimos y agregados para **estudios epidemiológicos**, facilitando así la investigación sobre patrones de salud mental en entornos digitales.

En contextos formativos, puede ser útil como **herramienta didáctica** para estudiantes y profesionales de la psicología, proporcionando un entorno para comprender cómo la inteligencia artificial puede aplicarse al análisis del lenguaje humano en relación con la salud mental.

Asimismo, al ofrecer visualizaciones claras y accesibles de los síntomas detectados, el sistema puede contribuir a la **sensibilización social** sobre la depresión y otros trastornos emocionales, ayudando a generar conciencia sobre la importancia de la salud mental.

Desde el punto de vista de los **Objetivos de Desarrollo Sostenible (ODS)**, el proyecto se

alinea de forma directa con el ODS 3, "**Salud y bienestar**", al facilitar la detección temprana de síntomas relacionados con la depresión, lo que puede permitir intervenciones más oportunas y eficaces, especialmente en poblaciones vulnerables.

7. Presupuesto

La ejecución del proyecto implica la utilización de recursos de software, hardware y personal especializado. Para estimar de manera aproximada los costes asociados, se presentan a continuación los recursos necesarios, su justificación y los gastos previstos en cada categoría.

1. Recursos de Software

- **Kaggle y Google Colab:** se emplearon las versiones gratuitas para entrenamiento y pruebas del modelo, con acceso limitado a GPU. Para un despliegue real o entrenamiento más intensivo, podría optarse por **Google Colab Pro**, que ofrece mayor capacidad de cómputo (100 unidades de cómputo al mes, cada equivalentes aproximadamente a 2 unidades por hora de uso de TPU/T4).
- **Almacenamiento de datos:** en este prototipo los datos se han almacenado localmente. En un escenario de uso real y escalable, sería necesario utilizar una base de datos que gestione grandes volúmenes de publicaciones y resultados de inferencia. Una opción gratuita y eficiente es **PostgreSQL**.

Recurso Software	Precio
Google Colab Pro	11,19 €/mes (aprox. 22 € por 2 meses)
PostgreSQL	Gratuito

Tabla 7.1: Coste de Recursos de Software

2. Recursos de Hardware

Para el desarrollo del prototipo se ha utilizado el siguiente equipo:

Ordenador portátil Asus ZenBook 14 OLED UX3402VA-KM208W

Sus principales características son:

- **Procesador (CPU):** Intel Core i5 1340P, 12 núcleos, hasta 4,6 GHz
- **Memoria RAM:** 16 GB LPDDR5
- **Almacenamiento:** SSD 512 GB NVMe
- **Tarjeta gráfica (GPU):** Intel Iris Xe

Este equipo proporciona un rendimiento adecuado para el desarrollo de la aplicación web y la integración del modelo DistilBERT, siendo suficiente para ejecutar el modelo con cargas moderadas.

Material Hardware	Precio
Ordenador Asus ZenBook 14	1.199 €

Tabla 7.2: Coste de Recursos de Hardware

3. Recursos de Personal

En caso de que el proyecto se implementase, serían necesarios distintos perfiles especializados:

- **Profesional del ámbito de la salud mental:** participaría en el etiquetado de las publicaciones, asegurando una interpretación precisa de los síntomas según criterios clínicos. Su experiencia garantizaría la calidad de los datos para entrenar el modelo de machine learning.

- **Ingeniero de Datos:** se encargaría de la recopilación, limpieza, preprocesamiento y almacenamiento de los datos, así como del desarrollo, ajuste y evaluación de los modelos de machine learning, asegurando eficiencia y precisión en la clasificación de publicaciones.
- **Desarrollador Fullstack:** responsable del diseño, desarrollo e integración de la aplicación web, gestionando tanto el frontend como el backend, incluyendo la conexión con la base de datos y la integración del modelo, garantizando una aplicación funcional y escalable.
- **Supervisor del proyecto:** coordinaría los distintos perfiles, planificaría y supervisaría las etapas del proyecto, asegurando el cumplimiento de plazos y objetivos, y verificando que el desarrollo cumpla con los estándares de calidad establecidos.

Perfil profesional	Sueldo medio anual	Horas	Coste total
Ingeniero de datos	39.500 €	260	3.875 €
Desarrollador Fullstack	33.000 €	260	3.750 €
Supervisor del proyecto	45.500 €	86	1.750 €
Profesional de la salud mental	26.000 €	86	1.042 €
TOTAL	-	-	10.417 €

Tabla 7.3: Coste de Recursos Humanos

7.1. Resumen de Gastos Totales

A continuación se presenta un resumen del coste total del proyecto, considerando software, hardware y recursos humanos:

Categoría	Gasto Total
Software	22 €
Hardware	1.199 €
Recursos Humanos	10.417 €
Gasto Total del Proyecto	11.638 €

Tabla 7.4: Resumen de Gastos Totales

8. Conclusiones y líneas futuras

8.1. Conclusiones

El desarrollo de esta aplicación web ha demostrado la viabilidad de utilizar técnicas avanzadas de procesamiento de lenguaje natural y modelos de aprendizaje automático para la detección temprana de indicios de depresión a partir de publicaciones en redes sociales. La integración de un flujo funcional que abarca desde la recolección de datos hasta la visualización interactiva de resultados, junto con herramientas de interpretación como los mapas de saliencia, aporta transparencia y usabilidad al sistema.

Los resultados obtenidos permiten extraer las siguientes conclusiones principales:

- El modelo entrenado logra un **rendimiento notable sobre datos de validación y test provenientes del mismo dominio de entrenamiento** (Reddit), con métricas F1 superiores al 0.87, precisión media de 0.94 y baja Hamming Loss, lo que demuestra su capacidad para identificar múltiples síntomas con alta exactitud. El análisis por clase muestra una generalización uniforme, sin que se degrade el desempeño en síntomas clínicamente críticos como ideación suicida o autolesiones.
- En el conjunto de publicaciones reales de Tumblr, el modelo mantiene un rendimiento aceptable, aunque decrece de forma moderada (F1 micro de 0.66). Esta disminución se atribuye a diferencias estilísticas entre plataformas, cambios en la distribución de clases y al tamaño reducido de la muestra manualmente anotada. Aun así, los valores de AUC por clase, en su mayoría superiores a 0.85, reflejan una buena capacidad de discriminación sintomática incluso en un dominio informal y heterogéneo.
- El experimento de conversión a tarea binaria sugiere que el modelo también puede ser utilizado como herramienta para la **identificación de casos con alta probabilidad de depresión**. Con una precisión de 0.96 y un F1-score de 0.74, el modelo muestra potencial clínico en contextos donde se prioriza minimizar falsos positivos, como en sistemas de alerta o derivación.
- La aplicación web desarrollada permite una interacción fluida con el sistema, facilitando tanto la **recolección de datos desde Tumblr** como su **análisis** sintomático e interpretación visual. La integración del modelo en esta plataforma demuestra su aplicabilidad práctica y sirve de base para herramientas de investigación en salud mental basada en texto.

La herramienta resulta especialmente útil para la recolección y etiquetado de publicaciones en la red social Tumblr, un espacio del que tradicionalmente se dispone de pocos datos públicos o etiquetados para investigación, lo que facilita el acceso a conjuntos de datos valiosos para estudios futuros.

No obstante, se identificaron ciertas limitaciones. En particular, los **mapas de saliencia** no diferencian claramente entre distintos síntomas, ya que tienden a seleccionar palabras muy similares para varias etiquetas, dificultando establecer una relación directa entre términos específicos y síntomas individuales. Sin embargo, esta técnica sigue siendo útil para visualizar y comprender las decisiones generales del modelo.

Además, algunos síntomas no se detectan con la precisión esperada, posiblemente debido a la calidad y distribución del dataset utilizado para el entrenamiento. A pesar de ello, la detección de la depresión en su conjunto es robusta y fiable.

8.2. Líneas futuras

A partir de los resultados obtenidos y las limitaciones identificadas, se proponen diversas líneas de trabajo orientadas a mejorar las capacidades del sistema desarrollado, además de formas de utilizar la aplicación web.

Una primera línea de desarrollo se centra en la **recolección y corrección continua de datos**. La actual funcionalidad de descarga de predicciones facilita la recopilación sistemática de publicaciones en Tumblr, lo que permitiría construir un corpus específico de esta red social. A este corpus se podrían incorporar mecanismos dentro de la interfaz para la revisión manual de etiquetas, de modo que los usuarios, o idealmente profesionales de la salud mental, puedan corregir aquellas clasificaciones que el modelo haya etiquetado incorrectamente. Estos datos corregidos, específicos del dominio y validados clínicamente, permitirían reentrenar el modelo de forma incremental dentro de la misma aplicación, generando un ciclo de mejora continua que reduzca progresivamente los errores de clasificación.

En este contexto, podrían explorarse diversas técnicas como el fine-tuning supervisado en subconjuntos reducidos de datos de Tumblr, el aprendizaje adversarial, el pseudoetiquetado con revisión parcial, o la adaptación de dominio mediante preentrenamiento no supervisado (Domain-Adaptive Pretraining). Estas estrategias permitirían ajustar mejor el modelo al lenguaje, estilo y particularidades discursivas propias de Tumblr, cerrando así la brecha entre el **dominio fuente** (Reddit) y el **dominio objetivo** (Tumblr).

En paralelo, se ha identificado la necesidad de gestionar adecuadamente el **lenguaje informal**, las **abreviaciones** y los **eufemismos**, ya que muchos términos clave para la detección de síntomas suelen expresarse mediante estas formas lingüísticas. La incorporación de técnicas específicas de normalización o de comprensión contextual de la jerga contribuiría de manera significativa a mejorar el rendimiento del modelo.

También se considera prioritaria la adaptación del sistema a la **lengua española**. Si bien el predominio de recursos en inglés ha facilitado el desarrollo inicial del modelo, la incorporación de modelos multilingües o el uso de traducción automática permitiría ampliar la búsqueda y análisis de publicaciones redactadas en español. Siguiendo la lógica aplicada en la adaptación al dominio de Tumblr, el sistema podría ajustarse también al contexto lingüístico del castellano, permitiendo a los usuarios seleccionar el idioma de análisis. Esta línea es especialmente relevante, dado que la mayoría de los modelos actuales están centrados en el inglés, y avanzar en el desarrollo de modelos específicos para el español contribuiría a reducir esta brecha.

Otra línea de trabajo relevante es la integración de técnicas avanzadas de **interpretabilidad**. Aunque el sistema actual ya proporciona un primer nivel de visualización publicación a publicación, se propone incorporar herramientas más sofisticadas, como mecanismos de atención multicanal, visualizaciones basadas en SHAP o LIME. Estas mejoras permitirían una comprensión más precisa de las decisiones del modelo, incrementando su transparencia y fomentando la confianza en su uso.

Desde la perspectiva de la interacción con el usuario, futuras versiones del sistema podrían centrarse en mejoras en la **interfaz** y la **experiencia de uso**, integrando funcionalidades como el filtrado por síntomas, la exportación de reportes, o la conexión con bases de datos externas.

Respecto a la estrategia de recolección de datos, se podrían implementar funciones que permitieran realizar **búsquedas simultáneas por múltiples etiquetas**, lo que incrementaría la cantidad y diversidad de publicaciones analizadas en cada ejecución. Una vez resueltos los ac-

tuales problemas de autenticación de la API de Tumblr, será posible extender la captura de publicaciones a partir de comunidades específicas.

Además, se podría incorporar **imágenes** como fuente de información, habilitando la recolección selectiva de publicaciones con contenido visual, lo cual permitiría construir datasets multimodales y desarrollar modelos especializados en el análisis de imágenes relacionadas con sintomatología depresiva.

Finalmente, se propone que las publicaciones analizadas y sus respectivas predicciones sean almacenadas en una **base de datos**, lo que facilitaría la visualización de resultados mediante gráficos agregados, no solo de la última búsqueda, sino del conjunto completo de publicaciones recolectadas. Esta funcionalidad permitiría obtener métricas acumulativas sobre la presencia de síntomas a lo largo del tiempo o por comunidades específicas.

En conclusión, este trabajo constituye un punto de partida con amplio margen de mejora, brindando la posibilidad de optimizar la robustez del sistema, ampliar sus aplicaciones y avanzar en la exploración de nuevas funcionalidades.

BIBLIOGRAFÍA

- [1] World Health Organization, «Teens, screens and mental health,» 2024, [Online]. Available: <https://www.who.int/europe/news-room/25-09-2024-teens--screens-and-mental-health>.
- [2] L. Azem et al., «Social media use and depression in adolescents: A scoping review,» *Behavioral Sciences*, vol. 13, n.º 6, pág. 475, 2023, [Online]. dirección: <https://doi.org/10.3390/bs13060475>.
- [3] A. Holpuch. «'I just feel less alone': how Tumblr became a source for mental health care.» *The Guardian* [Online]. (mayo de 2016), dirección: <https://www.theguardian.com/lifeandstyle/2016/may/19/tumblr-mental-health-information-community-disorders-healthcare>.
- [4] Ayuda Tumblr. «Tumblr.» [Online]. (2025), dirección: <https://www.tumblr.com/help>.
- [5] J. Ahuja y P. A. Fichadia, «Concerns regarding the glorification of mental illness on social media,» *Cureus*, vol. 16, n.º 3, e56631, mar. de 2024, [Online]. DOI: 10.7759/cureus.56631. dirección: <https://pmc.ncbi.nlm.nih.gov/articles/PMC11032084/>.
- [6] S. J. Russell y P. Norvig, *Artificial Intelligence: A Modern Approach*, 4.ª ed. Pearson, 2020.
- [7] S. Raschka, Y. (Liu y V. Mirjalili, *Machine Learning with PyTorch and Scikit-Learn*. Packt Publishing, 2022, [Online]. Available: O'Reilly Media. dirección: https://learning.oreilly.com/library/view/machine-learning-with/9781801819312/Text/Preface.xhtml#_idParaDest-7.
- [8] I. Goodfellow, Y. Bengio y A. Courville, *Deep Learning*. MIT Press, 2016.
- [9] S. Samuri, T. Nova, B. Rahmatullah, S. Wang y Z. Al-Qaysi, «CLASSIFICATION MODEL FOR BREAST CANCER MAMMOGRAMS,» *IIUM Engineering Journal*, vol. 23, págs. 187-199, ene. de 2022. DOI: 10.31436/iiumej.v23i1.1825.
- [10] T. Hastie, R. Tibshirani y J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer, 2009.
- [11] P. Mishra. «A layman's introduction to principal components.» Accessed: 2025-09-02. (2018), dirección: <https://hackernoon.com/a-laymans-introduction-to-principal-components-2fca55c19fa0>.
- [12] O. Chapelle, B. Scholkopf y A. Zien, *Semi-Supervised Learning*. MIT Press, 2009.
- [13] R. Khandelwal. «Semi-Supervised Learning using Label Propagation.» Accessed: 2025-09-02. (ago. de 2021), dirección: <https://medium.com/geekculture/semi-supervised-learning-using-label-propagation-2f2d69b8f3ae>.
- [14] R. S. Sutton y A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd. MIT Press, 2018.

- [15] I. Sarker, «Machine Learning: Algorithms, Real-World Applications and Research Directions,» *SN Computer Science*, vol. 2, mar. de 2021. DOI: 10.1007/s42979-021-00592-x.
- [16] A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow*, 2nd. O'Reilly Media, 2019.
- [17] G. James, D. Witten, T. Hastie y R. Tibshirani, *An Introduction to Statistical Learning: with Applications in R*, 2.^a ed. New York: Springer, 2021, ISBN: 978-1-0716-1417-4. DOI: 10.1007/978-1-0716-1418-1.
- [18] B. Öztornaci, B. Ata y S. Kartal, «Analysing Household Food Consumption in Turkey Using Machine Learning Techniques,» *Agris on-line Papers in Economics and Informatics*, vol. 16, págs. 97-105, jun. de 2024. DOI: 10.7160/ao1.2024.160207.
- [19] U. Repository, *Modelos de regresión logística*, 2025. dirección: <https://openaccess.uoc.edu/server/api/core/bitstreams/e830ec11-40f0-4aec-bc45-8bc10b123f7e/content>.
- [20] V. Kanade. «What Is Logistic Regression? Equation, Assumptions, Types, and Best Practices.» Accessed: 2025-09-02. (abr. de 2022), dirección: https://www.spiceworks.com/tech/artificial-intelligence/articles/what-is-logistic-regression/#_002.
- [21] R. Brereton y G. Lloyd, «Support Vector Machines for classification and regression,» *The Analyst*, vol. 135, págs. 230-67, feb. de 2010. DOI: 10.1039/b918972f.
- [22] M. Z. Sheriff, M. Mansouri, M. N. Karim, H. Nounou y M. Nounou, «Fault detection using multiscale PCA-based moving window GLRT,» *Journal of Process Control*, vol. 54, págs. 47-64, jun. de 2017.
- [23] scikit-learn developers. «1.12.2. Multilabel classification.» Recuperado el 24 de junio de 2025. (2025).
- [24] R. Arrora, *Multi-label classification using fastai — a shallow dive into fastai data-block API*, <https://medium.com/analytics-vidhya/multi-label-classification-using-fastai-a-shallow-dive-into-fastai-data-block-api-54ea57b2c78b>.
- [25] D. Jurafsky y J. H. Martin, *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models*, 3.^a ed. Stanford University, 2025. dirección: <https://web.stanford.edu/~jurafsky/slp3/>.
- [26] M. Honnibal e I. Montani, *Natural Language Processing in Action*, 2.^a ed. Manning Publications, 2022.
- [27] I. Goodfellow, Y. Bengio y A. Courville, *Deep Learning*. MIT Press, 2016, <http://www.deeplearningbook.org>.
- [28] C. Zhang, S. Bengio, M. Hardt, B. Recht y O. Vinyals, «Understanding Deep Learning Requires Rethinking Generalization,» *arXiv preprint arXiv:1611.03530*, 2016, Disponible en: <https://arxiv.org/abs/1611.03530>.

- [29] I. Goodfellow, «Neural Network Architectures and Generative Models,» *Functionize Blog*, 2023, Disponible en: <https://www.functionize.com/blog/neural-network-architectures-and-generative-models-part1>.
- [30] M. Nielsen. «Neural Networks and Deep Learning: Chapter 1 - Using neural nets to recognize handwritten digits.» Accessed: 2025-06-24. (2019), dirección: <http://neuralnetworksanddeeplearning.com/chap1.html>.
- [31] NCBI, *Fundamentals of Artificial Neural Networks and Deep Learning*. 2021. dirección: <https://www.ncbi.nlm.nih.gov/books/NBK583971/>.
- [32] GeeksforGeeks, «Weights and Biases in Neural Networks,» *GeeksforGeeks*, 2023. dirección: <https://www.geeksforgeeks.org/deep-learning/the-role-of-weights-and-bias-in-neural-networks/>.
- [33] I. H. Sarker, «Deep learning: a comprehensive overview on techniques, taxonomy, applications and research directions,» *SN Computer Science*, vol. 2, n.º 6, pág. 420, 2021. DOI: 10.1007/s42979-021-00815-1.
- [34] S. Raschka y V. Mirjalili, *Python Machine Learning: Machine Learning and Deep Learning with Python-scikit-learn, and TensorFlow*. Birmingham, England: Packt Publishing, 2017.
- [35] A. Deshpande, *A Beginner's Guide To Understanding Convolutional Neural Networks*, CS Undergrad at UCLA (2019), 2017. dirección: <https://adeshpande3.github.io/A-Beginner%27s-Guide-To-Understanding-Convolutional-Neural-Networks/>.
- [36] P. Mukherjee, *Convolution Neural Networks vs Fully Connected Neural Networks*, Medium, DataDrivenInvestor, ene. de 2019. dirección: <https://medium.datadriveninvestor.com/convolution-neural-networks-vs-fully-connected-neural-networks-8171a6e86f15>.
- [37] J. Dancker, *A Brief Introduction to Recurrent Neural Networks*, <https://towardsdatascience.com/a-brief-introduction-to-recurrent-neural-networks-638f64a61ff4>, 2024.
- [38] Z. Cao, Y. Wang, C.-C. Chu y R. Gadh, «Scalable Distribution Systems State Estimation Using Long Short-Term Memory Networks as Surrogates,» *IEEE Access*, vol. PP, págs. 1-1, ene. de 2020. DOI: 10.1109/ACCESS.2020.2967638.
- [39] J. Jordan, *Variational Autoencoders*, Accedido: 2025-09-04, mar. de 2018. dirección: <https://www.jeremyjordan.me/variational-autoencoders/>.
- [40] M. L. Pasini, V. Gabbi, J. Yin, S. Perotto y N. Laanait, «Scalable balanced training of conditional generative adversarial neural networks on image data,» *The Journal of Supercomputing*, vol. 77, n.º 11, págs. 13 358-13 384, 2021, ISSN: 1573-0484. DOI: 10.1007/s11227-021-03808-2. dirección: <https://doi.org/10.1007/s11227-021-03808-2>.
- [41] T. Young, D. Hazarika, S. Poria y E. Cambria, «Recent trends in deep learning based natural language processing,» *IEEE Computational Intelligence Magazine*, vol. 13, n.º 3, págs. 55-75, 2018. DOI: 10.1109/MCI.2018.2840738.

- [42] C. T. Kien. «Explanation – Attention Is All You Need.» Medium blog post, 10 min read. (oct. de 2022).
- [43] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser e I. Polosukhin, *Attention Is All You Need*, 2023. arXiv: 1706.03762 [cs.CL]. dirección: <https://arxiv.org/abs/1706.03762>.
- [44] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, J. Davison, S. Shleifer, P. von Platen, C. Ma, Y. Jernite, J. Plu, C. Xu, T. Le Scao, S. Gugger, M. Drame, Q. Lhoest y A. Rush, «Transformers: State-of-the-Art Natural Language Processing,» en *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, Q. Liu y D. Schlangen, eds., Online: Association for Computational Linguistics, oct. de 2020, págs. 38-45. DOI: 10.18653/v1/2020.emnlp-demos.6. dirección: <https://aclanthology.org/2020.emnlp-demos.6/>.
- [45] K. Doshi, *Transformers Explained Visually (Part 3): Multi-head Attention, deep dive*, <https://medium.com/data-science/transformers-explained-visually-part-3-multi-head-attention-deep-dive-1c1ff1024853>, Medium, ene. de 2021.
- [46] S. Gautam, *Transformers — A New Paradigm in NLP*, Medium, Medium article, ago. de 2020. dirección: <https://sanjivgautamofficial.medium.com/transformers-a-new-paradigm-in-nlp-e40e012bfe4c>.
- [47] J. Devlin, M.-W. Chang, K. Lee y K. Toutanova, «BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,» *arXiv preprint arXiv:1810.04805*, 2019.
- [48] C. R. Wolfe, *Language Understanding with BERT*, <https://cameronrwolfe.substack.com/p/language-understanding-with-bert>, Deep (Learning) Focus - Substack, oct. de 2022.
- [49] T. Barros, C. Santos Pires y D. Nascimento, «Leveraging BERT for extractive text summarization on federal police documents,» *Knowledge and Information Systems*, vol. 65, págs. 1-31, jun. de 2023. DOI: 10.1007/s10115-023-01912-8.
- [50] J. Devlin, M.-W. Chang, K. Lee y K. Toutanova, «BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,» *arXiv preprint arXiv:1810.04805*, 2018. dirección: <https://arxiv.org/abs/1810.04805>.
- [51] V. Sanh, L. Debut, J. Chaumond y T. Wolf, «DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter,» en *Proceedings of the 5th Workshop on Energy Efficient Machine Learning and Cognitive Computing*, 2019.
- [52] ResearchGate, *Improving Crisis Events Detection Using DistilBERT with Hunger Games Search Algorithm - Scientific Figure on ResearchGate*, https://www.researchgate.net/figure/The-DistilBERT-model-architecture-and-components_fig2_358239462, Consultado el 15 de junio de 2025, 2025.
- [53] D. Shulman, *Optimization Methods in Deep Learning: A Comprehensive Overview*, feb. de 2023. DOI: 10.48550/arXiv.2302.09566.

- [54] A. Salimath, *How to use the Learning Rate Finder in TensorFlow*, Accedido: 2019-10-06, 2019. dirección: <https://medium.com/octavian-ai/how-to-use-the-learning-rate-finder-in-tensorflow-126210de9489>.
- [55] P. Thanapol, K. Lavangnananda, F. Leprévost, A. Glad, J. Schleich y P. Bouvry, «Round-Based Mechanism and Job Packing with Model-Similarity-Based Policy for Scheduling DL Training in GPU Cluster,» *Applied Sciences*, vol. 14, n.º 6, 2024, ISSN: 2076-3417. dirección: <https://www.mdpi.com/2076-3417/14/6/2349>.
- [56] C. Li, K. Liu y S. Liu, «A Survey of Loss Functions in Deep Learning,» *Mathematics*, vol. 13, n.º 15, 2025, ISSN: 2227-7390. DOI: 10.3390/math13152417. dirección: <https://www.mdpi.com/2227-7390/13/15/2417>.
- [57] A. Ng, *Machine Learning*, Coursera online course, Accedido: 2025-09-06, 2011. dirección: <https://www.coursera.org/learn/machine-learning>.
- [58] M. Satria Wibawa, I. M. May Sanjaya, K. D. P. Novianti y P. Crisnapati, «Abnormal Heart Rhythm Detection Based on Spectrogram of Heart Sound using Convolutional Neural Network,» ago. de 2018, págs. 1-4. DOI: 10.1109/CITSM.2018.8674341.
- [59] X. Ying, «An Overview of Overfitting and its Solutions,» *Journal of Physics: Conference Series*, vol. 1168, pág. 022022, feb. de 2019. DOI: 10.1088/1742-6596/1168/2/022022.
- [60] A. Zheng, *Evaluating Machine Learning Models*. Sebastopol, CA: O'Reilly Media, Inc., sep. de 2015.
- [61] B. Ghogh y M. Crowley, «The Theory Behind Overfitting, Cross Validation, Regularization, Bagging, and Boosting: Tutorial,» *arXiv preprint arXiv:1905.12787*, 2019.
- [62] R. Bhowmick, S. R. Mishra, S. Tiwary y H. Mohapatra, «Machine learning for brain-stroke prediction: comparative analysis and evaluation,» *Multimedia Tools and Applications*, vol. 84, n.º 21, págs. 24025-24057, 2025, ISSN: 1573-7721. DOI: 10.1007/s11042-024-20057-6. dirección: <https://doi.org/10.1007/s11042-024-20057-6>.
- [63] V. Cerqueira, L. Torgo e I. Mozetič, «Evaluating time series forecasting models: An empirical study on performance estimation methods,» *arXiv preprint arXiv:1905.11744*, 2019.
- [64] R. Yang, *Omphalos, Uber's parallel and language-extensible time series backtesting tool*, <https://eng.uber.com/omphalos>, 2019.
- [65] A. Botchkarev, «Performance Metrics (Error Measures) in Machine Learning Regression, Forecasting and Prognostics: Properties and Typology,» *arXiv preprint arXiv:1809.03006*, 2018.
- [66] C. Miller, «A review of model evaluation metrics for machine learning in genetics and genomics,» *Frontiers in Bioinformatics*, 2024.
- [67] J. H. Jeppesen, R. H. Jacobsen, F. Inceoglu y T. S. Toftegaard, «A cloud detection algorithm for satellite imagery based on deep learning,» *Remote Sensing of Environment*, vol. 229, págs. 247-259, 2019, ISSN: 0034-4257. DOI: <https://doi.org/10.1016/j.rse>.

- 2019.03.039. dirección: <https://www.sciencedirect.com/science/article/pii/S0034425719301294>.
- [68] H. Chen, «Novel machine learning approaches for modeling variations in semiconductor manufacturing,» Tesis doct., ene. de 2017.
- [69] J. Deuschel, A. Foltyn, K. Roscher y S. Scheele, «The Role of Uncertainty Quantification for Trustworthy AI,» en jul. de 2024, págs. 95-115, ISBN: 978-3-031-64831-1. DOI: 10.1007/978-3-031-64832-8_5.
- [70] M. T. Ribeiro, S. Singh y C. Guestrin, *"Why Should I Trust You?": Explaining the Predictions of Any Classifier*, 2016. arXiv: 1602.04938 [cs.LG]. dirección: <https://arxiv.org/abs/1602.04938>.
- [71] A. AI, *What Are Local Interpretable Model-Agnostic Explanations – or What is LIME – In Machine Learning?* <https://arize.com/glossary/local-interpretable-model-agnostic-explanations-lime/>, Accessed: 2025-09-11, n.d.
- [72] S. Lundberg y S.-I. Lee, *A Unified Approach to Interpreting Model Predictions*, 2017. arXiv: 1705.07874 [cs.AI]. dirección: <https://arxiv.org/abs/1705.07874>.
- [73] J. Ponce-Bobadilla, «Practical Guide to SHAP Analysis,» *Journal of Machine Learning Applications*, 2024. dirección: <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC11513550/>.
- [74] A. AI. «Local Interpretable Model-Agnostic Explanations (LIME).» Accedido: 2025-09-07. (2025), dirección: <https://arize.com/glossary/local-interpretable-model-agnostic-explanations-lime/>.
- [75] V. Vimbi, N. Shaffi y M. Mahmud, «Interpreting artificial intelligence models: a systematic review on the application of LIME and SHAP in Alzheimer's disease detection,» *Brain Informatics*, vol. 11, n.º 1, pág. 10, 2024, ISSN: 2198-4026. DOI: 10.1186/s40708-024-00222-1. dirección: <https://doi.org/10.1186/s40708-024-00222-1>.
- [76] Captum. «Interpreting text models: IMDB sentiment analysis.» Accedido: 2025-09-07. (2025), dirección: https://captum.ai/tutorials/IMDB_TorchText_Interpret.
- [77] X. Ma y et al., «MD-ResNet: Multimodal deep residual networks for detection of major depressive disorder,» *arXiv preprint arXiv:1811.08592*, 2019. dirección: <https://arxiv.org/abs/1811.08592>.
- [78] Y. Liu y et al., «Remote physiological signal sensing for depression detection,» *arXiv preprint arXiv:2206.04399*, 2022. dirección: <https://arxiv.org/abs/2206.04399>.
- [79] S. Shrestha y et al., «Comparing traditional and machine learning methods to identify depressive symptoms in a national survey during COVID-19,» *Journal of Affective Disorders*, 2025. DOI: 10.1016/j.jad.2025.01.022. dirección: <https://pubmed.ncbi.nlm.nih.gov/40143723>.

- [80] X. Yuan y et al., «Transformers vs LSTM for mental disorder detection in social media: a comparative study,» *arXiv preprint arXiv:2507.19511*, 2025. dirección: <https://arxiv.org/abs/2507.19511>.
- [81] S. Ji y et al., «MentalBERT and MentalRoBERTa: Pretrained language models for mental health NLP applications,» *arXiv preprint arXiv:2304.10447*, 2023. dirección: <https://ar5iv.labs.arxiv.org/html/2304.10447>.
- [82] A. Ali y et al., «Evaluation of BERT-based models for mental health detection on social media,» *AI*, vol. 4, n.º 3, pág. 157, 2023. DOI: 10.3390/ai6030157. dirección: <https://www.mdpi.com/2673-2688/6/7/157>.
- [83] Y. Zhou y et al., «Hybrid transformer and CNN models for multi-disorder mental health detection from social media,» *International Journal of Information Systems and Technology*, vol. 3, n.º 1, pág. 1494, 2024. dirección: <https://journal.50sea.com/index.php/IJIST/article/view/1494>.
- [84] L. Zhang y et al., «Artificial intelligence methods for eating disorder detection on social media: a systematic review,» *Frontiers in Psychiatry*, vol. 15, pág. 1319522, 2024. DOI: 10.3389/fpsyt.2024.1319522. dirección: <https://www.frontiersin.org/articles/10.3389/fpsyt.2024.1319522/full>.
- [85] P. Du, X. Zhu y et al., «An ensemble learning approach combining large language models and machine learning for detecting cognitive impairment in older adults,» *medRxiv*, 2024. DOI: 10.1101/2024.04.03.24305298. dirección: <https://www.medrxiv.org/content/10.1101/2024.04.03.24305298v1>.
- [86] V. Health, «What Is the DSM-5 Used For?» *Verywell Health*, 2021, Resumen sobre propósito e historia del DSM-5. dirección: <https://www.verywellhealth.com/an-overview-of-the-dsm-5-5197607>.
- [87] F. B. H. Center, *DSM-5 Criteria: Major Depressive Disorder*, Resumen clínico de síntomas del trastorno depresivo mayor, 2019. dirección: https://floridabhcenter.org/wp-content/uploads/2021/03/MDD_Adult-Guidelines-2019-2020.pdf.
- [88] InFamousCoder, *Depression: Reddit Dataset (Cleaned)*, Kaggle, Accessed: Nov. 10, 2024, 2022. dirección: <https://www.kaggle.com/datasets/infamouscoder/depression-reddit-cleaned>.
- [89] J. Morris. «Does Model Size Matter? A Comparison of BERT and DistilBERT.» *Weights & Biases* [Online]. Created on May 11, last edited on October 13. (2023), dirección: <https://wandb.ai/jack-morris/david-vs-goliath/reports/Does-Model-Size-Matter-A-Comparison-of-BERT-and-DistilBERT--VmlldzoxMDUxNzU>.
- [90] S. Godbole y S. Sarawagi, «Discriminative Methods for Multi-labeled Classification,» en *Advances in Knowledge Discovery and Data Mining*, H. Dai, R. Srikant y C. Zhang, eds., Berlin, Heidelberg: Springer Berlin Heidelberg, 2004, págs. 22-30, ISBN: 978-3-540-24775-3.